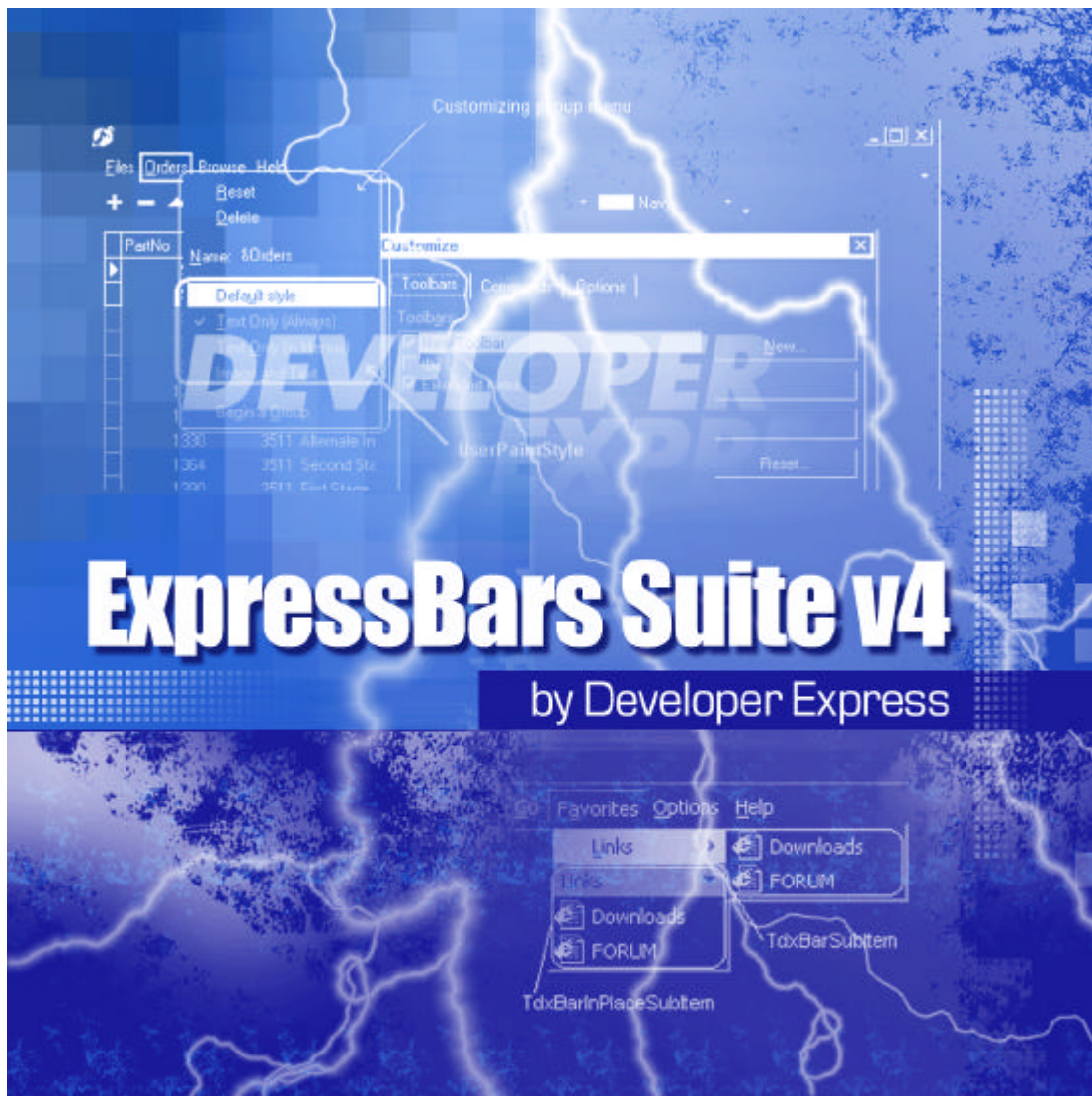




ExpressBars Suite v4

by Developer Express

General Reference

Table of Contents

ExpressBars Overview	6
TdxBarManager	9
TdxBarManager Properties	10
TdxBarManager Methods	34
TdxBarManager Events	45
TdxBarManagerList	53
TdxBarManagerList Properties	54
TdxBar	55
TdxBar Properties	56
TdxBar Methods	72
TdxBars	74
TdxBars Properties	75
TdxBars Methods	77
TdxBarItemLink	78
TdxBarItemLink Properties	79
TdxBarItemLink Methods	88
TdxBarItemLinks	90
TdxBarItemLinks Properties	91
TdxBarItemLinks Methods	96
TdxBarItemLinks Events	99
TdxBarGroup	100
TdxBarGroup Properties	102
TdxBarGroup Methods	105
TdxDockControl	107
TdxDockControl Properties	108

TdxDockControl Methods	112
TdxDockCol	113
TdxDockCol Properties	114
TdxDockCol Methods	115
TdxDockRow	116
TdxDockRow Properties	117
TdxDockRow Methods	120
TdxBarDockControl	121
TdxBarDockControl Properties	122
TdxBarDockControl Methods	125
TdxBarControl	126
TdxBarControl Properties	127
TdxBarControl Methods	129
TCustomdxBarControl	132
TCustomdxBarControl Properties	133
TCustomdxBarControl Methods	138
TdxBarSubMenuControl	145
TdxBarSubMenuControl Properties	146
TdxBarSubMenuControl Methods	148
TdxBarSubMenuControl Events	151
TdxBarDBNavigator	152
TdxBarDBNavigator Properties	153
TdxBarDBNavigator Methods	156
TdxBarDBNavigatorList	157
TdxBarDBNavigatorList Properties	158
TdxBarDBNavigatorList Methods	159

TdxBarDBNavButton	160
TdxBarDBNavButton Properties	161
TdxBarDBNavButton Methods	163
TdxBarDBNavButton Events	165
TdxBarPopupMenu	166
TdxBarPopupMenu Properties	167
TdxBarPopupMenu Methods	169
TdxBarPopupMenu Events	171
TdxBarPopupMenuLink	173
TdxBarPopupMenuLink Properties	174
TdxBarPopupMenuLink Methods	175
TdxBarPopupMenuLink Events	176
TdxBarPopupMenuLinks	177
TdxBarPopupMenuLinks Properties	178
TdxBarPopupMenuLinks Methods	179
Additional Objects	180
TdxBarConverter	180
TdxBarConverter Properties	181
TdxBarConverter Methods	182
TdxBarItemActionLink	183
TdxBarItemActionLink Methods	184
TdxBarDateNavigator	189
TdxBarDBCheckLink	190
TdxBarDBCheckLink Properties	191
TdxBarDBCheckLinks	192
TdxBarDBCheckLinks Properties	193
TdxBarDBCheckLinks Methods	194

TdxBarTreeView	195
TdxBarLookupLink	196
TdxBarPopupLookupControl	197
Global Types, Procedures and Variables	198
Index	205

ExpressBars Overview

The ExpressBars Suite of components allows visual development of menus, toolbars, and popup menus. It includes a series of elements with which to create complex toolbar and menu systems.

All ExpressBars objects are divided into two distinct groups: visual and non-visual. The non-visual group defines the style and properties of the object. The visual group of elements represents objects on screen based on information obtained from non-visual elements. Elements in the visual and non-visual groups work together to represent the object to a user.

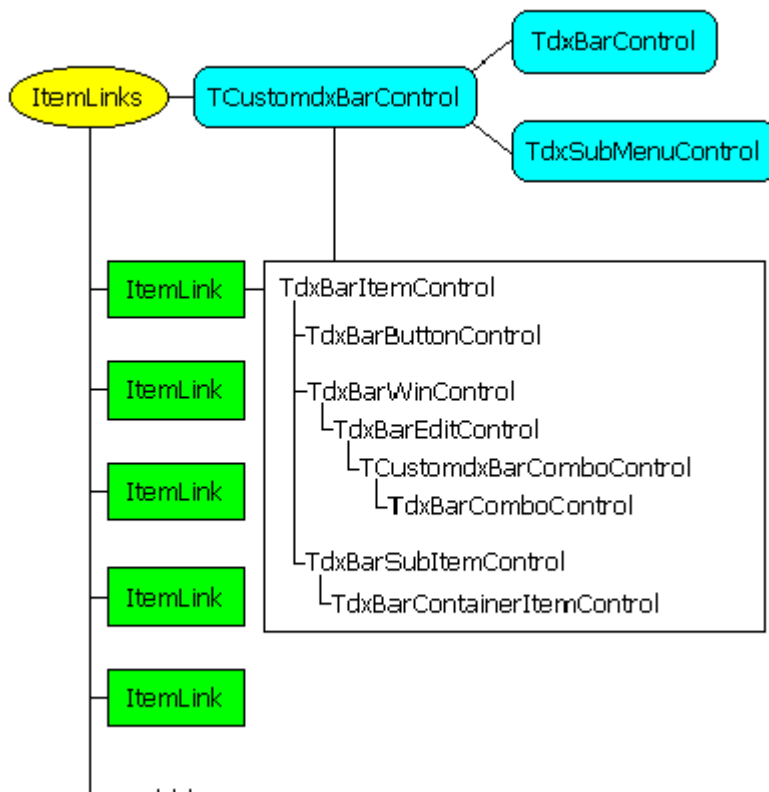
All objects you see within the customizing form list (buttons, combo boxes, edit controls, etc.), describe non-visual elements called *items*. *Items* are non-visual elements that contain methods and properties of the object and process specific object events. *Item* defines an object's response to mouse clicks or key press events. Properties displayed within the Object Inspector are *item* properties.

An *item* appears on screen as a control called an *item control*. When you drop an *item* onto a toolbar, an *item control* is created on the toolbar. In other words, *item control* represents the *item* on the *bar controls* (the phrase '*bar controls*' refers to all controls that look like a toolbar, e.g. submenus). *Item control* defines only the look of an *item* on screen, but all processing is defined by the *item*.

Every toolbar on screen, like other objects, is based on a non-visual bar object. This non-visual bar object includes references to all elements that appear on the toolbar. When you drop an object onto a toolbar, an *item control* is created on this toolbar and a link is established between the *item* and the *item control*. The non-visual bar element refers to the *item* via a link called *item link*. The *item link* also defines the look of the *item control* on the toolbar. It contains information about the *item*, the bar containing this *item link*, and the *item control* itself.

One *item* may correspond to several *item controls* on different toolbars. For example, you can create one 'Open file' button and place it on different toolbars. If you want to place the same *item* on different toolbars and these corresponding *item controls* must look different, use an *item link* to customize the look of the particular *item control*.

The following figure explains the role of *item link*:



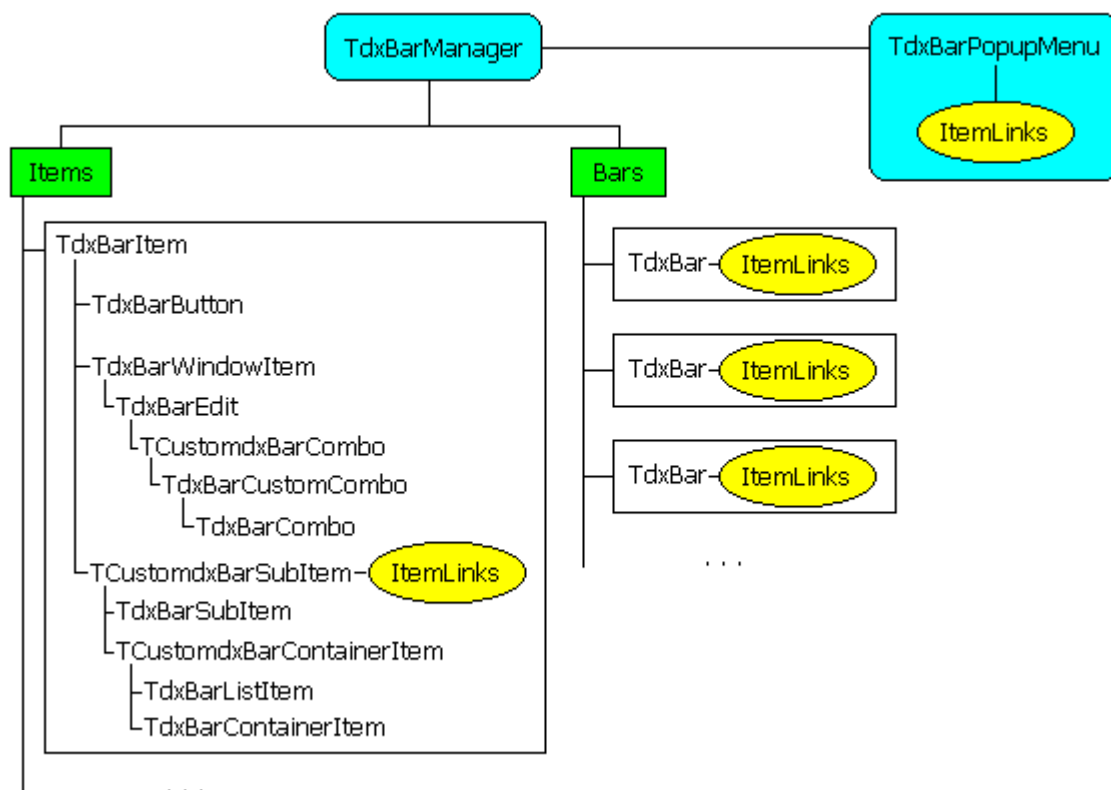
Use the customizing popup menu to customize the appearance of a particular *item control* on screen. To call the customizing popup menu, click the right mouse button on an *item control*.

Note: Some items may be presented on the toolbar by different item controls, that is, the way an item appears on screen depends on information from the item link, the bar control type and bar control location.

Items are collected into the groups. Item groups are called the *categories*. Grouping items into the *categories* allows you to easily configure item properties and to make item behavior management more convenient.

The TdxBarManager object is a form component that manages *items* and toolbars. A 'central storage' contains information about all available *items* and all existing toolbars. Every form that uses the ExpressBar toolbars must include TdxBarManager. A given form may contain only a single TdxBarManager, but within an application each form may include its own TdxBarManager. The list of TdxBarManager objects is stored within the TdxBarManagerList object.

The figure below describes the relationship between non-visual form objects:



The ExpressBars allows you to use external dock controls. The external dock control is an addition to internal default dock controls of a bar manager. You can place the external dock control to any position on a form or panel and then embed a toolbar within it. Visually this control appears as a rectangle on the top/bottom/right/left side of the form or floats over it.

TdxBarManager

TdxBarManager is the main component which encapsulates information on all toolbars and items.

Unit

dxBar

Description

TdxBarManager is a non-visual component that allows users to create, delete and modify basic components such as:

Toolbars

Categories

Items

It allows you to modify the default behavior of the interface including elements such as:

Menu Animation Effects

Button-Glyph Scaling

Font Manipulation

Hierarchy

TObject

TPersistent

TComponent

TdxBarManager Properties

AllowCallFromAnotherForm	published / read / write	IsCustomizing	public / read only
AllowReset	published / read / write	ItemCount	public / read only
AlwaysMerge	published / read / write	Items	public / read only
AlwaysSaveText	published / read / write	LargeButtonArrowWidth	published / read / write
AutoDockColor	published / read / write	LargeIcons	published / read / write
AutoHideEmptyBars	published / read / write	LargeImages	published / read / write
BarControlFocused	protected / read only	LockUpdate	public / read / write
Bars	published / read / write	MainForm	public / read only
BeforeFingersSize	protected / read only	MainMenuBar	public / read only
ButtonArrowWidth	published / read / write	MainMenuControl	public / read only
ButtonHeight	protected / read only	MenuAnimations	published / read / write
ButtonWidth	protected / read only	MenusShowRecentItemsFirst	published / read / write
CanCustomize	published / read / write	Modified	public / read / write
Categories	published / read / write	MostRecentItemsPercents	published / read / write
CategoryItemsVisible	public / read / write	NotDocking	published / read / write
CategoryVisible	public / read / write	PopupMenuLinks	published / read / write
Designing	public / read only	RealButtonArrowWidth	protected / read only
DockColor	published / read / write	RegistryPath	published / read / write
DockControlCount	public / read only	Scaled	published / read / write
DockControls	public / read only	SelectedItem	public / read / write
Dragging	protected / read only	ShowCloseButton	published / read / write
DraggingItem	protected / read only	ShowFullMenusAfterDelay	published / read / write
DraggingItemLink	protected / read only	ShowHelpButton	published / read / write
FingersSize	protected / read only	ShowHint	published / read / write
FlatCloseButton	published / read / write	Show HintForDisabledItems	published / read / write
FocusedBarControl	protected / read only	ShowShortcutInHint	published / read / write
Font	published / read / write	StoreInIniFile	published / read / write
GlyphSize	protected / read only	StoreInRegistry	published / read / write
GroupCount	public / read only	StretchGlyphs	published / read / write
Groups	public / read only	Style	published / read / write
HelpButtonGlyph	published / read / write	SubMenuBeginGroupIndent	protected / read only
HelpContext	published / read / write	SunkenBorder	published / read / write
HotImages	published / read / write	UseF10ForMenu	published / read / write
ImageListBkColor	published / read / write	UseFullReset	published / read / write
Images	published / read / write	UseSystemFont	published / read / write
IniFileName	published / read / write	WaitForDockingTime	published / read / write

AllowCallFromAnotherForm property**read / write**

Enables a user to activate items of the main application form when another application form is focused.

property AllowCallFromAnotherForm: Boolean;

Description

AllowCallFromAnotherForm allows you to activate items of the main application form when a different form of the same application has focus. Said differently, the bar manager of the main form tracks short-cut press events in different forms of the current application.

The bar manager of the main form processes shortcut keys executed from another form if:

- A form is a MDI child form and incorporates either a main menu or a bar manager.

or

- A form is neither the MDI Parent nor MDI Child form and does not contain a bar manager.

Use the AllowCallFromAnotherForm property when developing MDI applications to avoid using duplicate code and centralized shortcut processing.

AllowReset property**read / write**

Allows toolbars be restored.

property AllowReset: Boolean;

Description

If AllowReset is set to True (the default), a user can restore original settings for a selected toolbar by clicking the 'Reset...' button on the toolbar customizing form. Set AllowReset to False to prevent such behavior and to hide the 'Reset...' button.

AlwaysMerge property**read / write**

Specifies a merge mechanism for main menus of parent and child windows within an MDI application.

property AlwaysMerge: Boolean;

Description

If True, the main menu of the active child window always merges with the main menu of the parent. Otherwise, merging occurs only when a child window is maximized.

AlwaysSaveText property**read / write**

Determines whether items automatically save edited values on exit.

property AlwaysSaveText: Boolean;

Description

If the AlwaysSaveText property is True, an item control automatically saves its edited value upon losing focus. The value is always saved and is not dependant on how a user exits the item control.

***Note:** Pressing ESC does not save the edited value.*

If AlwaysSaveText is False, press ENTER (or TAB/Shift+TAB if a toolbar contains other controls that can accept focus) to save the edited value.

AutoDockColor property**read / write**

Determines whether a dock control has a specific color.

property AutoDockColor: Boolean;

Description

If AutoDockColor is set to True, the color of a dock control is equivalent to the background color of the form. Otherwise, the color is taken from the TdxBarManager.DockColor property.

The default value is False.

See Also

TdxBarManager.DockColor

AutoHideEmptyBars property**read / write**

This property controls the appearance of toolbars.

property AutoHideEmptyBars: Boolean;

Description

If True, and a toolbar does not contain any visible item controls, the toolbar is hidden at run time, and if False, the toolbar appears empty. This property works only with the CategoryItemsVisible property.

This property setting affects only situations when items become hidden through changing the value of the **CategoryItemsVisible** property. For instance, the following code will hide a toolbar if this operation hides all its items:

```
dxBarManager1.CategoryItemsVisible[0] := ivNever;
```

See Also*TdxBarManager.CategoryItemsVisible***BarControlFocused property****read only**

Determines if the TdxBarManager has a focused bar control.

property BarControlFocused: Boolean;

Description

If any toolbar has focus, the BarControlFocus property is set to True; Else it is False.

See Also*TdxBarManager.FocusedBarControl***BeforeFingersSize property****read only**

property BeforeFingersSize: Integer;

Description

This property is used internally. It determines the distance from the toolbar to its 'vertical lines' (Lines at the left edge or at the top of a toolbar).

ButtonArrowWidth property**read / write**

property ButtonArrowWidth: Integer;

Description

Determines the width of the quick customization button. The quick customization button is drawn to the left of toolbar buttons and looks similar to a dropdown button.



quick customizing button

ButtonHeight property**read only**

property ButtonHeight: Integer;

Description

Used internally to specify the proper value of a button's height. The height of a toolbar button depends on its glyph size.

ButtonWidth property**read only**

property ButtonWidth: Integer;

Description

Used internally to specify the proper value of a button's width. The button width of a toolbar depends on its glyph size.

CanCustomize property**read / write**

Specifies customizing capability.

property CanCustomize: Boolean;

Description

If True, then a customizing form may be invoked from the context popup menu; Else, the button that invokes customizing is disabled.

Categories property**read / write**

Determines a list of TdxBarManager categories.

property Categories: TStrings;

Description

Contains a list of categories into which items are divided. The category number is stored in the Category property of each item. Use categories to centralize access to items.

See Also

TdxBarItem.Category

CategoryItemsVisible property**read / write**

Specifies the visibility of category items.

type

```
TdxBarItemVisible = (ivNever, ivInCustomizing, ivAlways);
```

```
property CategoryItemsVisible [AIndex: Integer]: TdxBarItemVisible;
```

Description

AIndex specifies the category index in the TdxBarManager category list. This property defines the appearance style of category items:

Value	Meaning
ivAlways	Items are visible at design and run time.
ivNever	Items do not appear at runtime.
ivInCustomizing	Items only appear during toolbar customization.

See Also

TdxBarItem.Visible; TdxBarManager.CategoryVisible

CategoryVisible property**read / write**

Determines visibility of a Category.

```
property CategoryVisible[AIndex: Integer]: Boolean;
```

Description

This property determines visibility of a category within the customizing form at run-time. If True, the corresponding category is visible in the category listbox of the customizing form.

At design-time, all categories are visible regardless of this property value. AIndex is a category index in the TdxBarManager category list (index of the first category is 0).

See Also

TdxBarManager.CategoryItemsVisible

Designing property**read only**

Indicates whether the bar manager is in design mode.

```
property Designing: Boolean;
```

Description

The Designing property contains a Boolean value that corresponds to the bar manager mode. If this property is **True**, the bar manager is in design time mode. If this property is **False**, the bar manager is in runtime mode.

DockColor property**read / write**

Specifies the background color of a dock control.

property DockColor: TColor;

Description

The DockColor property defines the background color of a dock control if the TdxBarManager.AutoDockColor property is False.

See Also

TdxBarManager.AutoDockColor

DockControlCount property**read only**

Returns the number of dock controls that belong to the current bar manager.

property DockControlCount: Integer;

Description

Use the DockControlCount property to obtain the number of accessible dock controls via the TdxBarManager.DockControls property.

See Also

TdxBarManager.DockControls

DockControls property**read only**

Contains a list of available dock controls.

property DockControls[Index: Integer]: TdxDockControl;

Description

Dock controls are used to display toolbars. The DockControls property returns the dock control specified by the Index parameter. The first dock control index is 0. The number of dock controls is stored in the DockControlCount property.

See Also

TdxBarManager.DockControlCount

Dragging property

read only

property Dragging: Boolean;

Description

This property is used internally. If True, one TdxBarManager item is being dragged.

See Also

TdxBarManager.DraggingItem; TdxBarManager.DraggingItemLink

DraggingItem property

read only

property DraggingItem: TdxBarItem;

Description

This property is used internally. It specifies the item whose item control is being dragged.

See Also

TdxBarManager.Dragging; TdxBarManager.DraggingItemLink

DraggingItemLink property

read only

property DraggingItemLink: TdxBarItemLink;

Description

This property is used internally. It specifies the item link of the item control being dragged.

See Also

TdxBarManager.Dragging; TdxBarManager.DraggingItem

FingersSize property

read only

property FingersSize: Integer;

Description

This property is used internally to specify the size of 'vertical lines' ('Lines' are the figures at the left side or at the top of a toolbar). The FingersSize includes the BeforeFingersSize value and the size of the fingers.

FlatCloseButton property**read / write**

Determines the border style of the Close button.

property FlatCloseButton: Boolean;

Description

If FlatCloseButton is True, the Close button is drawn flat. Otherwise, it has a 3D look.

FocusedBarControl property**read only**

Specifies the focused bar control.

property FocusedBarControl: TCustomdxBarControl;

Description

Returns the currently focused bar control (if it exists). If a focused bar control does not exist, returns nil.

See Also

TdxBarManager.BarControlFocused

Font property**read / write**

Specifies item font.

property Font: TFont;

Description

Determines the font used for all items which belong to a given TdxBarManager.

Note: If this property is modified, the UseSystemFont property is False.

See Also

TdxBar.Font; TdxBar.UseOwnFont; TdxBarManager.Scaled; TdxBarManager.UseSystemFont

GlyphSize property**read only**

property GlyphSize: Integer;

Description

Used internally to set the size of a button glyph to a proper value. The GlyphSize depends on the LargeIcons property of the TdxBarManager object.

See Also

TdxBarManager.LargeIcons

GroupCount property**read only**

Returns the number of groups that belongs to the current bar manager.

property GroupCount: Integer;

Description

Use the GroupCount property to obtain the number of accessible groups via the TdxBarManager Groups property.

See Also

TdxBarManager.Groups

Groups property**read only**

Contains a list of available groups that belong to the current bar manager.

property Groups[Index: Integer]: TdxBarGroup;

Description

Groups can contain other groups and items. These elements are designed to help users manage groups of items (enable/disable or hide them).

The Groups property returns the appropriate group specified by the Index parameter which contains different items. The first group index is 0. The number of groups is stored in the GroupCount property.

See Also

TdxBarManager.GroupCount

HelpButtonGlyph property**read / write**

Specifies a bitmap, which appears on the Help button in the customizing form.

property HelpButtonGlyph: TBitmap;

Description

Set HelpButtonGlyph to a bitmap object, which contains the image, that must appear on the face of the button in the left-bottom corner of the toolbar customization form. For more detailed information see the TSpeedButton.Glyph property.

See Also

TdxBarManager.ShowHelpButton; TdxBarManager.OnHelpButtonClick; TdxBarManager.HelpContext

HelpContext property**read / write**

Provides a context number for use when calling context-sensitive Help.

property HelpContext: THelpContext;

Description

Assign a value to HelpContext to determine which help screen appears when a user clicks the Help button on the toolbar customizing form to request help. The default value is 0.

See Also

*TdxBarManager.ShowHelpButton; TdxBarManager.OnHelpButtonClick;
TdxBarManager.HelpButtonGlyph*

HotImages property**read / write**

Contains a list of images that appear on buttons when the mouse passes over them at runtime.

property HotImages: TImageList;

Description

The ImageIndex property of each button determines the specific image displayed when the mouse points to the button. If HotImages is unassigned, toolbar buttons retain their regular images (specified by the Images property) when pointed to.

See Also

TdxBarLargeButton; TdxBarManager.Images; TdxBarManager.LargeImages

ImageListBkColor property**read / write**

property ImageListBkColor: TColor;

Description

Determines the transparent color of pictures within the TdxBarManager Image list.

See Also

TdxBarManager.HotImages; TdxBarManager.Images; TdxBarManager.LargeImages

Images property**read / write**

Determines which image list is associated with items.

property Images: TImageList;

Description

Use Images to provide a customized list of bitmaps that may be displayed as item control images.

See Also

TdxBarItem.ImageIndex; TdxBarManager.HotImages; TdxBarManager.LargeImages

IniFileName property**read / write**

Specifies the name of the INI file in which to store TdxBarManager settings.

property IniFileName: string;

Description

If the TdxBarManager StoreIniFile property is True, then the TdxBarManager properties are stored in an INI file. The IniFileName property specifies the name of this INI file. The IniFileName property determines the file name; that is, the 'ini' extension isn't added by the component to the file name specified by the IniFileName property.

Note: You should call the SaveToIniFile from an OnClose event handler to save settings of a child form TdxBarManager in MDI applications.

See Also

TdxBarManager.LoadFromIniFile; TdxBarManager.SaveToIniFile; TdxBarManager.StoreIniFile

IsCustomizing property**read only**

Returns the status of the customizing form.

property IsCustomizing: Boolean;

Description

Returns True, if the TdxBarManager customizing form is active. Otherwise, returns False.

See Also

TdxBarManager.Modified; TdxBarManager.OnShowCustomizingForm

ItemCount property**read only**

Returns the number of accessible items.

property ItemCount: Integer;

Description

Returns the number of accessible items via the TdxBarManager Items property.

See Also

TdxBarManager.Items

Items property**read only**

Contains a list of available items.

property Items[Index: Integer]: TdxBarItem;

Description

Items are the elements you wish to show on a toolbar. This property returns the item specified by Index. The first item index is 0. The number of items is stored in the ItemCount property.

See Also

TdxBarManager.ItemCount; TdxBarItem

LargeButtonArrowWidth property**read / write**

property LargeButtonArrowWidth: Integer;

Description

This property determines the size of dropdown buttons for large buttons. A dropdown button is drawn to the left of a large button and looks similar to a combobox dropdown button.

LargeIcons**read / write**

Specifies the draw style of toolbar buttons.

property LargeIcons: Boolean;

Description

If LargeIcons is True, toolbars buttons are enlarged.

LargeImages property**read / write**

Specifies a list of images to draw for item controls.

property LargeImages: TImageList;

Description

Determines which image list is used to draw large images for item controls.

See Also

TdxBarLargeButton; TdxBarManager.HotImages

LockUpdate property**read / write**

Sets the repaint mode for item controls that belong to toolbars.

property LockUpdate: Boolean;

Description

Setting a value for this property assigns the same value to the LockUpdate property of all toolbars belonging to a given TdxBarManager. If True, any modifications to toolbar item controls of a given TdxBarManager does not invoke screen repainting. If False, toolbars of a given TdxBarManager, whose LockUpdate property is True, are refreshed.

This property is used to prevent numerous toolbar refresh events during modification of a group of item controls (e.g. the Enabled property of a given set of items).

See Also

TdxBar.LockUpdate

MainForm property**read only**

Specifies the parent form of a TdxBarManager object.

property MainForm: TCustomForm;

Description

Returns the form which contains a given TdxBarManager.

Important: A form can contain only one TdxBarManager.

MainMenuBar property**read only**

Returns a main menu component.

property MainMenuBar: TdxBar;

Description

Returns the TdxBar object presented as the main menu, that is, its IsMainMenu property is True.

Note: *There may be only one control of this type in TdxBarManager.*

See Also

TdxBar.IsMainMenu; TdxBarManager.MainMenuControl

MainMenuControl property**read only**

Returns a main menu control pointer.

property MainMenuControl: TdxBarControl;

Description

Returns the bar control presented as a main menu. If true, the IsMainMenu property of this bar control is True.

Note: *There may be only one control of this type in TdxBarManager.*

See Also

TdxBarManager.MainMenuBar

MenuAnimations property**read / write**

Specifies menu animation effects.

type

TdxBarMenuAnimations = (maNone, maRandom, maUnfold, maSlide);

property MenuAnimations: TdxBarMenuAnimations;

Description

Determine the style of menu activation. The following menu animation effects are available:

Value	Meaning
maNone	No animation.
maRandom	Random choice (maUnfold or maSlide).
maUnfold	Unfolding menu.
maSlide	Sliding dropdown or popup menu.

MenusShowRecentItemsFirst property**read / write**

Determines whether the most frequently used items will be placed in menus at first position.

property MenusShowRecentItemsFirst: Boolean;

Description

If MenusShowRecentItemsFirst is True (by default), when a user clicks menu items, they are “promoted” and displayed higher on the list. Unused menu items are visually suppressed and not displayed on the menu. Else, all menu items are displayed in the order of creation.

Note: *MenusShowRecentItemsFirst is effective only if the Style property is equal to bmsEnhanced or bmsFlat.*

See Also

*TdxBarItemLink.BringToTopInRecentList; TdxBarItemLink.SendToBottomInRecentList;
TdxBarManager.MostRecentItemsPercents; TdxBarManager.ShowFullMenusAfterDelay*

Modified property**read / write**

Determines whether a user has made changes to the layout of a toolbar and item controls.

property Modified: Boolean;

Description

Use this property if you save toolbar settings manually. If Modified is True, a user changed the layout of the toolbar and item controls and therefore your application must save these changes to the Registry or INI file.

Note: *Be sure to set Modified to False after saving changes.*

MostRecentItemsPercents property**read / write**

Specifies the time bounds in percentage for calculating the most frequently used items.

property MostRecentItemsPercent: Byte;

Description

If the quantity of press events of an item control is less than the value of MostRecentItemsPercents, then this item control is added to the list of the infrequently used items. The default value is 95.

Note: *MenusShowRecentItemsFirst is effective only if the Style property is equal to bmsEnhanced.*

See Also

*TdxBarItemLink.BringToTopInRecentList; TdxBarItemLink.SendToBottomInRecentList;
TdxBarManager.MenusShowRecentItemsFirst; TdxBarManager.ShowFullMenusAfterDelay*

NotDocking property**read / write**

Specifies toolbar location restrictions.

type

```
TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);
```

```
TdxBarDockingStyles = set of TdxBarDockingStyle;
```

```
property NotDocking: TdxBarDockingStyles;
```

Description

Specifies application form sides to which all toolbars cannot be docked.

Note: *If dsNone is included, the toolbars not be float.*

See Also

TdxBar.NotDocking

PopupMenuLinks property**read / write**

Contains an array of menu links.

```
property PopupMenuLinks: TdxBarPopupMenuLinks;
```

Description

Any control on a form can be tied to a standard TPopupMenu. This popup menu is activated when a user right-clicks a control, the context menu, or Shift+F10 keys. ExpressBars v. 3.1 enables a user to associate controls with popup menus of type TdxBarPopupMenu. The PopupMenuLinks property is presented as an array if menu links are used by a bar manager to tie a control to popup menus.

See Also

TdxBarPopupMenu; TdxBarPopupMenuLink

RealButtonArrowWidth property**read only**

```
property RealButtonArrowWidth: Integer;
```

Description

Used internally to specify the proper width of an arrow button. The Arrow button is similar to a combo box button. Arrow button width depends on a bar manager's LargeIcons property.

See Also

TdxBarManager.LargeIcons

RegistryPath property**read / write**

Specifies the Registry path to store a component's settings.

property RegistryPath: string;

Description

The Registry key specified by the RegistryPath property is used to store TdxBarManager toolbar configurations, if and when StoreInRegistry is True.

See Also

TdxBarManager.LoadFromRegistry; TdxBarManager.SaveToRegistry;
TdxBarManager.StoreInRegistry

Scaled property**read / write**

Specifies whether a toolbar is sized according to a specified font.

property Scaled: Boolean;

Description

Scaled is used to change toolbar size according to a used font. It works only when UseSystemFont is False.

SelectedItem property**read / write**

Specifies the selected toolbar item control.

property SelectedItem: TdxBarItemControl;

Description

This property returns the currently selected item control. A toolbar can contain several item controls corresponding to different items. If an item control is selected, its underlying item is selected as well. The item can be selected on one of the toolbars or submenus or on the customizing form.

Important: Do not assign any value to this property.

ShowCloseButton property**read / write**

Specifies whether the toolbar Close button is displayed.

property ShowCloseButton: Boolean;

Description

If the ShowCloseButton property is True, the main menu includes a Close button. Once clicked, the OnCloseButtonClick event is generated. Use the Close button to close a child form if it has no title.

Note: For docked toolbars, the Close button is available only if the toolbar's IsMainMenu property is set to True.

See Also

TdxBar.IsMainMenu

ShowFullMenusAfterDelay property**read / write**

Determines whether hidden items are automatically displayed in a menu.

property ShowFullMenusAfterDelay: Boolean;

Description

If ShowFullMenusAfterDelay is True (by default), menu items that are infrequently used will be automatically displayed after a delay. Else, the menu expands only after clicking its bottom-most button.

Note: *MenusShowRecentItemsFirst* is effective only if the *Style* property is equal to *bmsEnhanced*.

See Also

TdxBarManager.MenusShowRecentItemsFirst; *TdxBarManager.MostRecentItemsPercents*

ShowHelpButton property**read / write**

Specifies whether the Help button is displayed in the customizing form.

property ShowHelpButton: Boolean;

Description

If ShowHelpButton is set to True, the Help button is displayed in the left-bottom corner of the toolbar customization form. Once clicked, the OnHelpButtonClick event is generated. The default value is False.

See Also

TdxBarManager.HelpButtonGlyph; *TdxBarManager.HelpContext*; *TdxBarManager.OnHelpButtonClick*

ShowHint property**read / write**

Determines whether the help Hint is shown when the mouse pointer rests over item controls.

property ShowHint: Boolean;

Description

If the ShowHint is True, an item hint is displayed.

See Also

TdxBarItem.Hint; TdxBarManager.ShowShortCutInHint; TdxBarManager.ShowHintForDisabledItems

ShowHintForDisabledItems property**read / write**

Specifies whether to show a hint for disabled items.

property ShowHintForDisabledItems: Boolean;

Description

If the ShowHintForDisabledItems property is True, a hint is shown for every item control on the toolbars, including disabled item controls. A hint for disabled item controls will be shown if the TdxBarManager ShowHint property is True.

See Also

TdxBarItem.Hint; TdxBarManager.ShowHint; TdxBarManager.ShowShortCutInHint

ShowShortCutInHint property**read / write**

Determines if the control shortcut is displayed with the item controls Hint.

property ShowShortCutInHint: Boolean;

Description

If the ShowShortCutInHint property is True, the shortcut of a given element (if there is any) is displayed along with the hint of the element.

See Also

TdxBarItem.Hint; TdxBarManager.ShowHint; TdxBarManager.ShowHintForDisabledItems

StoreInIniFile property**read / write**

Specifies whether to store the TdxBarManager settings in an INI file.

property StoreInIniFile : Boolean;

Description

If StoreInIniFile is True, then the TdxBarManager object stores the TdxBarManager settings in an INI file specified by the IniFileName property. The TdxBarManager stores TdxBarManager settings,

toolbars settings that belong to this TdxBarManager, settings of the toolbar item links, and settings of all sub items.

Note: You should call the *SaveToIniFile* from an *OnClose* event handler to save settings of a child form *TdxBarManager* in MDI applications.

See Also

TdxBarManager.IniFileName; TdxBarManager.LoadFromIniFile; TdxBarManager.SaveToIniFile; TdxBarManager.StoreInRegistry

StoreInRegistry property

read / write

Determines whether a component's settings are stored in the Registry.

property StoreInRegistry: Boolean;

Description

If True, toolbar properties are stored in the Registry specified by the RegistryPath property.

Note: Within MDI applications, you should call the *SaveToRegistry* in the *OnClose* event handler of the child form to save the settings of its *TdxBarManager*.

See Also

TdxBarManager.RegistryPath; TdxBarManager.LoadFromRegistry; TdxBarManager.SaveToRegistry; TdxBarManager.StoreInIniFile

StretchGlyphs property

read / write

Indicates whether a glyph should be altered so that it fits exactly in the bounds of the item control.

property StretchGlyphs: Boolean;

Description

Set StretchGlyphs to True to force button images to assume the size and shape of item controls. When a control resizes, the image resizes as well. All new items will have the same StretchGlyphs property as the TdxBarManager to which they belong.

Style property

[read / write](#)

Specifies toolbar draw style.

type

```
TdxBarManagerStyle = (bmsStandard, bmsEnhanced);
```

property Style: TdxBarManagerStyle;

Description

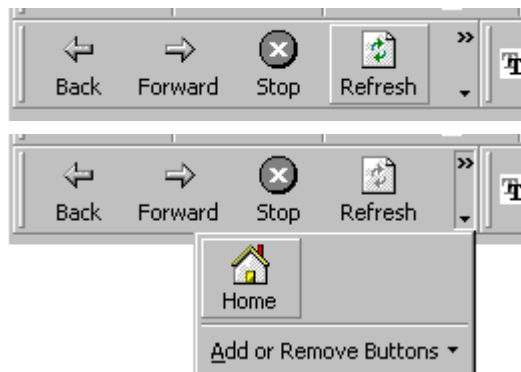
Changes to this property effects toolbar and menu paint styles. If drawing style is bmsStandard, then two 'vertical lines' appear on the left side of a toolbar.

bmsStandard



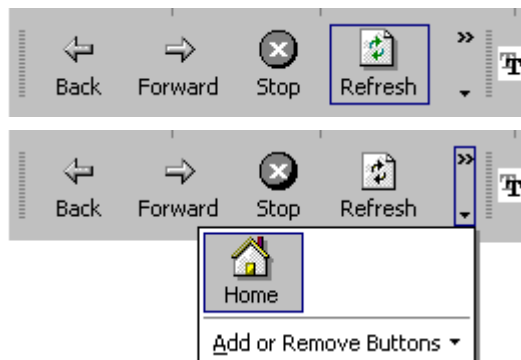
If the Style property is set to bmsEnhanced, then only one 'vertical line' appears (the 'lines' appear only if the DockingStyle of the bar controls is dsTop or dsBottom) and separator lines in menus are narrower than standard menu separators.

bmsEnhanced



If the Style property is set to bmsFlat, then toolbars and items have MS Office 10 look and feel.

bmsFlat



See Also

TdxBar.DockingStyle; TdxBarManager.SunkenBorder

SubMenuBeginGroupIndent property**read only**

property SubMenuBeginGroupIndent: Integer;

Description

Used internally to correctly draw separators in a sub menu. Separator length depends on the bar manager Style property.

SunkenBorder**read / write**

Determines a toolbar's border style.

property SunkenBorder: Boolean;

Description

Use SunkenBorder to switch between toolbar border styles. If the SunkenBorder property is False, ExpressBars creates a toolbar with standard borders. Otherwise, border lines are sunken.

UseF10ForMenu property**read / write**

Determines whether TdxBarManager responds to the F10 key press event.

property UseF10ForMenu: Boolean;

Description

If UseF10ForMenu is set to False, TdxBarManager does not respond to the F10 key press events and the menu will not be called. The default value is True.

UseFullReset property**read / write**

Determines whether a toolbar (not just item links) is updated on calls to the Reset method.

property UseFullReset: Boolean;

Description

If UseFullReset is True, then the Reset method of a TdxBar object restores not only the properties of the item links tied to a toolbar, but also the toolbar itself, i.e. its size, position, docking style, state, etc.

If UseFullReset is False, the Reset method of the TdxBar object only changes item links properties.

See Also

TdxBar.Reset; TdxBarManager.OnBarAfterReset; TdxBarManager.OnBarBeforeReset

UseSystemFont property**read / write**

Specifies the font used to draw the toolbar controls.

property UseSystemFont: Boolean;

Description

If the UseSystemFont property is True, the Font property is assigned to the parameter set for the menus via Windows system settings. Additionally, if UseSystemFont is True, the Scaled property does not affect toolbar appearance.

See Also

TdxBarManager.Font; TdxBarManager.Scaled

WaitForDockingTime property**read / write**

property WaitForDockingTime: Integer;

Description

Defines the timespan in milliseconds before docking a toolbar to a form.

TdxBarManager Methods

BarByCaption	public	DoShowToolbarsPopup	protected
BarByName	public	ExchangeItems	public
Create	public	GetAllItemsByCategory	public
CreateGroup	public	GetCountByCategory	public
CreateToolbarsPopupList	public	GetItemByCategory	public
Customizing	public	GetItemByName	public
Destroy	public	GetItemsByCategory	public
DoBarAfterReset	protected	HideAll	public
DoBarBeforeReset	protected	LoadFromIniFile	public
DoBarClose	protected	LoadFromRegistry	public
DoBarDockingStyleChanged	protected	MoveItem	public
DoClickItem	protected	ResetUsageData	public
DoCloseButtonClick	protected	ResetUsageDataWithConfirmation	public
DoDocking	protected	SaveToIniFile	public
DoHelpButtonClick	protected	SaveToRegistry	public
DoShowCustomizingPopup	protected		

BarByCaption method

Returns a toolbar by its caption.

```
function BarByCaption(const ACaption: string): TdxBar;
```

Description

Each bar of a bar manager has a caption which uniquely identifies it. Use the BarByCaption method to access a bar via the Caption property specified by the ACaption parameter.

See Also

TdxBarManager.BarByName

BarByName method

Returns a bar by its name.

```
function BarByName(const AName: string): TdxBar;
```

Description

Each bar of a bar manager has a name which can be used to uniquely identify it. Use the BarByName method to access a bar via the Name property specified by the AName parameter.

See Also

TdxBarManager.BarByCaption

Create method

Creates and initializes the TdxBarManager instance.

```
constructor Create(AOwner: TComponent); override;
```

Description

The Create method

- Calls the inherited Create method.
- Sets the MainForm property of the TdxBarManager to the application main form
- Registers the TdxBarManager
- Sets LargeButtonArrowWidth, ButtonArrowWidth, WaitForDockingTime to their initial values.
- Sets the Scaled, ShowHint, StretchGlyphs, UseSystemFont properties to True.
- Sets Style to the bmsEnhanced.

Note: *A form can contain only a single TdxBarManager.*

CreateGroup method

Creates and initializes an instance of the TdxBarGroup.

```
function CreateGroup: TdxBarGroup;
```

Description

The CreateGroup method

- ? Calls the Create method of the TdxBarGroup.
- ? Adds this group to the list of available groups of the current bar manager.

See Also

TdxBarManager.GroupCount;TdxBarManager.Groups

CreateToolbarsPopupList method

Creates a list of links where each link corresponds to a specific toolbar.

```
procedure CreateToolbarsPopupList(ItemLinks: TdxBarItemLinks);
```

Description

Only toolbars whose Hidden property is set to False can be added to a list of links. The generated list is used when activating a toolbar's popup menu. CreateToolbarsPopupList calls the DoShowToolbarsPopup method which generates the OnShowToolbarsPopup event. Use the OnShowToolbarPopup event handler to customize the toolbars list.

Customizing method

Shows or hides the toolbar customizing form.

```
procedure Customizing(Show: Boolean);
```

Description

Use this procedure to manage the display of the customizing form. If the Show parameter is True, the customizing form is displayed.

See Also

TdxBarManager.IsCustomizing; TdxBarManager.Modified

Destroy method

Destroys an instance of the TdxBarManager type.

```
destructor Destroy; override;
```

Description

The TdxBarManager Destroy method destroys the TdxBarManager and all items and toolbars it manages.

See Also

TdxBarManager.Create

DoBarAfterReset method

```
procedure DoBarAfterReset(ABar: TdxBar); dynamic;
```

Description

This method is called following a call to the Reset method of a toolbar specified by ABar. DoBarAfterReset generates an OnBarAfterReset event of the bar manger to which ABar belongs.

See Also

TdxBar.Reset; TdxBarManager.OnBarAfterReset; TdxBarManager.DoBarBeforeReset

DoBarBeforeReset method

```
procedure DoBarBeforeReset(ABar: TdxBar); dynamic;
```

Description

This method is called before calling the Reset method of a toolbar specified by ABar. DoBarAfterReset generates the OnBarAfterReset event of the bar manger to which ABar belongs.

See Also

TdxBar.Reset; TdxBarManager.DoBarAfterReset; TdxBarManager.OnBarBeforeReset

DoBarClose method

```
procedure DoBarClose(ABar: TdxBar); dynamic;
```

Description

This method generates an OnBarClose event when closing a toolbar, specified by ABar (the toolbar is closed when pressing the Close button in its upper rightmost corner).

See Also

TdxBarManager.OnBarClose

DoBarDockingStyleChanged

```
procedure DoBarDockingStyleChanged(ABar: TdxBar); dynamic;
```

Description

This method generates the OnBarDockingStyleChanged event when the toolbar, specified by the ABar parameter, changes its docking style.

See Also

TdxBarManager.OnBarDockingStyleChange

DoClickItem

```
procedure DoClickItem(AItem: TdxBarItem); virtual;
```

Description

Called on clicking any item control on the toolbar. This simply generates an OnClickItem event.

DoCloseButtonClick method

```
procedure DoCloseButtonClick; dynamic;
```

Description

The DoCloseButtonClick method is called when the Close button is clicked. This procedure simply generates the OnCloseButtonClick event. Override it to perform special actions without writing an event handler.

DoDocking method

Determines the docking style of a bar control.

```
procedure DoDocking(ABar: TdxBar; AStyle: TdxBarDockingStyle; var
CanDocking: Boolean);
```

Description

This procedure checks if the docking style specified by the AStyle parameter is a valid docking style for a bar control. CanDocking is set to True, if the TdxBarManager NotDocking does not include AStyle. Otherwise, CanDocking is set to False. This method is called when a bar control's location changes. This procedure also generates the OnDocking event.

See Also

TdxBar; TdxBarManager.NotDocking; TdxBarManager.OnDocking

DoHelpButtonClick method

```
procedure DoHelpButtonClick; dynamic;
```

Description

This method generates an OnHelpButtonClick event when a user presses the Help button in the customizing form.

See Also

TdxBarManager.OnHelpButtonClick; TdxBarManager.ShowHelpButton

DoShowCustomizingPopup method

```
procedure DoShowCustomizingPopup(PopupItemLinks: TdxBarItemLinks); dynamic;
```

Description

Generates an OnShowCustomizingPopup event. PopupItemLinks are item links in the customizing popup menu. Use this method to perform specific actions immediately before generating the OnShowCustomizingPopup event, for example, to add, delete or modify certain items in the customizing popup menu.

See Also

TdxBarManager.OnShowCustomizingPopup

DoShowToolbarsPopup method

procedure DoShowToolbarsPopup(PopupItemLinks: TdxBarItemLinks); **dynamic**;

Description

Generates the OnShowToolbarsPopup event.

See Also

TdxBarManager.OnShowToolbarsPopup

ExchangeItems method

Exchanges two items in an item list.

procedure ExchangeItems(Index1, Index2: Integer);

Description

Exchanges two items in a TdxBarManager item list. Index1 and Index2 specify the item indexes in the list. This method is called when you move items from one category to another in the customizing form.

See Also

dxBarCustomizingForm

GetAllItemsByCategory method

Gets all items of a specified category.

function GetAllItemsByCategory(ACategory: Integer; List: TList): Integer;

Description

Stores the items list of a category specified by ACategory in the List variable. The Function returns the number of all category items (both visible and invisible).

See Also

TdxBarManager.GetCountByCategory; TdxBarManager.GetItemByCategory;
TdxBarManager.GetItemByName; TdxBarManager.GetItemsByCategory

GetCountByCategory method

Returns the number of visible items in a specified category.

```
function GetCountByCategory(ACategory: Integer): Integer;
```

Description

The ACategory parameter specifies the number of categories which belong to TdxBarManager. This Function returns the number of visible category items.

See Also

*TdxBarManager.GetAllItemsByCategory; TdxBarManager.GetItemByCategory;
TdxBarManager.GetItemByName; TdxBarManager.GetItemsByCategory*

GetItemByCategory method

Returns the specified item of a specified category.

```
function GetItemByCategory(ACategory, AIndex: Integer): TdxBarItem;
```

Description

Use this function to get the item specified by the AIndex parameter from the items list of a category. The Category is specified by the ACategory parameter. The index of the first category is 0. The number of the categories is returned by the GetCountByCategory function.

See Also

*TdxBarManager.GetAllItemsByCategory; TdxBarManager.GetCountByCategory;
TdxBarManager.GetItemByName; TdxBarManager.GetItemsByCategory*

GetItemByName method

Returns an item by name.

```
function GetItemByName(const AName: string): TdxBarItem;
```

Description

The function looks for an item with the name, specified by the AName parameter in the TdxBarManager item list and returns it (if it exists). If matches are not found, the function returns nil.

See Also

*TdxBarManager.GetAllItemsByCategory; TdxBarManager.GetCountByCategory;
TdxBarManager.GetItemByCategory; TdxBarManager.GetItemsByCategory*

GetItemsByCategory method

Returns items by category number.

```
function GetItemsByCategory(ACategory: Integer; List: TList): Integer;
```

Description

The Function returns the list of items whose category number is equal to ACategory.

See Also

*TdxBarManager.GetAllItemsByCategory; TdxBarManager.GetCountByCategory;
TdxBarManager.GetItemByCategory; TdxBarManager.GetItemByName*

HideAll method

Hides all open toolbar popup menus or subitems.

```
procedure HideAll;
```

Description

Use this function to make the active toolbar, popup menu, or subitem inactive. For instance, if there is a set of open menus, HideAll closes them.

LoadFromIniFile method

Loads TdxBarManager settings from an INI file.

```
procedure LoadFromIniFile(AFileName: String);
```

Description

This procedure loads the TdxBarManager, toolbars, item links, and sub items settings from an INI file specified by AFileName. TdxBarManager uses this method to load settings from the INI file specified by the IniFileName property. If you want to load TdxBarManager settings from a specific INI file, call the LoadFromIniFile method with the AFileName parameter specifying the appropriate INI file.

See Also

TdxBarManager.IniFileName; TdxBarManager.SaveToIniFile; TdxBarManager.StoreInIniFile

LoadFromRegistry method

Loads component settings from the Registry.

```
procedure LoadFromRegistry(ARegistryPath: string);
```

Description

You can store component settings in the Registry. This procedure loads these settings from the Registry key specified by the ARegistryPath parameter.

See Also

TdxBarManager.RegistryPath; TdxBarManager.StoreInRegistry

MoveItem method

Moves an item inside TdxBarManager to a new position.

```
procedure MoveItem(CurIndex, NewIndex: Integer);
```

Description

Moves an item inside TdxBarManager item list from its current position specified by CurIndex to a new position specified by NewIndex. This function is called when you drag an item from one category to another.

ResetUsageData method

```
procedure ResetUsageData;
```

Description

Deletes a record of commands a user has used in an application and restores the default set of visible item controls to menus and toolbars. This action will not undo any explicit customizations.

See Also

TdxBarManager.ResetUsageDataWithConfirmation

ResetUsageDataWithConfirmation method

```
procedure ResetUsageDataWithConfirmation;
```

Description

Displays a generic dialog box with a request message. In case of a positive answer the TdxBarManager.ResetUsageData method is called to undo user customizations.

See Also

TdxBarManager.ResetUsageData

SaveToIniFile method

Saves TdxBarManager settings to an ini file.

```
procedure SaveToIniFile( AFileName : string);
```

Description

This method saves TdxBarManager settings to the INI file specified by the AFileName parameter. This procedure is called automatically on destroying the component. In this instance, the file name is specified by the IniFileName property. Call SaveToIniFile with a unique file name if you wish to save TdxBarManager settings to another file rather than that specified by the IniFileName property.

Note: You should call the SaveToIniFile from an OnClose event handler to save settings of a child form TdxBarManager in MDI applications.

```
procedure TForm1.FormClose(Sender: TObject; var Action: TCloseAction);  
begin  
    with dxbarManager1 do  
        SaveToIniFile(IniFileName);  
    Action := caFree;  
end;
```

See Also

TdxBarManager.LoadFromIniFile; TdxBarManager.IniFileName; TdxBarManager.StoreInIniFile

SaveToRegistry method

Saves component settings to the Registry.

```
procedure SaveToRegistry(ARegistryPath: string);
```

Description

If the `StoreInRegistry` property is `True`, the `SaveToRegistry` method saves the configuration of toolbars, items, and some `TdxBarManager` properties in the Registry.

Note: *In child forms of MDI applications, you must save TdxBarManager settings manually.*

Call the `SaveToRegistry` method from the `OnClose` event handler:

```
procedure TForm1.FormClose(Sender: TObject; var Action: TCloseAction);  
begin  
    with dxbarManager1 do  
        SaveToRegistry(RegistryPath);  
    Action := caFree;  
end;
```

See Also

TdxBarManager.LoadFromRegistry; TdxBarManager.RegistryPath; TdxBarManager.StoreInRegistry

TdxBarManager Events

OnBarAdd	published	OnHelpButtonClick	published
OnBarAfterReset	published	OnHideCustomizingForm	published
OnBarBeforeReset	published	OnItemLinkAdd	published
OnBarClose	published	OnItemLinkChange	published
OnBarDelete	published	OnItemLinkDelete	published
OnBarDockingStyleChange	published	OnMenuMerge	published
OnBarVisibleChange	published	OnShowCustomizingForm	published
OnClickItem	published	OnShowCustomizingPopup	published
OnCloseButtonClick	published	OnShowToolbarsPopup	published
OnDocking	published		

OnBarAdd event

This event occurs just after adding a new toolbar to the TdxBarManager.

type

```
TdxBarEvent = procedure(Sender: TdxBarManager; ABar: TdxBar) of object;  
property OnBarAdd: TdxBarEvent;
```

Description

Use this event when you need special code in response to toolbar creation. The created toolbar is specified by the ABar parameter.

See Also

TdxBarManager.OnBarDelete

OnBarAfterReset event

Occurs after calling the Reset method of a TdxBar object.

type

```
TdxBarEvent = procedure(Sender: TdxBarManager; ABar: TdxBar) of object;  
property OnBarAfterReset: TdxBarEvent;
```

Description

This event is generated after calling the Reset method of a toolbar specified by ABar. ABarManager is the bar manager to which ABar belongs.

See Also

TdxBar.Reset; TdxBarManager.OnBarBeforeReset

OnBarBeforeReset event

Occurs before calling the Reset method of a TdxBar object.

type

TdxBarEvent = **procedure**(Sender: TdxBarManager; ABar: TdxBar) **of object**;
property OnBarAfterReset: TdxBarEvent;

Description

This event is generated before calling the Reset method of a toolbar specified by ABar. ABarManager is the bar manager to which ABar belongs.

See Also

TdxBar.Reset; TdxBarManager.OnBarAfterReset

OnBarClose event

Occurs when a toolbar closes.

type

TdxBarEvent = **procedure**(Sender: TdxBarManager; ABar: TdxBar) **of object**;
property OnBarClose: TdxBarEvent;

Description

Use OnBarClose to perform special processing when a floating toolbar closes. This event specifies which event handler to call when a user clicks the toolbar's Close button.

OnBarDelete event

This event occurs before removal of a toolbar from the toolbar list of a TdxBarManager.

type

TdxBarEvent = **procedure**(Sender: TdxBarManager; ABar: TdxBar) **of object**;
property OnBarDelete: TdxBarEvent;

Description

Use this event to write special code in response to toolbar removal. The toolbar is specified by the ABar parameter.

See Also

TdxBarManager.OnBarAdd

OnBarDockingStyleChange event

This event occurs when a user changes the docking style of a toolbar.

type

```
TdxBarEvent = procedure(Sender: TdxBarManager; ABar: TdxBar) of object;  
property OnBarDockingStyleChange: TdxBarEvent;
```

Description

Use this event to implement special code on docking style changes to the ABar toolbar.

Note: *OnBarDockingStyle* is the notifying event. Use the *OnDocking* event to set a specific *DockingStyle* for a toolbar. It is not recommended to set the *DockingStyle* for a toolbar in the *OnBarDockingStyle* event handler.

See Also

TdxBarManager.NotDocking; *TdxBarManager.OnDocking*

OnBarVisibleChange event

This event occurs on toolbar visibility change.

type

```
TdxBarVisibleChangeEvent = procedure(Sender: TdxBarManager; ABar: TdxBar)  
of object;  
property OnBarVisibleChange: TdxBarVisibleChangeEvent;
```

Description

This event occurs after changing toolbar visibility. The toolbar whose Visible property changes is specified by the ABar parameter.

OnClickItem event

type

```
TdxBarClickItemEvent = procedure(Sender: TdxBarManager; ClickedItem:  
TdxBarItem) of object;  
property OnClickItem: TdxBarClickItemEvent;
```

Description

This event occurs on clicking the toolbar items, that is, when the user clicks one of the toolbar item controls.

OnCloseButtonClick event

Occurs on clicking a close button.

property OnCloseButtonClick: TNotifyEvent;

Description

Assume your child form has no title, so the child form border icons are invisible, but you wish to give users an opportunity to close the child form. Make one of the toolbars on the form the main menu toolbar and set the TdxBarManager ShowCloseButton property to True. Write an OnCloseButtonClick event handler to close the child form.

See Also

TdxBarManager.ShowCloseButton

OnDocking event

type

TdxBarCanDockingEvent = **procedure**(Sender: TdxBar; Style: TdxBarDockingStyle; **var** CanDocking: Boolean) **of object**;

property OnDocking: TdxBarCanDockingEvent;

Description

This event occurs before toolbar docking. The Sender parameter specifies the toolbar to be docked. The application form side to which a toolbar will be docked is determined by the Style parameter. If CanDocking is False, no docking occurs. Write an OnDocking event handler to prevent docking of particular toolbars to particular forms.

The following code prevents the main menu toolbar from docking to any side except a form's top:

```
procedure TForm1.dxBBarManager1Docking(Sender: TdxBar; Style:
TdxBarDockingStyle; var CanDocking: Boolean);
begin
    if Sender = GetBarManagerByForm(Self).MainMenuBar then
        if Style <> dsTop then
            CanDocking := False;
end;
```

See Also

TdxBarManager.NotDocking;

OnHelpButtonClick event

Occurs when clicking the Help button in the customization form.

property OnHelpButtonClick: TNotifyEvent;

Description

Use the OnHelpButtonClick event handler to write code that responds to the click on the Help button in the left-bottom corner of the toolbar customization form. By default, when a user clicks this button, the help screen appears.

See Also

TdxBarManager.HelpContext; TdxBarManager.ShowHelpButton

OnHideCustomizingForm event

property OnHideCustomizingForm: TNotifyEvent;

Description

This event is called after closing the customizing form of a given TdxBarManager. Write an OnHideCustomizingForm event handler to perform special actions on hiding the customizing form.

See Also

TdxBarManager.Customizing; TdxBarManager.OnShowCustomizingForm

OnItemLinkAdd event

type

TdxBarItemLinkEvent = **procedure**(Sender: TdxBarManager; AItemLink: TdxBarItemLink) **of object**;

property OnItemLinkAdd: TdxBarItemLinkEvent;

Description

This event occurs after adding AItemLink to the ItemLinks property of any object that has the ItemLinks property, for example, when you add an item control to the toolbar, popup menu, or subitem.

See Also

TdxBarManager.OnItemLinkDelete; TdxBarManager.OnItemLinkChange

OnItemLinkChange event

type

```
TdxBarItemLinkEvent = procedure(Sender: TdxBarManager; AItemLink:  
TdxBarItemLink) of object;  
property OnItemLinkChange: TdxBarItemLinkEvent;
```

Description

This event occurs after modifications to the ItemLink specified by the AItemLink parameter.

For instance, this event occurs when you change UserCaption, BeginGroup, UserGlyph, UserPaintStyle, and UserWidth (for combobox or edit style controls).

See Also

TdxBarManager.OnItemLinkAdd; TdxBarManager.OnItemLinkDelete

OnItemLinkDelete event

type

```
TdxBarItemLinkEvent = procedure(Sender: TdxBarManager; AItemLink:  
TdxBarItemLink) of object;  
property OnItemLinkDelete: TdxBarItemLinkEvent;
```

Description

This event occurs before destroying the AItemLink, that is, before item control deletion from a toolbar, popup menu, or subitem.

See Also

TdxBarManager.OnItemLinkAdd; TdxBarManager.OnItemLinkChange

OnMenuMerge event

This event occurs when activating an MDI child form which contains a bar manager.

type

```
TdxBarMenuMergeEvent = procedure(Sender, ChildBarManager: TdxBarManager)  
of object;  
property OnMenuMerge: TdxBarMenuMergeEvent;
```

Description

When a child form is activated, items of its main menu can be added to the end of the main menu of the parent form. Write an OnMenuMerge event handler to add child menu items to the parent menu in a specific manner. For instance, the OnMenuMerge event handler can insert items between parent menu items instead of adding them to the end of the parent menu.

The Sender parameter specifies the bar manager of the MDI parent form. ChildBarManager is the bar manager of the MDI child form.

Important: If the OnMenuMerge event handler is assigned, the bar manager on the parent form does not automatically merge child and parent menus.

Note: The OnMenuMerge event is generated only if the AlwaysMerge property is set to True.

OnShowCustomizingForm event

Occurs when the TdxBarManager customizing form is to display.

```
property OnShowCustomizingForm: TNotifyEvent;
```

Description

This event occurs before activating the customizing form of a given TdxBarManager. Write an OnShowCustomizingForm event handler to perform special actions on customizing form display.

See Also

TdxBarManager.Customizing; TdxBarManager.OnHideCustomizingForm

OnShowCustomizingPopup

Occurs before the customization menu appears.

type

```
TdxBarShowToolbarsPopupEvent = procedure(Sender: TdxBarManager;  
PopupItemLinks: TdxBarItemLinks) of object;  
property OnShowCustomizingPopup: TdxBarShowPopupEvent;
```

Description

Write an OnShowCustomizingPopup event handler to perform specific actions before displaying the customizing popup menu for an item. The PopupItemLinks parameter contains item links of the

customizing popup menu. Use this parameter to add or delete items from the customizing popup menu.

See Also

TdxBarManager.OnShowToolbarsPopup

OnShowToolbarsPopup event

type

TdxBarShowToolbarsPopupEvent = **procedure**(Sender: TdxBarManager;
PopupItemLinks: TdxBarItemLinks) **of object**;

property OnShowToolbarsPopup: TdxBarShowToolbarsPopupEvent;

Description

This event occurs before the customizing menu appears (When a user presses the right mouse button within the toolbar region). The customizing menu represents a list of available toolbars. The menu consists of item links passed by the PopupItemLinks parameter. Using these item links you can add/delete any menu items.

TdxBarManagerList

TdxManagerList contains references to the TdxBarManager of each form that contains a TdxBarManager.

Unit

dxBar

```
TdxBarManagerList = class(TObject);
```

Description

TdxBarManagerList is used in multi-form applications where several forms have their own TdxBarManager objects. TdxBarManagerList contains the list of all BarManagers of an application.

Hierarchy

TObject

TdxBarManagerList Properties

BarManagers	public / read only
Count	public / read only
CustomizingBarManager	public / read only

BarManagers property

read only

Represents the TdxBarManager object array of the forms.

property BarManagers[Index: Integer]: TdxBarManager;

Description

Several forms in any application can have their own TdxBarManager objects. The BarManagers property is an array containing the reference to these TdxBarManager objects. Use the integer Index to get the particular TdxBarManager.

See Also

TdxBarManager

Count property

read only

Returns the number of TdxBarManager objects in a TdxBarManagerList.

property Count: Integer;

Description

The Count property contains the number of TdxBarManager objects in the TdxBarManagerList BarManagers array. Since the BarManagers array is indexed beginning with 0, the Count value is always one greater than the index of the last member of the BarManagers.

See Also

TdxBarManagerList.BarManagers

CustomizingBarManager property

read only

Returns the TdxBarManager object being customized.

property CustomizingBarManager: TdxBarManager;

Description

The CustomizingBarManager returns the TdxBarManager which is currently being customized, that is, its customize form is open and its IsCustomizing property is True.

TdxBar

TdxBar is a non-visual element representing the non-visual portion of a toolbar.

Unit

dxBar

```
TdxBar = class(TCollectionItem);
```

Description

This object stores all the information required for a toolbar's existence and for the creation of its visual representation. Objects of this type are stored in the TdxBars class. The required TdxBar properties are stored in the Registry if it is specified in a corresponding TdxBarManager.

Hierarchy

TObject

TPersistent

TCollectionItem

TdxBar Properties

AllowClose	published / read / write	Hidden	published / read / write
AllowCustomizing	published / read / write	IsMainMenu	published / read / write
AllowQuickCustomizing	published / read / write	IsPredefined	public / read only
AllowReset	published / read / write	ItemLinks	published / read / write
BarManager	public / read only	MultiLine	published / read / write
Bars	protected / read only	Name	published / read / write
BorderStyle	published / read / write	NotDocking	published / read / write
Caption	published / read / write	LockUpdate	public / read / write
Control	public / read only	OneOnRow	published / read / write
DockControl	published / read / write	RealDockControl	public / read only
DockedDockControl	published / read / write	RotateWhenVertical	published / read / write
DockedDockingStyle	published / read / write	Row	published / read / write
DockedLeft	published / read / write	ShowMark	published / read / write
DockedTop	published / read / write	SizeGrip	published / read / write
DockingStyle	published / read / write	UseOwnFont	published / read / write
FloatClientHeight	published / read / write	UseRecentItems	published / read / write
FloatClientWidth	published / read / write	UseRestSpace	published / read / write
FloatLeft	published / read / write	Visible	published / read / write
FloatTop	published / read / write	WholeRow	published / read / write
Font	published / read / write		

AllowClose property

read / write

Determines whether a user can hide a toolbar.

property AllowClose: Boolean;

Description

If AllowClose is True, then a user can hide a given toolbar. If False, a user cannot hide it. Additionally, if AllowClose is False, the toolbar is displayed neither in TdxBarToolbarsListItem, nor in the toolbars popup list, which appears every time a user presses the right mouse button over a docking area.

AllowCustomizing property**read / write**

Specifies toolbar customization capability.

property AllowCustomizing: Boolean;

Description

If AllowCustomizing is set to True (by default), users can add, move and remove items on a given toolbar at runtime.

See Also

TdxBarManager.CanCustomize

AllowQuickCustomizing property**read / write**

Specifies quick toolbar customization capability.

property AllowQuickCustomizing: Boolean;

Description

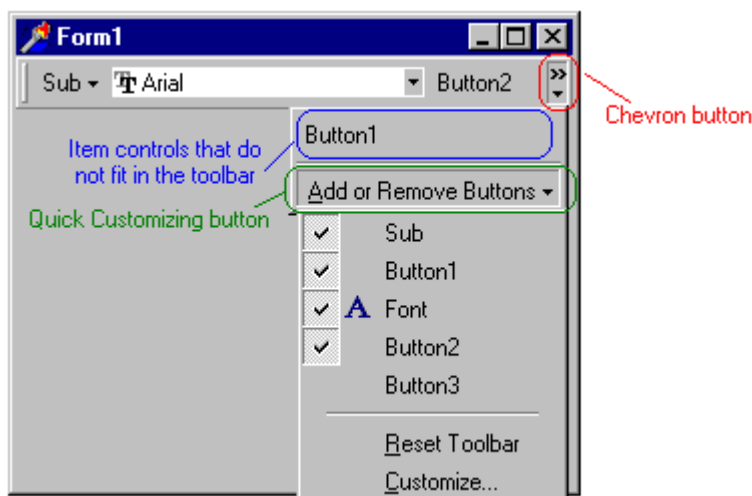
If AllowQuickCustomizing is set to True (by default), the quick customizing button is displayed after items that do not fit in the toolbar for a docked toolbar, and on the caption title for a floating toolbar.



quick customizing button

Clicking the quick customizing button activates a popup menu, which contains a list of potential buttons on that toolbar. Users can select the ones they wish to display and uncheck the ones they want to suppress visually.

If False, the quick customizing button is not available.



See Also*TdxBar.AllowCustomizing*

AllowReset property**read / write**

Allows a toolbar to be restored.

property AllowReset: Boolean;

Description

If AllowReset is set to True (by default), a user can restore original settings for a given toolbar by clicking the 'Reset...' button on the toolbar customizing form or using the Quick Customizing popup. Set AllowReset to False to prevent this. In such an instance, the 'Reset...' button is disabled.

See Also*TdxBar.Reset; TdxBarManager.AllowReset*

BarManager property**read only**

Specifies the TdxBarManager object to which a given TdxBar belongs.

property BarManager: TdxBarManager;

Description

Returns the TdxBarManager that owns a given TdxBar. The TdxBar is referenced via the TdxBarManager Bars property.

Bars property**read only**

Specifies the TdxBars object to which a given toolbar belongs.

property Bars: TdxBars;

Description

The Bars property specifies a list of toolbars controlled by TdxBarManager. A given toolbar belongs to this TdxBars object.

See Also*TdxBarManager; TdxBars*

BorderStyle property**read / write**

Determines the appearance style of toolbar borders.

type

```
TdxBarBorderStyle = (bbsNone, bbsSingle);
```

property BorderStyle: TdxBarBorderStyle;

Description

The BorderStyle property is used to specify the border style of a toolbar. Available border styles include:

Value	Description	Example
sbsNone	No border.	
bbsSingle	Single-line border.	

The default value of border style is bbsSingle.

Caption property**read / write**

Specifies a text string identifying a toolbar to a user.

property Caption: string;

Description

The Caption is displayed in the header of a movable bar control stored in the Control property as well as in the toolbars list on the customizing form. The Caption is also displayed on the toolbar popup menu. A Toolbar popup menu is activated at run time on the right mouse button click event within the toolbar docking area.

Control property**read only**

Returns the bar control created by the information from a given TdxBar.

property Control: TdxBarControl;

Description

This is a ReadOnly property. If Visible is True, then this property returns the bar control that has been created by the information stored in a given TdxBar. Otherwise, it returns nil.

DockControl property**read / write**

Specifies the external dock control to which the current toolbar is placed.

property DockControl: TdxBarDockControl;

Description

The ExpressBars allows you to use external dock controls. This feature extends the functionality of your application. Use the DockControl property to specify new dock control for the toolbar.

See Also

TdxBar.DockedDockControl

DockedDockControl property**read / write**

Specifies the external dock control to which the current toolbar was docked.

property DockedDockControl: TdxBarDockControl;

Description

When you drag a toolbar from the dock control, the DockedDockControl property specifies the dock control to which the current toolbar belongs. If this property value is not **nil**, a mouse double-click moves the toolbar back to the dock control.

See Also

TdxBar.DockControl

DockedDockingStyle property**read / write**

Specifies the docking style of the previous toolbar's docking style.

property DockedDockingStyle: TdxBarDockingStyle;

Description

This property specifies the docking style of the previous toolbar's docking style whereas DockingStyle corresponds to the current docking style of the toolbar. It is used when a user double-clicks on a toolbar to move the toolbar to its previous location.

See Also

TdxBar.DockingStyle

DockedLeft property**read / write**

Specifies the left coordinate of a docked toolbar.

property DockedLeft: Integer;

Description

The DockedLeft property is the left coordinate for a docked toolbar. DockedLeft is presented within the dock control coordinates. For example, if there is one toolbar docked to the right side of a form, then its DockedLeft equals 0.

See Also

TdxBar.DockedTop; TdxBar.FloatLeft; TdxBar.FloatTop

DockedTop property**read / write**

Specifies the top coordinate of a docked toolbar.

property DockedTop: Integer;

Description

DockedTop is a toolbar's top coordinate for docked toolbar. For example, if there is one toolbar docked to the bottom of a form, then its DockedTop equals 0.

See Also

TdxBar.DockedLeft; TdxBar.FloatLeft; TdxBar.FloatTop

DockingStyle property**read / write**

Specifies the layout of a given bar.

type

`TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);`

property DockingStyle: TdxBarDockingStyle;

Description

Specifies a form's side to which a toolbar is docked. If dsNone is included, the toolbar floats.

If Visible is True, then assigning this property changes the style and position of the corresponding bar control. The last position in both movable and docked modes is stored in internal variables as well as in a form's resources, and when defined, in the Registry.

See Also

IsVertical; TdxBarManager.StoreInRegistry

FloatClientHeight property**read / write**

Determines the height of a floating toolbar.

property FloatClientHeight: Integer;

Description

This property specifies toolbar height when its DockingStyle property is dsNone. The FloatClientHeight specifies toolbar height in the screen coordinates.

FloatClientWidth property**read / write**

Determines the width of a floating toolbar.

property FloatClientWidth: Integer;

Description

Specifies toolbar width when its DockingStyle is dsNone. FloatClientWidth specifies toolbar width in the screen coordinates.

FloatLeft**read / write**

Determines the left coordinate of a floating toolbar.

property FloatLeft: Integer;

Description

FloatLeft is a toolbar's left coordinate when DockingStyle is dsNone. The FloatLeft property specifies the left toolbar coordinate in the screen coordinates.

FloatTop property**read / write**

Determines the top coordinate of a floating toolbar.

property FloatTop: Integer;

Description

FloatTop is a toolbar's top coordinate when DockingStyle is dsNone. The FloatTop property specifies the top toolbar coordinate in the screen coordinates.

Font property**read / write**

Specifies item font.

property Font: TFont;

Description

Determines the font used for all items which belong to a given toolbar.

Note: *If this property is modified, the UseOwnFont property is True.*

Hidden property**read / write**

Determines if a toolbar is visible or invisible in the toolbar list.

property Hidden: Boolean;

Description

If the Hidden property is True, a toolbar is not displayed in the customizing form or in the toolbars popup menu. The default value is False.

Note: *Setting the Hidden property to True removes a toolbar from the toolbars popup menu and from the toolbar list of the customizing form, but does not hide the toolbar. Use the Visible property to hide or show a toolbar.*

IsMainMenu property**read / write**

Determines whether a bar is the main menu.

property IsMainMenu: Boolean;

Description

If IsMainMenu is True, a given TdxBar performs the role of a main menu. In this instance, the MainMenuBar property of the TdxBarManager component refers to this TdxBar and the TdxBarManager MainMenuControl refers to the Control property of a given TdxBar. You can set the TdxBar IsMainMenu property programmatically or within the customizing form.

Note: *Only one instance of the TdxBar class may have this property set to True. The value of this property controls the behavior and visualization of the bar control, specified by the Control.*

IsPredefined property**read only**

Determines whether a toolbar was created by a user or it was defined by the programmer.

property IsPredefined: Boolean;

Description

IsPredefined returns True if toolbar settings were defined by the programmer and were not loaded from the Registry or ini file. If IsPredefined is True, the toolbar cannot be deleted or renamed. If existing toolbar settings were loaded from the Registry or INI file, IsPredefined returns False.

ItemLinks property**read / write**

Specifies links to items whose item controls are placed on a toolbar.

property ItemLinks: TdxBarItemLinks;

Description

The property is an array of links to TdxBarManager items. The toolbar item controls are created from these item links.

See Also

TdxBarItemLink; TdxBarItemLinks

MultiLine property**read / write**

Specifies whether items can be placed across several lines within a toolbar.

property MultiLine: Boolean;

Description

When there are a lot of items within a toolbar, some of them are not displayed, though they are not hidden. Set the MultiLine property to **True** to allow the display of items across several toolbar rows.

Name property**read / write**

Determines the name of a bar.

property Name: string;

Description

Each bar of a bar manager has a caption which uniquely identifies it. When the caption is set, the same string value is assigned to the Name property. You can change this name and use it, for instance, to find the bar via the BarByName method.

See Also

TdxBar.Caption

NotDocking property**read / write**

Specifies toolbar location restrictions.

type

```
TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);
```

```
TdxBarDockingStyles = set of TdxBarDockingStyle;
```

```
property NotDocking: TdxBarDockingStyles;
```

Description

Specifies application form sides to which a given toolbar cannot be docked.

Note: If *dsNone* is included, a toolbar can not be floating. Pay attention to the state of the *TdxBarManager.NotDocking* property.

See Also

TdxBarManager.NotDocking

LockUpdate property**read / write**

Controls TdxBar's screen updates.

```
property LockUpdate: Boolean;
```

Description

If this property is True, the modifications of items and item controls, that have links in the TdxBar ItemLinks property, do not refresh the corresponding bar control, which is stored in the Control property.

Assigning False to this property refreshes the control. This property can be used to prevent numerous toolbar refresh operations during changes to a group of item controls (For example, when the Enabled property is toggled for several items).

See Also

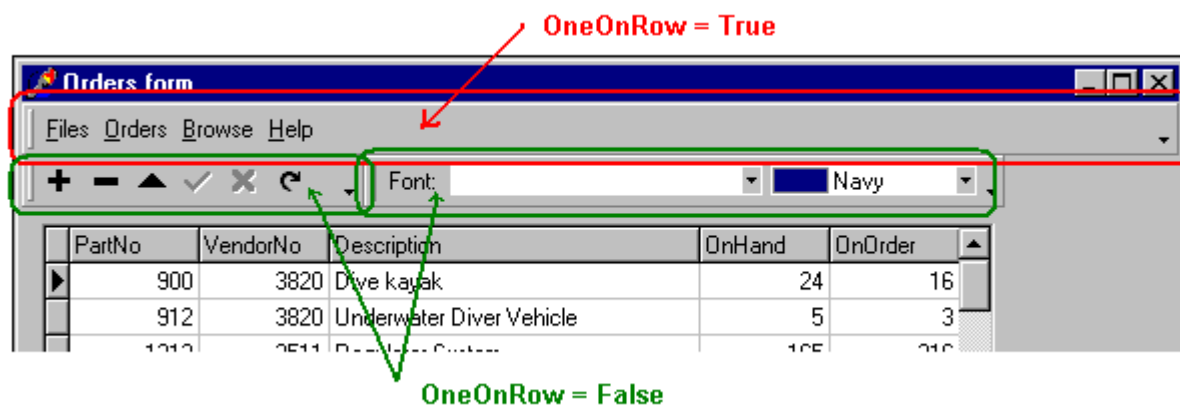
TdxBarManager.LockUpdate

OneOnRow property**read / write**

property OneOnRow: Boolean;

Description

Two or more toolbars may be located on the same row of a dock control; e.g., two small toolbars may be located one after another. Use this property to determine if a toolbar is the only toolbar in the row. OneOnRow returns False if a row contains more than one toolbar.

**RealDockControl property****read only**

Specifies the dock control onto which the current toolbar is placed.

property RealDockControl: TdxDockControl;

Description

Along with external dock controls the ExpressBars contains four default internal dock controls. Use the RealDockControl property to specify the dock control for a toolbar.

See Also

TdxBar.DockControl

RotateWhenVertical property**read / write**

Defines docked toolbar drawing style.

property RotateWhenVertical: Boolean;

Description

If RotateWhenVertical is set to True (by default), when a given toolbar is docked to the left/right side of a form, all item controls on it are rotated 90 degrees to the right. Otherwise, orientation of the item controls does not change.

See Also

TdxBar.DockingStyle

Row property**read / write**

property Row: Integer;

Description

Toolbars may be docked on several rows. For example, if DockingStyle of two bars is dsTop, then one of the toolbars may be placed under another. This property determines the row on which a toolbar is located.

See Also

TdxBar.DockingStyle

ShowMark property

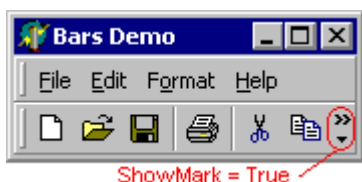
[read / write](#)

Specifies whether to display the 'additional buttons' symbol.

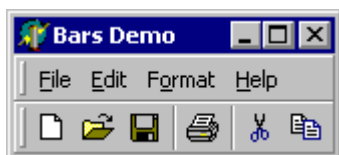
property ShowMark: Boolean;

Description

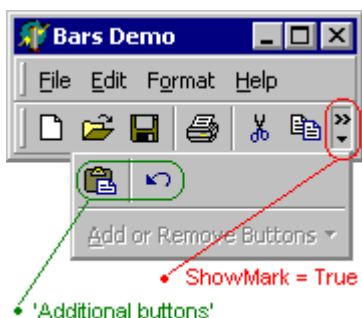
When the size of a toolbar is too small to entirely display all items, some of them are hidden and the 'additional buttons' symbol appears at the left of the toolbar. Set the ShowMark property to **True** to enable it



or **False** to disable.



When pressing this symbol, the window with 'additional buttons' appears:



The default value of the ShowMark property is **True**.

SizeGrip property

[read / write](#)

Specifies whether to display a sizing grip at a toolbar's bottom-right corner.

property SizeGrip: Boolean;

Description

This property is in effect only when the toolbar is docked to the bottom and there are no other controls under it. Use the SizeGrip property to enable a sizing grip at the toolbar's bottom-right corner. This grip can be used to resize a form.

UseOwnFont property**read / write**

Specifies the font used to draw toolbar controls.

property UseOwnFont: Boolean;

Description

If the UseOwnFont property is False (by default), then use the font parameter set for a given toolbar from the TdxBarManager.Font property. Else, use the font parameter set from the TdxBar.Font property.

See Also

TdxBar.Font; TdxBarManager.Font

UseRecentItems property**read / write**

Determines whether new features are used for a given toolbar.

property UseRecentItems: Boolean;

Description

If UseRecentItems is set to True (by default), the most frequently used controls are placed first and other items follow accordingly. If you do not use this feature, item controls will be displayed on a given toolbar in fixed order.

UseRestSpace property**read / write**

Defines toolbar draw style.

property UseRestSpace: Boolean;

Description

If UseRestSpace is True, a toolbar extends to the opposite edge of a form; Else, a toolbar terminates after the last item control.

See Also

TdxBar.WholeRow

Visible property**read / write**

Specifies whether a toolbar is visible.

property Visible: Boolean;

Description

If the Visible property is True, the TdxBar Control property is not nil and returns the visible bar control. Otherwise, the Control is nil and the given TdxBar is not displayed on the screen. Use this property to hide or display a toolbar. The following code hides a form's main toolbar:

```
// Delphi
if GetBarManagerByForm(Form1) <> nil then
  with GetBarManagerByForm(Form1) do
    if MainMenuBar <> nil then
      MainMenuBar.Visible := False;

// C++Builder
TdxBarManager * BarManager;

BarManager = GetBarManagerByForm(Form1);
if (BarManager != NULL)
  if (BarManager->MainMenuBar != NULL)
    BarManager->MainMenuBar->Visible = false;
```

See Also

TdxBar.Hidden

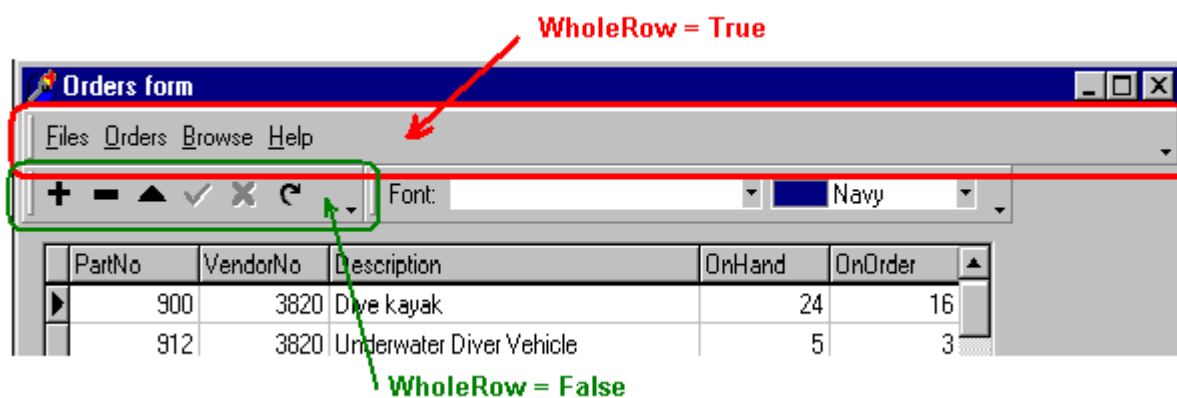
WholeRow property**read / write**

Defines toolbar drawing style.

property WholeRow: Boolean;

Description

If WholeRow is set to True, the toolbar is single in the row and occupies all free space in it. The default value is False.



Setting this property to True or False redraws a toolbar to match the value of WholeRow.

See Also

TdxBar.UseRestSpace

TdxBar Methods

ChangeDockingStyle	public	Reset	public
Create	public	ResetWithConfirmation	public
Destroy	public		

ChangeDockingStyle method

Changes the layout of a specific bar.

type

```
TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);
```

```
procedure ChangeDockingStyle(AStyle: TdxBarDockingStyle; ADockControl:
TdxDockControl;
```

Description

Call the ChangeDockingStyle method to change the side to which a toolbar is docked, specified by the AStyle parameter. If the **dsNone** value is chosen, the toolbar floats. The ADockControl specifies the internal dock control onto which a toolbar is placed.

See Also

TdxBar.DockingStyle

Create method

Creates a new TdxBar instance.

```
constructor Create(Collection: TCollection); override;
```

Description

Creates and initializes the TdxBar object. After calling the inherited constructor, Create sets AllowToDeleteOrRename to True and DockingStyle to dsNone.

Destroy method

Destroys an instance of TdxBar.

```
destructor Destroy; override;
```

Description

The Destroy property calls the OnDelete method, destroys every item belonging to TdxBar, and then destroys the TdxBar object itself.

Reset method

```
procedure Reset;
```

Description

Removes any changes a user makes to a toolbar and restores original settings.

See Also

TdxBar.AllowReset; TdxBar.ResetWithConfirmation

ResetWithConfirmation method

```
procedure ResetWithConfirmation;
```

Description

Displays the generic dialog box with a request message. In case of a positive answer the TdxBar.Reset method is called to restore original settings.

See Also

TdxBar.AllowReset; TdxBar.Reset

TdxBars

Contains a list of all TdxBarManager toolbars.

Unit

dxBar

```
TdxBars = class(TCollection);
```

Description

The TdxBars is an array of TdxBar objects. This class stores all information on position and configuration of every TdxBarManager toolbar. The TdxBars object is represented in TdxBarManager as the Bars property.

Hierarchy

TObject

 TPersistent

 TCollection

TdxBars Properties

BarManager	public / read only	DockingZoneSize	protected / read / write
DockControls	public / read only	Items	public / read / write

BarManager property

read only

Returns the TdxBarManager to which a given TdxBars object belongs.

```
property BarManager: TdxBarManager;
```

Description

Returns the TdxBarManager that owns a given TdxBars. TdxBars is represented in TdxBarManager as the Bars property.

DockControls property

read only

Contains an array of dock controls.

type

```
TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);
```

```
property DockControls[Index: TdxBarDockingStyle]: TdxDockControl;
```

Description

Use this property to work with dock controls on which toolbars are located.

DockingZoneSize property

read / write

```
property DockingZoneSize: Integer;
```

Description

Specifies the size of the docking zone, that is, the distance from dock control borders to the bar control within which the bar control may be 'docked' to this dock control.

See Also

TdxDockControl

Items property**read / write**

Represents a list of TdxBar elements.

property Items[Index: Integer]: TdxBar;

Description

Represents a list of TdxBar elements that are all toolbars of a TdxBarManager component. The first toolbar index in the list is 0. The number of elements is stored in the Count property of the ancestor, that is, in the Count property of TCollection.

TdxBars Methods

Add	public	Destroy	public
Create	public	GetUniqueToolBarName	public

Add method

Adds a new TdxBar to the toolbar list.

```
function Add: TdxBar;
```

Description

This method creates an instance of the TdxBar class, adds it to a given TdxBars list and returns the new instance. Use this function to create new toolbars.

Create method

Creates an instance of TdxBars.

```
constructor Create(ABarManager: TdxBarManager);
```

Description

This method creates and initializes a TdxBars instance. After creating the TdxBars object, its BarManager property is set to ABarManager and the DockingZoneSize is set to 10.

Destroy method

Destroys an instance of TdxBars.

```
destructor Destroy; override;
```

Description

This method destroys a TdxBars instance, but does not destroy the corresponding TdxBar object to which it refers.

GetUniqueToolBarName method

Returns a unique name for a toolbar.

```
function GetUniqueToolBarName(const BaseName: string): string;
```

Description

This method is called to provide a unique name for a toolbar. The BaseName parameter determines the initial name. If this name is already unique, it is returned without changes. If there is a toolbar with the same name, the appropriate numeral is added at the end of the BaseName string.

TdxBarItemLink

The TdxBarItemLink is a link between an item and the item control; created by the description taken from the item and TdxBarItemLink.

Unit

dxBar

```
TdxBarItemLink = class(TCollectionItem);
```

Description

Instances of this type are stored in TdxBarItemLinks. TdxBarItemLink defines the link between an item and item control on the bar control. TdxBarItemLink Control refers to the item control.

In order to perform actions on item controls, use the TdxBarItemLink instance, but not the item control itself. If you wish to add or remove an existing link from the ItemLinks property of any TdxBar, TdxBarPopupMenu, TCustomdxBarSubItem class or their descendants, do it at the TdxBarItemLink and TdxBarItemLinks level(s).

The published properties of TdxBarItemLink are stored in the Registry if defined in a corresponding TdxBarManager. Use the TdxBarItemLink methods and properties to customize the look of a particular item control. The easiest way to define the look of the item control is to use the customizing popup menu. To activate the customizing popup menu click the right mouse button on the item control.

Hierarchy

TObject

TPersistent

TCollectionItem

TdxBarItemLink Properties

AvailableIndex	public / read only	PaintStyle	public / read only
BarControl	public / read only	RealItemLink	public / read only
BarManager	public / read only	RowHeight	protected / read / write
BeginGroup	published / read / write	UserCaption	published / read / write
CanVisibleIndex	public / read only	UserDefine	published / read / write
Caption	public / read only	UserGlyph	published / read / write
Control	public / read only	UserPaintStyle	published / read / write
Data	published / read / write	UserWidth	published / read / write
Glyph	public / read only	Visible	published / read / write
Item	published / read / write	VisibleIndex	public / read only
ItemRect	public / read / write	Width	public / read only
MostRecentlyUsed	published / read / write		

AvailableIndex property

read only

Returns an item's position in the AvailableItems array of TdxBarItemLinks.

property AvailableIndex: Integer;

Description

A TdxBarItemLink object is indexed in the AvailableItems array of TdxBarItemLinks to which it belongs. The AvailableIndex property of TdxBarItemLink contains an item's index value in that array.

AvailableItems is a zero-based array. The first member of it has an index value of 0, the second member has an index value of 1, etc.

See Also

*TdxBarItemLinks.AvailableIndexOf; TdxBarItemLinks.AvailableItemCount;
TdxBarItemLinks.AvailableItems*

BarControl property

read only

Determines the bar control containing the item control, specified by the TdxBarItemLink Control property.

property BarControl: TCustomdxBarControl;

Description

Returns the bar control containing the item control managed by a given TdxBarItemLink. This bar control is a parent of the item control returned by the Control property.

See Also

TdxBarItemLink.Control

BarManager property**read only**

Specifies the TdxBarManager that owns a given TdxBarItemLink.

property BarManager: TdxBarManager;

Description

Specifies the TdxBarManager to which a given item link belongs. BarManager returns the same TdxBarManager as the TdxBarItemLinks object that includes the given TdxBarItemLink.

BeginGroup property**read /write**

Specifies whether an item control starts a group.

property BeginGroup: Boolean;

Description

If True, the item control is the first in a group. It cannot start the group if VisibleIndex equals 0. When you change the value of the BeginGroup property, a TdxBarManager OnItemLinkChange event is generated.

CanVisibleIndex property**read only**

Returns an item's position in the CanVisibleItems array of TdxBarItemLinks.

property CanVisibleIndex: Integer;

Description

TdxBarItemLink is indexed in the CanVisibleItems array of TdxBarItemLinks to which it belongs. The CanVisibleIndex property of TdxBarItemLink contains an item's index value in that array.

CanVisibleItems is a zero-based array. The first member of it has an index value of 0, the second member has an index value of 1, etc.

See Also

*TdxBarItemLinks.CanVisibleIndexOf; TdxBarItemLinks.CanVisibleItemCount;
TdxBarItemLinks.CanVisibleItems*

Caption property**read only**

Determines control caption.

property Caption: string;

Description

Returns the current caption used by a control. If the udCaption style is in the UserDefine property, Caption equals the UserCaption property, otherwise, the Caption property is the same as item Caption.

Control property**read only**

Returns the item control specified by TdxBarItemLink information.

property Control: TdxBarItemControl;

Description

Returns the item control created by information stored in a given TdxBarItemLink and the item specified by the TdxBarItemLink Item property. This item control belongs to the bar control specified by the BarControl property of TdxBarItemLink. If a bar control is not visible (it is equal to nil) or a TdxBarItemLink is not visible itself due to the value of the Item Visible property, then the Control property is nil.

See Also

TdxBarItemLink.BarControl

Data property**read / write**

Stores an integer value as a part of a TdxBarItemLink object.

property Data: Integer;

Description

The Data property has no predefined meaning similar to the Tag property of components. The default value is 0.

Glyph property**read only**

Specifies item control glyph.

property Glyph: TBitmap;

Description

Returns the current glyph used for item control visualization. If an additional picture has been specified for the item control (udGlyph is included in UserDefine), this property returns the glyph, specified by the UserGlyph property, otherwise, it returns the glyph, specified by the Glyph property of the item.

See Also

TdxBarItemLink.UserGlyph

Item property**read / write**

property Item: TdxBarItem;

Description

This property represents the item a given TdxBarItemLink object links to an item control. The BarManager property of an item may not equal the TdxBarManager of a form; (i.e. In MDI applications

when the MDI-child form is maximized and both the main form and the MDI child have components of type TdxBarManager with main menus).

See Also

TdxBarItem; *TdxBarItemLink.BarControl*

ItemRect property

read / write

Returns item control coordinates.

property ItemRect: TRect;

Description

If TdxBarItemLink is not nil, the ItemRect returns item coordinates relative to the client area of a bar control.

MostRecentlyUsed property

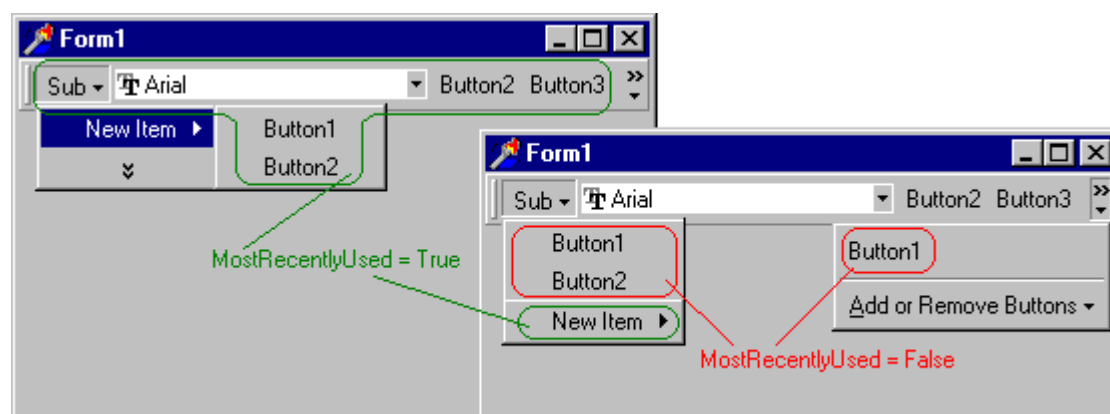
read / write

Determines whether an item control linked with a given TdxBarItemLink object is marked as the most frequently used.

property MostRecentlyUsed: Boolean;

Description

Sets the initial state of an item control. When drawing a toolbar, item controls with the MostRecentlyUsed property active are drawn first and then all others follow. This property is best to set at design time via the customizing popup menu. If MostRecentlyUsed is set to False for a menu's item, then it is not displayed at runtime in the menu.



PaintStyle property**read only**

Specifies item control paint style.

type

```
TdxBarPaintStyle = (psStandard, psCaption, psCaptionInMenu,  
psCaptionGlyph);
```

```
property PaintStyle: TdxBarPaintStyle;
```

Description

Returns the paint style of an item control. If udPaintStyle is included in UserDefine, the property returns UserPaintStyle; otherwise, it returns the PaintStyle of the item. In the latter, if the item specified by TdxBarItemLink is of type TdxBarButton, PaintStyle equals TdxBarButton paint style; For other item types, TdxBarItemLink equals psStandard.

RealItemLink property**read only**

Contains the actual item link of a given TdxBarItemLink.

```
property RealItemLink: TdxBarItemLink;
```

Description

Used when creating additional (new) item controls.

RowHeight property**read / write**

```
property RowHeight: Integer;
```

Description

This property is used internally. It specifies the height of a linked item control. If an item is the first in a group, height includes the height of the group separator. The height of an item control depends upon the DockingStyle of a parent bar control.

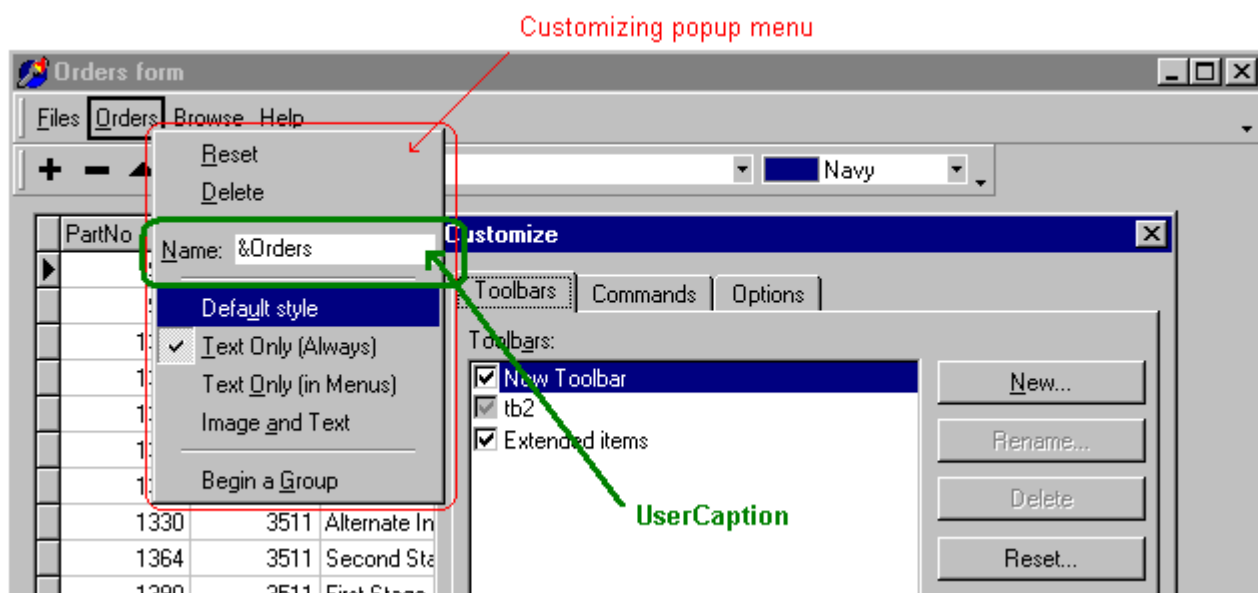
UserCaption property**read / write**

Determines an additional caption defined by a user.

property UserCaption: string;

Description

Contains the definitive caption of an item control. UserCaption always overrides item Caption. This property can be modified from the context-sensitive modification popup menu. This menu is invoked by a right mouse button click on an item control when toolbar customizing is active. Set UserCaption programmatically or by means of the customizing popup:

**See Also**

TdxBarItemLink.UserDefine

UserDefine property**read / write**

Specifies the properties defined by a user.

type

`TdxBarUserDefine = (udCaption, udGlyph, udPaintStyle, udWidth);`

`TdxBarUserDefines = set of TdxBarUserDefine;`

property UserDefine: TdxBarUserDefines;

Description

This property is a set of TdxBarUserDefine type elements. The elements determine the values of Caption, Glyph, PaintStyle, and Width properties. If an element is included in UserDefine, these properties will contain the UserCaption, UserGlyph, UserWidth, and UserPaintStyle accordingly. When changing these properties, the appropriate elements are added to UserDefine.

UserGlyph property**read / write**

Specifies a user-defined glyph.

property UserGlyph: TBitmap;

Description

This property determines a user-defined or programmatically defined glyph. If udGlyph is included in UserDefine, the additional glyph affects the Glyph property. This element is automatically added to UserDefine when changing UserGlyph.

See Also

TdxBarItemLink.UserDefine

UserPaintStyle property**read / write**

Specifies a user-defined paint style.

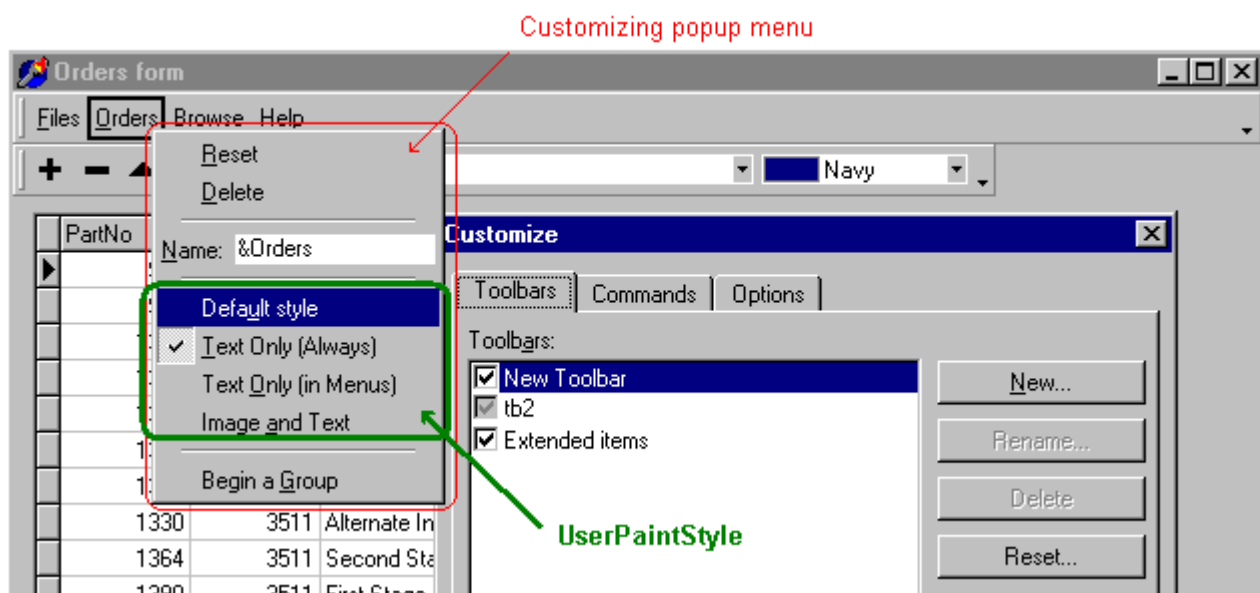
type

```
TdxBarPaintStyle = (psStandard, psCaption, psCaptionInMenu,
psCaptionGlyph);
```

```
property UserPaintStyle: TdxBarPaintStyle;
```

Description

This property can be modified from the context-sensitive modification popup menu or programmatically. The menu is invoked by clicking the right mouse button on an item control when the customizing form is active. By default, UserPaintStyle is psStandard. Set the UserPaintStyle programmatically or by means of the customizing popup menu:

**See Also**

TdxBarItemLink.UserDefine

UserWidth property**read / write**

Specifies item control width.

```
property UserWidth: integer;
```

Description

TdxBarItemLink describes the link between an item and item control. Use UserWidth to specify the width of a particular item control. By default, all item controls linked to the item have the same width. UserWidth may be used to customize different width values for different item controls.

See Also

TdxBarItemLink.UserDefine; Customizing Popup Menu

Visible property**read / write**

Determines whether an item control appears on a toolbar.

property Visible: Boolean;

Description

If Visible is True, an item control linked to a given TdxBarItemLink object can be placed on the toolbar. In this case a given TdxBarItemLink object is included into the TdxBarItemLinks.CanVisibleItems array.

See Also

TdxBarItemLinks.CanVisibleItems; TdxBarItemLinks.VisibleItems

VisibleIndex property**read only**

Returns item's position in the VisibleItems array of TdxBarItemLinks.

property VisibleIndex: Integer;

Description

TdxBarItemLink is indexed in the VisibleItems array of TdxBarItemLinks to which it belongs. The VisibleIndex property of TdxBarItemLink contains an item's index value in that array.

VisibleItems is a zero-based array. The first member of it has an index value of 0, the second member has an index value of 1, and so on.

See Also

TdxBarItemLinks.VisibleIndexOf; TdxBarItemLinks.VisibleItemCount; TdxBarItemLinks.VisibleItems

Width property**read only**

Returns item control width.

property Width : integer;

Description

Use this property to access width of an item control. If UserWidth is defined, Width equals UserWidth; Else Width equals 0.

TdxBarItemLink Methods

BringToTopInRecentList	public	Destroy	public
Create	public	SendToBottomInRecentList	public

BringToTopInRecentList method

“Promotes” an item link in the list of the most frequently used items.

```
procedure BringToTopInRecentList(IncCount: Boolean);
```

Description

Use this procedure to “promote” an item control, corresponding to a given item link in the list of the most frequently used menu items. If the IncCount parameter is True, the hit counter of an item link increases; i.e. The item is “promoted” as if it had been pressed. Otherwise, the hit counter remains unchanged.

Note: This procedure is used if the *MenusShowRecentItemsFirst* property of a bar manager is set to True and the *UseRecentItems* property of *TdxBar* is True.

See Also

TdxBar.UseRecentItems; TdxBarItemLink.SendToBottomInRecentList;
TdxBarManager.MenusShowRecentItemsFirst

Create method

Creates a TdxBarItemLink instance.

```
constructor Create(Collection: TCollection); override;
```

Description

This method creates and initializes a TdxBarItemLink instance. The BeginGroup property of the instance is set to False. The Create method generates a TdxBarManager OnItemLinkAdd event.

Destroy method

Destroys a TdxBarItemLink instance.

```
destructor Destroy; override;
```

Description

This method destroys a TdxBarItemLink instance and generates an TdxBarManager OnItemLinkDelete event and TdxBarItemLinks OnChange event.

SendToBottomInRecentList method

```
procedure SendToBottomInRecentList;
```

Description

Use this procedure to move an item control, corresponding to a given item link in the list of most frequently used menu items, to the bottom of the list. After calling this method, an item is set as least frequently used.

Note: This procedure is used when the *MenusShowRecentItemsFirst* property of a bar manager is set to *True* and the *UseRecentItems* property of *TdxBar* is *True*.

See Also

TdxBarItemLink.BringToTopInRecentList

TdxBarItemLinks

TdxBarItemLinks is a collection of TdxBarItemLink elements. TdxBarItemLinks represents TdxBarItemLink objects in bar controls or sub items.

Unit

dxBar

```
TxBarItemLinks = class(TCollection);
```

Description

Each TdxBarItemLinks class holds a group of TdxBarItemLink objects which represent links between TdxBarManager items and corresponding item controls. The TdxBarItemLinks object stores information on the number and behavior of the newly created item controls for toolbars, popup menus, and submenus. For instance, every toolbar uses the ItemLinks property of type TdxBarItemLinks to specify the items that have linked item controls on the toolbar.

The TdxBarItemLinks derives methods and properties from TCollection:

Use the Add method to add TdxBarItemLink objects to the TdxBarItemLinks collection.

Use the Clear method to remove all item links from the TdxBarItemLinks object.

The Count property contains the number of item links in TdxBarItemLinks.

Hierarchy

TObject

TPersistent

TCollection

TdxBarItemLinks Properties

AvailableItems	public / read only	Items	public / read / write
AvailableItemCount	public / read only	Owner	public / read only
BarControl	public / read / write	RealVisibleItemCount	protected / read only
BarManager	public / read only	VisibleItemCount	public / read only
CanVisibleItems	public / read only	VisibleItems	public / read only
CanVisibleItemCount	public / read only		

AvailableItems property

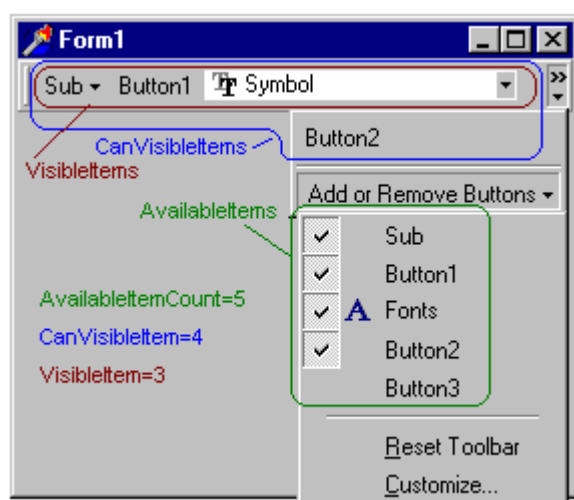
read only

Presents an indexed array of TdxBarItemLink elements with items that may be placed onto a toolbar.

property AvailableItems[Index: Integer]: TdxBarItemLink;

Description

The array contains only TdxBarItemLink objects with the Visible property of a corresponding item equal to ivAlways. The value of the Index parameter corresponds to the AvailableIndex property of TdxBarItemLink.



See Also

*TdxBarItemLink.AvailableIndex; TdxBarItemLinks.AvailableIndexOf;
TdxBarItemLinks.AvailableItemCount; TdxBarItemLinks.CanVisibleItems; TdxBarItemLinks.Items;
TdxBarItemLinks.VisibleItems*

AvailableItemCount property**read / write**

Returns the number of items in the AvailableItems array.

property AvailableItemCount: Integer;

Description

Contains the number of items in the TdxBarItemLinks.AvailableItems array. Since this array is indexed starting with 0, the value of AvailableItemCount is always one greater than the index of the last member of AvailableItems.

See Also

TdxBarItemLink.AvailableIndex; TdxBarItemLinks.AvailableIndexOf; TdxBarItemLinks.AvailableItems

BarControl property**read only**

Specifies the bar control that owns the item control specified in the TdxBarItemLink object.

property BarControl: TCustomdxBarControl;

Description

Returns the bar control created by the CreateBarControl method. It can equal nil if the Owner has no visual representation (that is if the Owner is currently invisible).

See Also

TCustomdxBarControl

BarManager property**read only**

Specifies the TdxBarManager to which a given TdxBarItemLinks object belongs.

property BarManager: TdxBarManager;

Description

Returns the TdxBarManager specified at TdxBarItemLinks creation as a constructor's parameter.

Note: As a rule, BarManager is the TdxBarManager of the object that is stored in the TdxBarItemLinks Owner property.

See Also

TdxBarManager

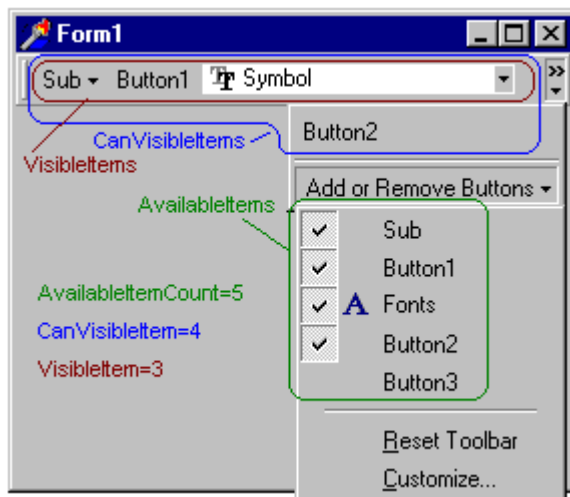
CanVisibleItems property**read only**

Presents an indexed array of TdxBarItemLink elements with visible and potential items.

property CanVisibleItems[Index: Integer]: TdxBarItemLink;

Description

The array contains TdxBarItemLink objects with visible items and items that do not currently fit in a toolbar. The value of the Index parameter corresponds to the CanVisibleIndex property of TdxBarItemLink.

**See Also**

*TdxBarItemLink.CanVisibleIndex; TdxBarItemLinks.AvailableItems;
TdxBarItemLinks.CanVisibleIndexOf; TdxBarItemLinks.CanVisibleItemCount; TdxBarItemLinks.Items;
TdxBarItemLinks.VisibleItems*

CanVisibleItemCount property**read only**

Returns the number of items in the CanVisibleItems array.

property CanVisibleItemCount: Integer;

Description

Contains the number of items in the TdxBarItemLinks.CanVisibleItems array. Since this array is indexed starting with 0, the CanVisibleCount value is always one greater than the index of the last member of CanVisibleItems.

See Also

*TdxBarItemLink.CanVisibleIndex; TdxBarItemLinks.CanVisibleIndexOf;
TdxBarItemLinks.CanVisibleItems*

Items property**read / write**

Represents an array of TdxBarItemLink elements. These elements are used to link an item to the item control on the toolbar.

property Items[Index: Integer]: TdxBarItemLink;

Description

Items is an indexed array containing all TdxBarItemLink type objects stored in a given TdxBarItemLinks object. The first element's index is 0. The number of elements in Items is stored in the ancestor's Count property (TCollection Count property).

See Also

TdxBarItemLinks.AvailableItems; TdxBarItemLinks.CanVisibleItems; TdxBarItemLinks.IndexOf; TdxBarItemLinks.VisibleItems

Owner property**read only**

Returns the owner of a given TdxBarItemLinks object.

property Owner: TObject;

Description

Returns the object which owns a given TdxBarItemLinks. Owner can include TdxBar, TdxBarPopupMenu, TCustomdxBarSubItem object, or it can be nil. The type of Owner impacts the bar control created by the CreateBarControl method. If Owner is TdxBar, then a TdxBarControl is created; Else a TdxBarSubMenuControl is created.

RealVisibleItemCount property**read only**

Returns the number of visible item links belonging to a given item links collection.

property RealVisibleItemCount: Integer;

Description

The RealVisibleItemCount property returns the number of visible item links on screen.

See Also

TdxBarItemLinks.VisibleItemCount; TdxBarItemLinks.VisibleItems

VisibleItemCount property

read only

Returns the number of items in the VisibleItems array.

property VisibleItemCount: Integer;

Description

VisibleItemCount contains the number of items in the TdxBarItemLinks.VisibleItems array. Since this array is indexed starting with 0, the value of VisibleCount is always one greater than the index of the last member of VisibleItems.

See Also

*TdxBarItemLink.VisibleIndex; TdxBarItemLinks.RealVisibleItemCount;
TdxBarItemLinks.VisibleIndexOf; TdxBarItemLinks.VisibleItems*

VisibleItems property

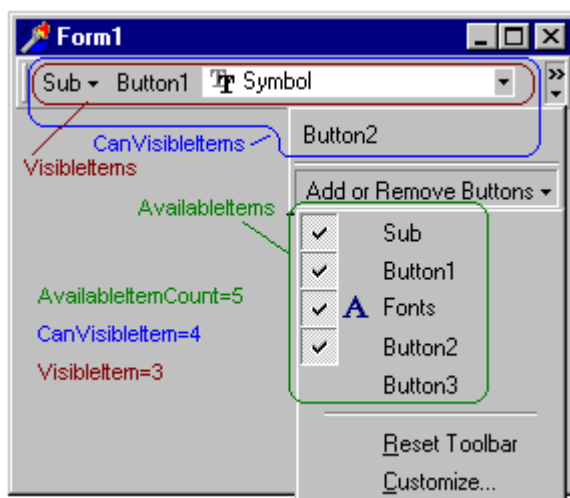
read only

VisibleItems is an indexed array of TdxBarItemLink elements with visible items.

property VisibleItems[Index: Integer]: TdxBarItemLink;

Description

The array contains TdxBarItemLink objects with visible items in a toolbar at the moment. The value of the Index parameter corresponds to the VisibleIndex property of TdxBarItemLink.



See Also

TdxBarItemLinks.VisibleItemCount; TdxBarItemLinks.VisibleItems

TdxBarItemLinks Methods

Add	public	Destroy	public
AvailableIndexOf	public	HasItem	public
CanVisibleIndexOf	public	IndexOf	public
Create	public	Update	protected
CreateBarControl	public	VisibleIndexOf	public

Add method

Add a new TdxBarItemLink element to a given TdxBarItemLinks object.

```
function Add: TdxBarItemLink;
```

Description

This method creates an instance of the TdxBarItemLink class, adds it to the list of a given TdxBarItemLinks and returns the new TdxBarItemLink. Use this function to create new item links and new item controls.

AvailableIndexOf method

Returns position of a specified TdxItemLink in the TdxBarItemLinks.AvailableItems array.

```
function AvailableIndexOf(Value: TdxBarItemLink: Integer;
```

Description

Call AvailableIndexOf to get the index of Value in AvailableItems. Index of the first element in the AvailableItems array is 0. AvailableIndexOf returns -1 if Value is not in the array.

See Also

*TdxBarItemLink.AvailableIndex; TdxBarItemLinks.AvailableItemCount;
TdxBarItemLinks.AvailableItems*

CanVisibleIndexOf method

Returns position of a specified TdxItemLink in the TdxBarItemLinks.CanVisibleItems array.

```
function CanVisibleIndexOf(Value: TdxBarItemLink: Integer;
```

Description

Call CanVisibleIndexOf to get the index of Value in CanVisibleItems. The index of the first element in the CanVisibleItems array is 0. CanVisibleIndexOf returns -1 if Value is not in the array.

See Also

*TdxBarItemLink.CanVisibleIndex; TdxBarItemLinks.CanVisibleItemCount;
TdxBarItemLinks.CanVisibleItems*

Create method

Creates a TdxBarItemLinks instance.

```
constructor Create(ABarManager: TdxBarManager);
```

Description

This method creates and initializes an object of type TdxBarItemLinks. The ABarManager parameter specifies the instance of TdxBarManager which owns the given TdxBarItemLinks object.

CreateBarControl method

Specifies the bar control created with information stored in TdxBarItemLinks.

```
procedure CreateBarControl;
```

Description

This method creates a bar control using information stored in a given TdxBarItemLinks, considering Owner. If Owner is TdxBar, an instance of type TdxBarControl is created; Else, an instance of type TdxBarSubMenuControl is created.

Note: The newly created bar control is stored in the BarControl property.

See Also

TdxBarControl; TCustomdxBarControl; TdxBarSubMenuControl

Destroy method

Destroys a given TdxBarItemLinks instance.

```
destructor Destroy; override;
```

Description

This method destroys a given TdxBarItemLinks instance and the object specified by its BarControl property.

HasItem

Determines whether the specified item belongs to a given TdxBarItemLinks object.

```
function HasItem(AItem: TdxBarItem): Boolean;
```

Description

If True, either the given TdxBarItemLinks or the child TdxBarItemLinks object contains an item link to the item specified by the AItem parameter.

IndexOf method

```
function IndexOf(Value: TdxBarItemLink): Integer;
```

Description

Returns the index of a TdxBarItemLink object specified by Value in the indexed property Items. The index of the first item link in the collection is 0.

See Also

TdxBarItemLinks.Items

Update method

```
procedure Update(Item: TCollectionItem); override;
```

Description

Generates an OnChange event and notifies TdxBarManager of changes in a TdxBarItemLink object.

VisibleIndexOf method

Returns position of a specified TdxItemLink in the TdxBarItemLinks.VisibleItems array.

```
function VisibleIndexOf(Value: TdxBarItemLink): Integer;
```

Description

Call VisibleIndexOf to get the index of Value in the indexed property of visible item links - VisibleItems. The index of the first element in the VisibleItems array is 0. VisibleIndexOf returns -1 if Value is not in the array.

See Also

TdxBarItemLink.VisibleIndex; TdxBarItemLinks.VisibleItemCount; TdxBarItemLinks.VisibleItems

TdxBarItemLinks Events

OnChange

public

OnChange event

The OnChange event is generated when adding or deleting elements from TdxBarItemLinks.

property OnChange: TNotifyEvent;

Description

This event occurs after changing a given TdxBarItemLinks (upon adding or deleting elements). This event is used internally to process changes in item links of toolbars.

TdxBarGroup

TdxBarGroup is an object that represents grouped items and/or other groups.

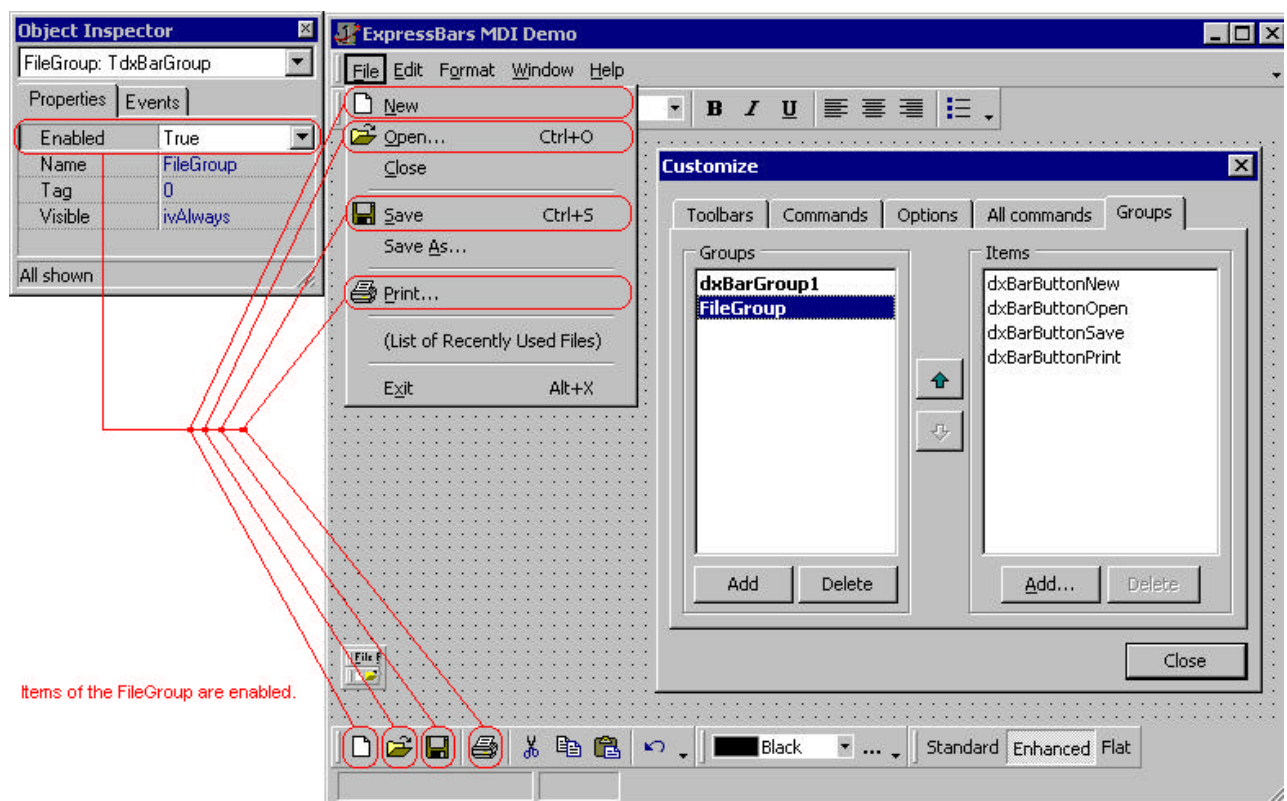
Unit

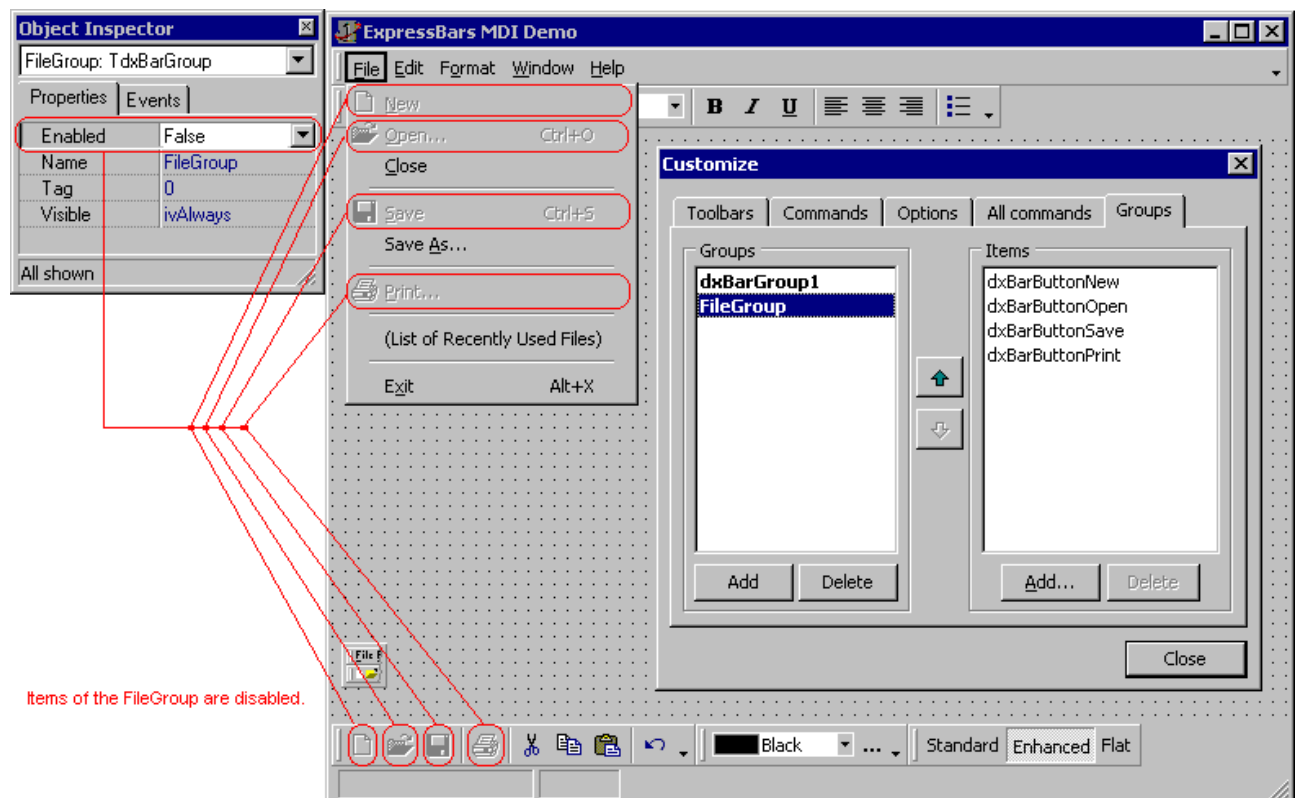
dxBar

```
TdxBarGroup = class(TComponent);
```

Description

A group can contain both groups and items. This feature is designed to help users manage item groups (enable/disable or hide them). The TdxBarGroup object contains a list of items or groups. You can access any grouped item via the Items property and its Index parameter. The first item index is 0. The number of items in a group is stored in the Count property.





Hierarchy

TObject

TPersistent

TComponent

TdxBarGroup Properties

BarManager	public / read only	Index	public / read / write
Count	public / read only	Items	public / read only
Enabled	published / read / write	Visible	published / read / write

BarManager property

read only

property BarManager: TdxBarManager;

Description

Specifies the TdxBarManager to which a given TdxBarGroup belongs.

Count property

read only

Returns the number of items in a group.

property Count: Integer;

Description

The Count property contains the number of grouped items in a group. Since the item array is zero-based, the Count value is always one greater than the index of the last member of the item array.

Enabled property

read / write

Specifies whether grouped items respond to mouse, keyboard, and timer events.

property Enabled: Boolean;

Description

Use the Enabled property to change grouped item controls available to the end users. To disable controls, set the Enabled property to **False**. All item controls that correspond to a given group will be disabled. If **False**, grouped item controls ignore mouse and keyboard events.

Index property

read / write

Specifies group index in the TdxBarManager.Groups list.

property Index: Integer;

Description

The Index property returns the index of a given group in the list represented by the Groups property of the TdxBarManager. The first item index is 0.

Items property

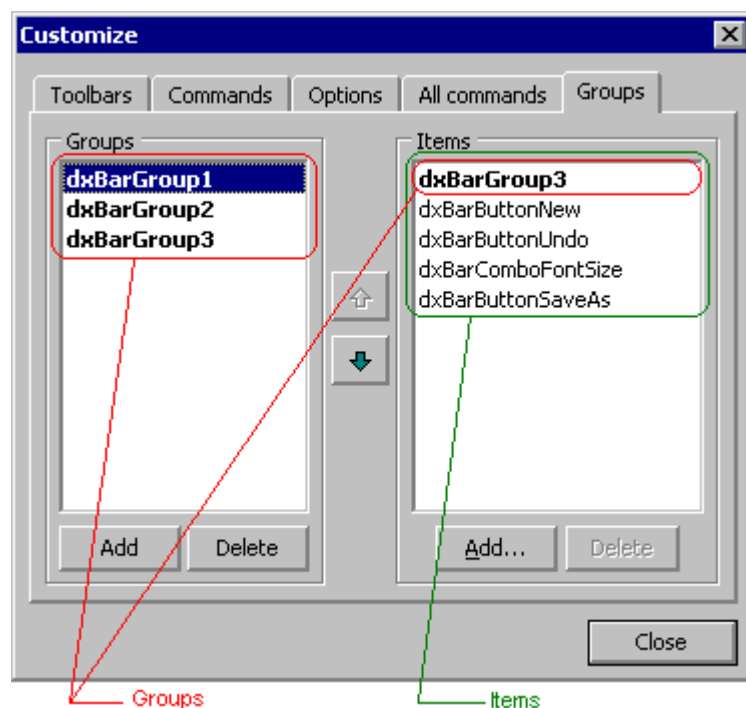
read only

Contains a list of grouped items.

property Items[Index: **Integer**]: TComponent;

Description

Items are the elements to display within a toolbar(s). You can arrange these items in groups. This property returns the grouped item specified by the Index parameter. The first item index is 0. The number of items is stored in the Count property.



Visible property**read / write**

Determines whether group items are displayed on screen.

type

```
TdxBarItemVisible = (ivNever, ivInCustomizing, ivAlways);
```

```
property Visible: TdxBarItemVisible;
```

Description

All the items that belong to the current group are non-visual objects, therefore, the Visible property affects appearance style:

Value	Meaning
ivAlways	Grouped item controls are visible at design and runtime.
ivNever	Grouped item controls are not visible at runtime.
ivInCustomizing	Grouped item controls are visible only during toolbar customization.

The default value is **ivAlways**.

TdxBarGroup Methods

Add	public	Move	public
Delete	public	Remove	public
IndexOf	public		

Add method

Adds a new item to the group item list.

```
procedure Add(AItem: TComponent);
```

Description

Use the Add method to create a new item (or a new group) within the current group. When adding a new group to the current group, all added group items become current group members.

Delete method

Deletes an item from the group item list.

```
procedure Delete(Index: Integer);
```

Description

Use the Delete method to delete an item from the current group. This method deletes the item specified by the Index parameter from the group item list.

IndexOf method

Returns the grouped item index.

```
function IndexOf(AItem: TComponent): Integer;
```

Description

Returns the index within the Items array of the item specified by the AItem parameter. The first item index in a group is 0.

Move method

Moves a group item to a new position.

```
procedure Move(FromIndex, ToIndex: Integer);
```

Description

Moves an item inside the current group item list from its current position specified by the FromIndex parameter to a new position specified by the ToIndex parameter. This method is called when you press the arrow buttons within the Groups page of the customization form.

Remove method

Deletes an item from the grouped items list.

```
procedure Remove(AItem: TComponent);
```

Description

Call the RemoveItem method to delete an item from the grouped items list. The AItem parameter represents the object to delete.

TdxDockControl

Used to draw dock control lines.

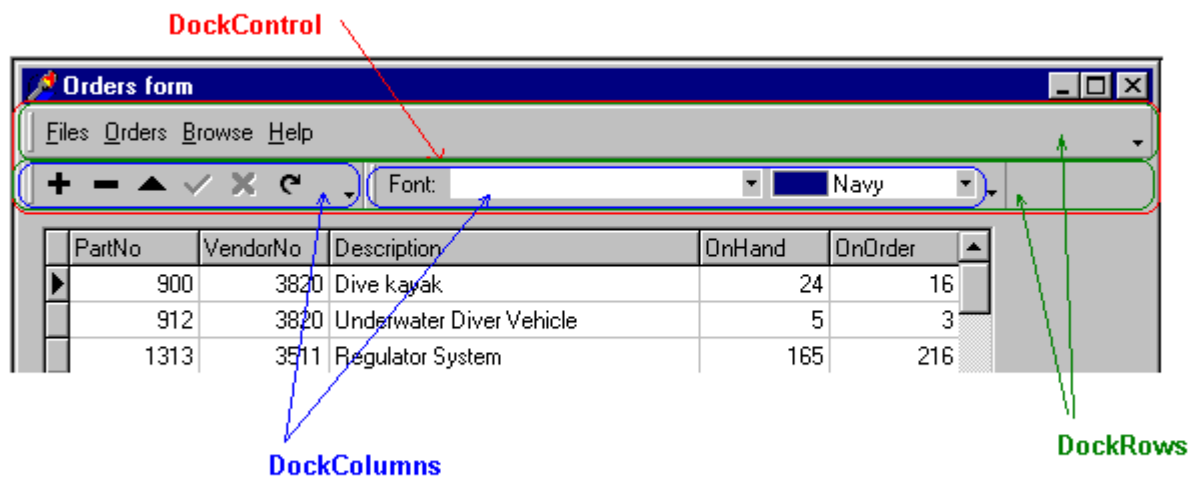
Unit

dxBar

```
TdxDockControl = class(TCustomControl)
```

Description

TdxDockControl draws a dock control, or location of a toolbar. Visually this control appears as a rectangle on the top/bottom/right/left side of a form. One dock control can hold several toolbars within it. A Dock consists of rows; rows in turn consist of columns. A Dock row is described by the TdxDockRow class, a dock column is described by the TdxDockCol class.



Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomControl

TdxDockControl Properties

BarManager	public / read only	RowCount	public / read only
Bars	public / read / write	RowList	public / read only
DockingStyle	public / read / write	Rows	public / read only

BarManager property

read only

Specifies the TdxBarManager to which a TdxDockControl object belongs.

property BarManager: TdxBarManager;

Description

Returns the TdxBarManager which owns a given TdxDockControl object.

Bars property

read / write

property Bars: TdxBars;

Description

Specifies the list of TdxBar objects in order to represent them as toolbars.

See Also

TdxBars

DockingStyle

read write

Determines docking style of the control.

property DockingStyle: TdxBarDockingStyle;

Description

Use this property to define the docking style of a TdxDockControl object.

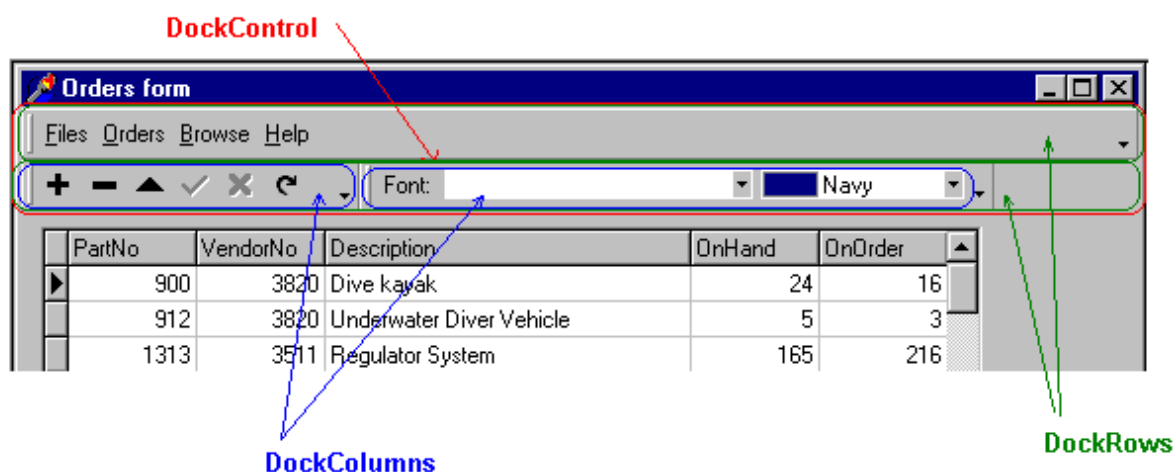
RowCount property**read only**

Returns the number toolbar rows in a dock control.

property RowCount: Integer;

Description

Use the RowCount property to obtain the number of rows which are used to display toolbars within a dock control.

**See Also**

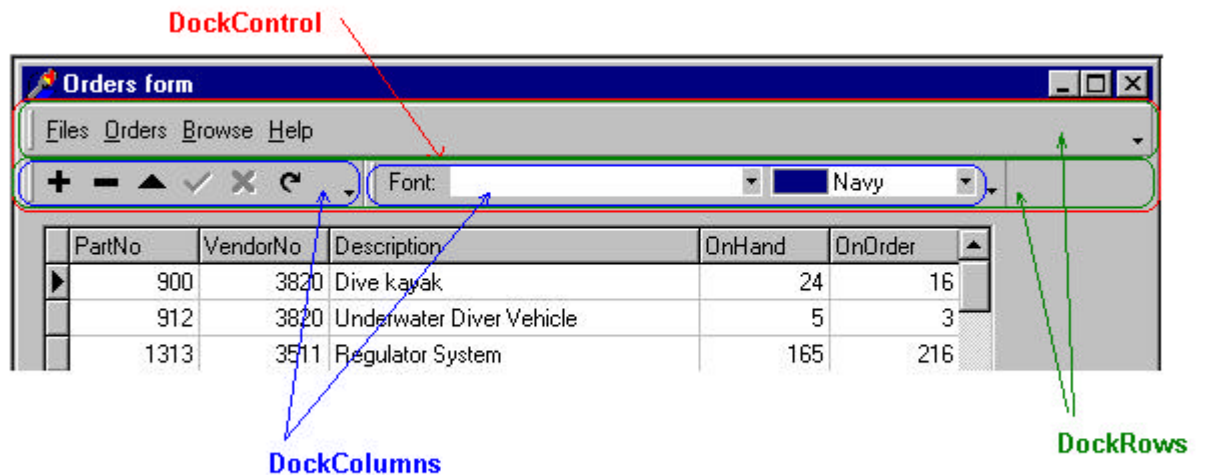
TdxBars;TdxDockControl.Rows

RowList property**read only**

```
property RowList: TList;
```

Description

Toolbar within a dock control can be arranged across several rows and columns. The `RowList` property contains a list of objects of type `TdxDockRow` which are presented as rows of toolbars. Use this property to access the `TdxDockCol` object which determines a toolbar's layout on screen.

**See Also**

TdxDockRow.ColList

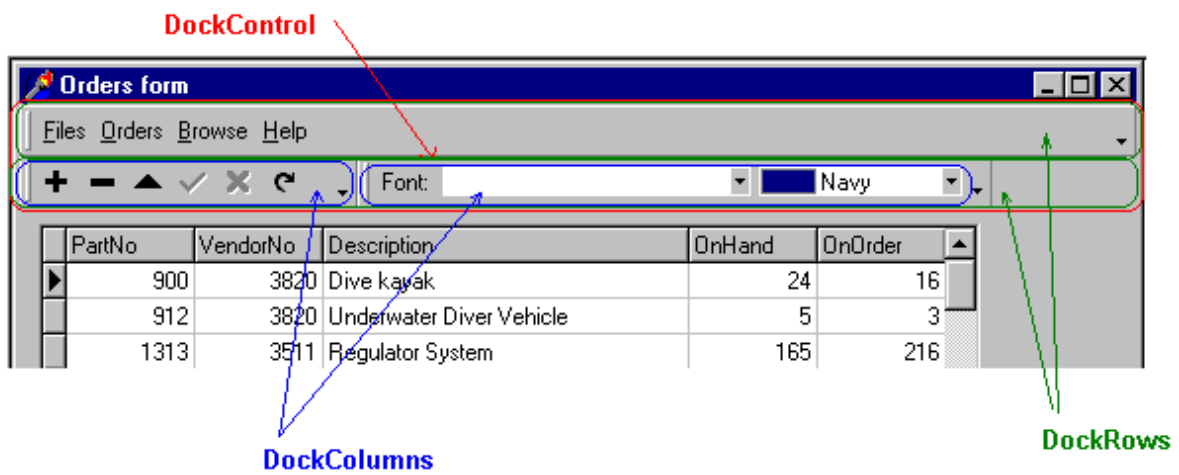
Rows property**read only**

Contains a list of rows within a dock control.

property Rows[Index: Integer]: TdxDockRow;

Description

Toolbars within a dock control can be arranged across several rows and columns. The Rows property contains a list of TdxDockRow objects which represent rows of toolbars. Use this property to access a specific row, specified by the Index parameter. The first row index is 0. The number of rows is stored in the RowCount property.

**See Also**

TdxBars;TdxDockControl.RowCount

TdxDockControl Methods

AlignControls	protected
CreateEx	public

AlignControls method

```
procedure AlignControls(AControl: TControl; var Rect: TRect); override;
```

Description

AlignControls aligns all controls within the dock control rectangle similar to the AlignControls method of the TWinControl class.

CreateEx method

Creates a TdxDockControl instance with a specified docking style.

```
constructor CreateEx(AOwner: TComponent; ABarManager: TdxBarManager;  
ADockStyle: TdxBarDockingStyle;
```

Description

This constructor creates a TdxDockControl object and docks it according to that specified by the ADockStyle parameter. The BarManager property of the new TdxDockControl control is set to ABarManager.

See Also

TdxDockControl

TdxDockCol

TdxDockCol describes columns in TdxDockRow.

Unit

dxBar

```
TdxDockCol = class(TObject);
```

Description

A TdxDockCol object represents a particular bar control in TdxDockControl. Objects of this class are for internal use.

Hierarchy

TObject

TdxDockCol Properties

BarControl	public / read only
DockRow	public / read only
Pos	public / read / write

BarControl property

read only

```
property BarControl: TdxBarControl;
```

Description

Specifies the toolbar that corresponds to a given TdxDockCol object.

See Also

TdxBarControl

DockRow property

read only

```
property DockRow: TdxDockRow;
```

Description

Specifies a TdxDockRow object to which a given TdxDockCol instance belongs.

See Also

TdxDockRow

Pos property

read write

```
property Pos: TPoint;
```

Description

Determines the coordinates of the top left corner of a TdxDockCol object. Pos specifies the point in the dock control coordinates.

TdxDockCol Methods

Create **public**

Create method

constructor Create(ADockRow: TdxDockRow; ABarControl: TdxBarControl);

Description

Creates a TdxDockCol instance. ADockRow and ABarControl parameters specify the TdxDockRow and TdxBarControl objects to which a given instance belongs.

TdxDockRow

The TdxDockRow describes a row in a TdxDockControl.

Unit

dxBar

```
TdxDockRow = class(TObject);
```

Description

Represents a toolbar in a TdxDockControl object. Dock controls can have several rows, for instance, when a user places one toolbar below another (when DockingStyle is dsTop or dsBottom). This object for internal use.

Hierarchy

TObject

TdxDockRow Properties

DockControl	public / read only	ColList	public / read only
ColCount	public / read only	Cols	public / read only

DockControl property

read only

property DockControl: TdxDockControl;

Description

Returns the TdxDockControl object that owns a given TdxDockRow instance.

See Also

TdxDockControl

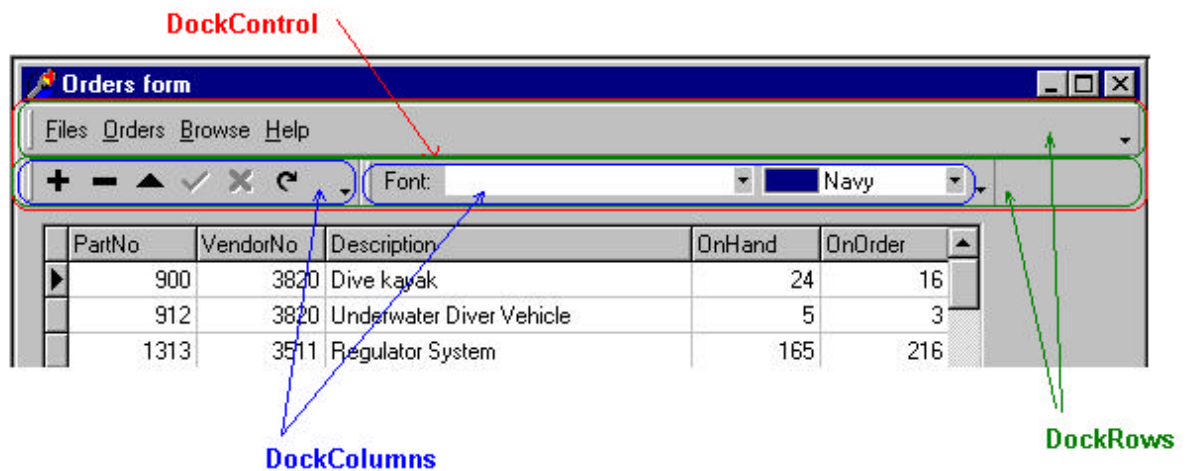
ColCount property**read only**

Returns the number of toolbar columns in a specific row.

property ColCount: Integer;

Description

Use the ColCount property to obtain the number of columns which are used to display toolbars within a row.

**See Also**

TdxDockRow.Cols

ColList property**read only**

property ColList: TList;

Description

Represents a list of TdxDickCol objects belonging to a given TdxDockRow instance, i.e. the toolbars on a row.

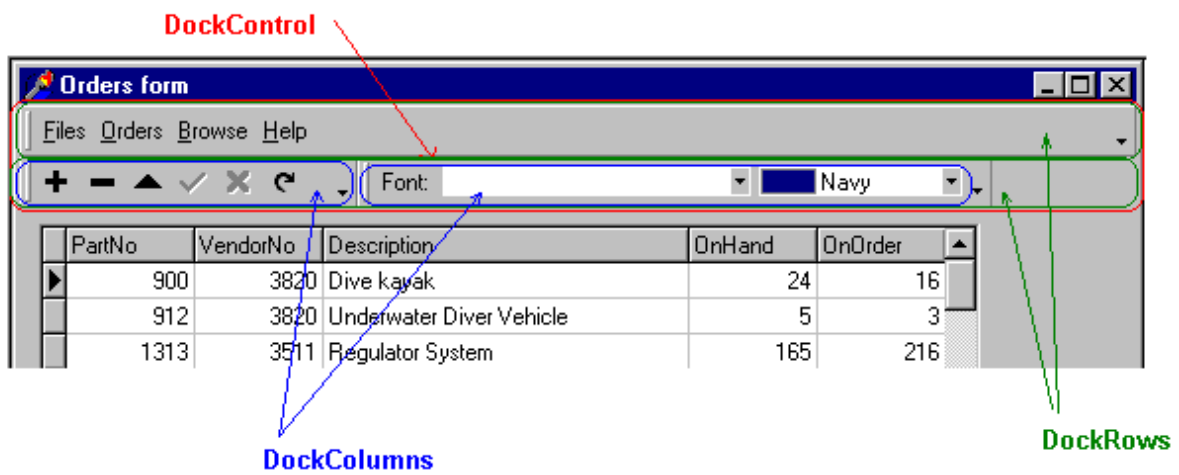
Cols property**read only**

Contains a list of rows within a dock control.

property Cols[Index: Integer]: TdxDockCol;

Description

Toolbars within a dock control can be arranged across several rows and columns. The Cols property contains a list of TdxDockCol objects which represent columns of toolbars. Use this property to access a specific column, specified by the Index parameter. The first column index is 0. The number of columns is stored in the ColCount property.

**See Also**

TdxDockRow.ColCount

TdxDockRow Methods

Create	public
Destroy	public

Create method

constructor Create(ADockControl: TdxDockControl);

Description

Creates a TdxDockRow instance. The ADockControl parameter specifies the TdxDockControl object-owner.

Destroy method

destructor Destroy;

Description

Destroys a TdxDockRow object and the TdxDockCol objects which it owns.

TdxBarDockControl

TdxBarDockControl is an object that represents an external dock control on a form.

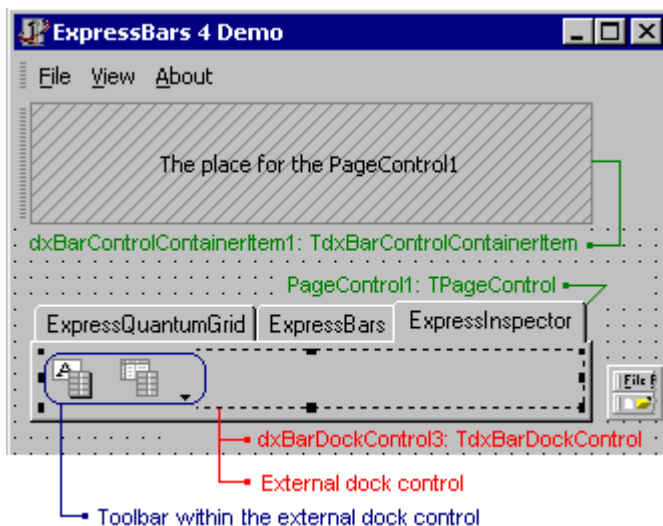
Unit

dxBar

```
TdxBarDockControl = class(TdxDockControl);
```

Description

The ExpressBars allows you to use external dock controls. An external dock control is an addition to internal default dock controls of a bar manager. You can place an external dock control to any position on a form and then embed a toolbar in it. Visually this control appears as a rectangle on the top/bottom/right/left side of the form or floats over it. Each dock control can hold several toolbars within it. TdxBarDockControl is a direct descendant of TdxDockControl and inherits all its functionality.



Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomControl

TdxDockControl

TdxBarDockControl Properties

TdxBarDockControl

Align	published / read / write	SunkenBorder	published / read / write
AllowZeroSizeInDesignTime	published / read / write	UseOwnColor	published / read / write
Color	published / read / write	UseOwnSunkenBorder	published / read / write
ParentColor	published / read / write		

Derived from TdxDockControl

BarManager	published / read only	RowCount	public / read only
Bars	public / read / write	RowList	public / read only
DockingStyle	public / read / write	Rows	public / read only

Align property

read / write

Specifies the layout of a specific external dock control.

type

```
TdxBarDockAlign = (dalNone, dalTop, dalBottom, dalLeft, dalRight);
```

```
property Align: TdxBarDockAlign;
```

Description

Specifies the side of a form to which an external dock control is docked. If the **daNone** setting is enabled, the dock control floats over the form. Possible values include.

Value	Meaning
dalNone	External dock control is not docked and floats over the form.
dalTop	External dock control is docked to the top of the form.
dalBottom	External dock control is docked to the bottom of the form.
dalLeft	External dock control is docked to left side of the form.
dalRight	External dock control is docked to the right side of the form.

AllowZeroSizeInDesignTime property

read / write

Specifies whether to display an empty external dock control at design time.

```
property AllowZeroSizeInDesignTime: Boolean;
```

Description

If the AllowZeroSizeInDesignTime property is set to **True** and there are no toolbars within it, its Height property is set to 0 and it is invisible. If the AllowZeroSizeInDesignTime property is set to **False**, its Height property is equal to 3 pixels, despite the fact it does not contain any toolbar.

The default value is **False**.

Color property**read / write**

Determines the background color within the TdxBarDockControl object area.

property Color;

Description

Use the Color property to read or change the background color within an external dock control. If the TdxBarDockControl's ParentColor property is **True**, then changing the Color property of the TdxBarDockControl's parent automatically changes the TdxBarDockControl's Color property. When the value of the Color property changes, the TdxBarDockControl's ParentColor property is automatically set to **False**.

ParentColor property**read / write**

Specifies where an external dock control obtains its color information.

property ParentColor: **Boolean**;

Description

Set the ParentColor property to **True** to use the same color as its parent control. If the ParentColor property is **False**, TdxBarDockControl uses its own Color property value. When the value of the TdxBarDockControl Color property changes, the ParentColor property is automatically set to **False**.

SunkenBorder property**read / write**

Specifies whether the TdxBarDockControl object has a sunken border.

property SunkenBorder: **Boolean**;

Description

If the SunkenBorder property is set to **True**, the TdxBarDockControl object border is drawn using sunken borders. Otherwise, no border is drawn around the TdxBarDockControl object.

UseOwnColor property**read / write**

Specifies whether a TdxBarDockControl uses its own color.

property UseOwnColor: **Boolean**;

Description

If the UseOwnColor property is set to **True**, the TdxBarDockControl object uses its own color specified by the Color property. If the UseOwnColor property is set to **False**, the TdxBarDockControl object color depends upon the DockColor property of the current TdxBarManager or on the parent control color (if the ParentColor is set to **True**).

The default value is **False**.

UseOwnSunkenBorder property**read / write**

Specifies whether a TdxBarDockControl uses its own value to display a border.

property UseOwnSunkenBorder: **Boolean**;

Description

If the UseOwnSunkenBorder property is set to **True**, the TdxBarDockControl object uses its own border setting specified by the SunkenBorder property. If the UseOwnSunkenBorder property is set to **False**, the TdxBarDockControl border style depends upon the SunkenBorder property of the current TdxBarManager.

The default value is **False**.

TdxBarDockControl Methods

Derived from TdxDockControl

CreateEx

public

TdxBarControl

Unit

dxBar

```
TdxBarControl = class(TCustomdxBarControl);
```

Description

TdxBarControl is inherited from TCustomdxBarControl and visually represents an informational object of type TdxBar.

Hierarchy

```

TObject
  TPersistent
    TComponent
      TControl
        TWinControl
          TCustomControl
            TCustomdxBarControl
```

TdxBarControl Properties

TdxBarControl

Bar	public / read only	MDIButtonHeight	protected / read only
IsDowned	protected / read / write	MDIButtonWidth	protected / read only
Moving	protected / read / write		

Derived from TCustomdxBarControl

BarManager	public / read only	IsCustomizing	public / read only
BkBrush	public / read only	ItemLinks	public / read / write
ChildBar	protected / read / write	MarkState	protected / read / write
ComboBoxArrowWidth	protected / read only	MenuArrowWidth	protected / read only
DockingStyle	protected / read / write	ParentBar	protected / read / write
EditFontHandle	public / read only	SelectedItem	protected / read / write
EditTextSize	public / read only	TextSize	public / read only
IsActive	public / read / write		

Bar property

read only

Specifies the TdxBar object to represent it on screen.

```
property Bar: TdxBar;
```

Description

TdxBarControl represents the TdxBar object on screen. Information for the appearance of TdxBarControl is taken from the TdxBar object specified by the Bar property.

See Also

TdxBar

IsDowned property

read / write

Specifies whether a bar control is unfolded.

```
property IsDowned: Boolean;
```

Description

This property is used to determine if a bar control is active and unfolded. For example, when the mouse is over the submenu control on a toolbar, the sub menu is selected, but not unfolded. Clicking the button forces the sub menu to unfold. When the bar control is unfolded, the IsDowned property is set to True. When the mouse moves to the next subitem and IsDowned is True, the next subitem unfolds automatically. It is useful for main menus because it allows navigation through main menu items with automatic subitems unfolding.

Note: Direct setting of the IsDowned property to True does not unfold the submenu.

Moving property**read / write**

Determines if a bar control is being moved.

property Moving: Boolean;

Description

When Moving equals True, a given bar control is being moved by the user. This property is used internally.

MDIButtonHeight property**read only**

Specifies the height of border icons on a toolbar.

property MDIButtonHeight: integer;

Description

In MDI applications, child forms may be maximized to fit the size of the parent form. In this instance, the main toolbar of the child form merges with the main toolbar of the parent; the border icons of the child form are drawn on the main toolbar of the parent window. The MDIButtonHeight property is used internally to draw border icons of the child form onto the toolbar of the parent form when the child is maximized.

See Also

TdxBarControl.MDIButtonWidth

MDIButtonWidth property**read only**

Specifies the width of border icons on a toolbar.

property MDIButtonWidth: integer;

Description

In MDI applications, child forms may be maximized to fit the size of the parent form. In this instance, the main toolbar of the child form merges with the main toolbar of the parent; the border icons of the child form are drawn on the main toolbar of the parent window. The MDIButtonWidth property is used internally to draw the border icons of the child form onto the toolbar of the parent form when the child form is maximized.

See Also

TdxBarControl.MDIButtonHeight

TdxBarControl Methods

TdxBarControl

BarGetFocus	public	CreateEx	public
BarLostFocus	public	Repaint	public
CaptionChanged	protected		

Derived from TCustomdxBarControl

AdjustSize	protected	HideOnClick	protected
CalcDrawingConsts	protected	IsChildBar	protected
Create	public	ItemAtPos	protected
CreateControls	protected	ItemAtPosEx	protected
Destroy	public	NotHandleMouseMove	protected
DestroyControls	protected	PaintItem	protected
GetFullItemRect	protected	PaintSelectedItem	protected
GetItemRectEx	protected	RepaintBar	public
GetItemRegion	protected	SetKeySelectedItem	protected
HideAll	public	SetMouseSelectedItem	protected

BarGetFocus method

Called each time TdxBarControl receives focus.

```
procedure BarGetFocus(ASelectedItem: TdxBarItemControl);
```

Description

Generally, TdxBarControl receives focus when clicking an item control within it. The SelectedItem property is set to ASelectedItem.

BarLostFocus method

This method is called when TdxBarControl loses focus.

```
procedure BarLostFocus;
```

Description

This method is called each time TdxBarControl loses focus. It sets the SelectedItem property to nil.

CaptionChanged method

This function draws the caption of a control window.

```
procedure CaptionChanged; virtual;
```

Description

This function is called internally when changing a TdxBarControl window caption (if it exists). It is required, for instance, when a user sets the new Caption property of the control.

CreateEx method

Creates a TdxBarControl instance.

```
constructor CreateEx(AOwner: TComponent; ABar: TdxBar); virtual;
```

Description

Creates a TdxBarControl object and sets its Bar property to ABar.

ItemAtPosEx method

Returns an item control by its coordinates.

```
function ItemAtPosEx(Pos: TPoint; var IsBeginGroup, IsFirstPart,  
IsVerticalDirection: Boolean): TdxBarItemControl; override;
```

Description

TdxBarControl ItemAtPosEx overrides the TCustomdxBarControl method. If the Pos specifies a point beyond a toolbar's last item control, the ItemAtPosEx returns the toolbar's last item control, and the IsBeginGroup and IsFirstPart parameters are set to False.

PaintItem method

```
procedure PaintItem(AControl: TdxBarItemControl); override;
```

Description

Paints the item control specified by the AControl parameter. The method by which an item control is represented on screen depends on a bar control's DockingStyle property.

Repaint method

Forces a TdxBarControl to repaint its image on screen.

```
procedure Repaint; override;
```

Description

Calls Repaint to force a toolbar control to repaint its images immediately. Repaint paints the toolbar on screen, but does not calculate toolbar size. Call this method to draw a toolbar when toolbar size has not changed, for example, when only the caption of the toolbar changes.

See Also

TdxBar.LockUpdate

TCustomdxBarControl

The TCustomdxBarControl is the base class for all bar controls. The phrase 'bar controls' refers to all controls that look like a toolbar such as submenus. TdxBarControl and TdxBarSubMenuControl are inherited from TCustomdxBarControl.

Unit

dxBar

```
TCustomdxBarControl = class(TCustomControl);
```

Description

Bar controls are created by the TdxBarItemLinks CreateBarControl method. The type of newly created bar control depends upon source information storage: for TdxBar, it is TdxBarControl, for TdxBarPopupMenu and TCustomdxBarSubItem it is TdxBarSubMenuControl. It is unnecessary for you to call bar controls directly. Use their non-visual representatives instead.

Hierarchy

TObject

 TPersistent

 TComponent

 TControl

 TWinControl

 TCustomControl

TCustomdxBarControl Properties

BarManager	public / read only	IsCustomizing	public / read only
BkBrush	public / read only	ItemLinks	public / read / write
ChildBar	protected / read / write	MarkState	protected / read / write
ComboBoxArrowWidth	protected / read only	MenuArrowWidth	protected / read only
DockingStyle	protected / read / write	ParentBar	protected / read / write
EditFontHandle	public / read only	SelectedItem	protected / read / write
EditTextSize	public / read only	TextSize	public / read only
IsActive	public / read / write		

BarManager property

read only

Specifies the TdxBarManager to which a given bar control belongs.

property BarManager: TdxBarManager read GetBarManager;

Description

This property specifies an object of type TdxBarManager to which a given bar control belongs. BarManager equals the BarManager property in a TdxBarItemLinks class that creates the given bar control.

BkBrush property

read only

property BkBrush: HBRUSH;

Description

Returns the handle of a brush used to draw a bar control. For internal use only.

ChildBar property

read write

Specifies a child bar control (if it exists).

property ChildBar: TCustomdxBarControl;

Description

An example of a child bar control is an unfolded toolbar subitem control. This bar control becomes a parent of the child bar control.

See Also

TCustomdxBarControl.ParentBar

ComboBoxArrowWidth property**read only**

Determines the width of a combobox arrow button.

property ComboBoxArrowWidth: Integer;

Description

The ComboBoxArrowWidth is used internally for drawing a combo box item control on a bar control.

DockingStyle property**read / write**

Determines the current docking style of a bar control.

type

TdxBarDockingStyle = (dsNone, dsLeft, dsTop, dsRight, dsBottom);

property DockingStyle: TdxBarDockingStyle;

Description

DockingStyle specifies the dock to which a bar control is docked. For instance, if DockingStyle is dsTop, the bar control is docked to the top of a form. If DockingStyle is dsNone, the toolbar floats.

EditFontHandle property**read only**

property EditFontHandle: HFONT;

Description

Returns the handle of a font used to write item captions. For internal use only.

EditTextSize property**read only**

property EditTextSize: Integer;

Description

Returns text height on a bar control in pixels. For internal use only.

IsActive property**read / write**

Determines whether a bar control has an active item control.

property IsActive: Boolean;

Description

IsActive returns True if any item control on a bar control has focus. If no focused item control exists, the bar control's IsActive property is False.

IsCustomizing property**read only**

Determines whether the customizing form is active.

property IsCustomizing: Boolean;

Description

IsCustomizing returns True if the TdxBarManager customizing form to which a given bar control belongs is active.

ItemLinks property**read only**

Specifies the TdxBarItemLinks object describing all item links on a given bar control.

property ItemLinks: TdxBarItemLinks;

Description

This property returns a TdxBarItemLinks object describing item links between item controls on a given bar control and the items.

Note: A bar control is created according to the information stored in the ItemLinks property of a corresponding TdxBar object.

See Also

TdxBarItemLinks

MarkState property**read / write**

Determines the state of the quick customizing button.

type

```
TdxBarMarkState = (msNone, msSelected, msPressed);
```

```
property MarkState: TdxBarMarkState;
```

Description

The quick customizing button is drawn to the right of a toolbar and looks like a dropdown button.



quick customizing button

The quick customizing button activates the **Add or Remove Buttons** and the list of items that did not fit on the toolbar. The MarkState property returns the state of the button:

Value	Meaning
msNone	The button is neither selected nor pressed.
msSelected	The button is selected, i.e the mouse is above the mouse.
msPressed	The button is pressed.

MenuArrowWidth property**read only**

Determines the width of an arrow button.

```
property MenuArrowWidth: Integer;
```

Description

An Arrow button is drawn next to a control's caption if the item control is in a menu. For example, the arrow button is drawn when the submenu is placed onto another submenu.

ParentBar property**read / write**

Determines the parent bar control of a given bar control.

```
property ParentBar: TCustomdxBarControl;
```

Description

The ParentBar property is generally used for submenus to determine the bar control to which a submenu belongs. For example, the open toolbar subitem is represented by the TdxBarSubMenuControl; the ParentBar for this TdxBarSubItemControl is the bar control for the toolbar to which the subitem belongs.

See Also

TCustomdxBarControl.ChildBar

SelectedItem property**read / write**

Specifies the current item control.

property SelectedItem: TdxBarItemControl;

Description

Generally, a bar control contains several item controls and only one of them may be selected at a time. This property contains the currently selected item control of the bar control. An Item control is selected when the mouse pointer hovers over it and its Enabled property is True.

See Also

TCustomdxBarControl.SetKeySelectedItem; TCustomdxBarControl.SetMouseSelectedItem

TextSize property**read only**

property TextSize: Integer;

Description

Used internally to correctly draw a bar control. TextSize depends on the vertical size of '0' in the current device context.

TCustomdxBarControl Methods

AdjustSize	protected	HideOnClick	protected
CalcDrawingConsts	protected	IsChildBar	protected
Create	public	ItemAtPos	protected
CreateControls	protected	ItemAtPosEx	protected
Destroy	public	NotHandleMouseMove	protected
DestroyControls	protected	PaintItem	protected
GetFullItemRect	protected	PaintSelectedItem	protected
GetItemRectEx	protected	RepaintBar	public
GetItemRegion	protected	SetKeySelectedItem	protected
HideAll	public	SetMouseSelectedItem	protected

AdjustSize method

Adjusts the size of a control according to its Left, Top, Height, and Width properties.

procedure AdjustSize; **override**;

Description

This method is derived from TCustomControl and performs no actions. Override it to perform additional actions when adjusting item controls. This method is only available in Delphi 4™.

CalcDrawingConsts method

Performs calculations to correctly draw a toolbar.

procedure CalcDrawingConsts; **virtual**;

Description

This method is used internally. CalcDrawingConsts is called when it is necessary to correctly draw a bar control on screen. It is necessary, for example, when the TdxBarManager Font property changes.

Create method

Creates a TCustomdxBarControl instance.

constructor Create(AOwner: TComponent); **override**;

Description

Creates and initializes a TCustomdxBarControl object. By default, the Visible property is set to False. Descendants can override this method to perform special actions upon creation.

CreateControls method

Creates visible item controls on a bar control.

```
procedure CreateControls; virtual;
```

Description

This method creates visible item controls on a bar control according to the `VisibleItems` property. `CreateControls` is called when creating a bar control and from the `RefreshBar` method.

See Also

TCustomdxBarControl.DestroyControls; TCustomdxBarControl.RepaintBar

Destroy method

```
destructor Destroy; override;
```

Description

Destroys a `TCustomdxBarControl` instance. Descendants can override this method to perform specific actions upon destroy.

DestroyControls method

Destroys item controls belonging to a given bar control.

```
procedure DestroyControls; virtual;
```

Description

This method destroys item controls which belong to a given bar control. `DestroyControls` is called when destroying a toolbar or from the `RepaintBar` method.

See Also

TCustomdxBarControl.CreateControls; TCustomdxBarControl.RepaintBar

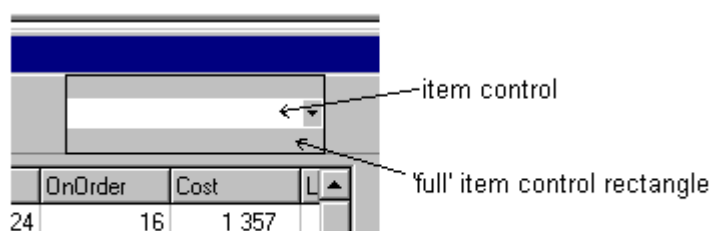
GetFullItemRect method

GetFullItemRect returns the 'full' rectangle occupied by an item control on a bar control.

```
function GetFullItemRect(Item: TdxBarItemControl): TRect; virtual;
```

Description

GetFullItemRect is used when processing drag-and-drop events. The 'full' rectangle includes an item control's rectangle and the area surrounding it. The following image describes the meaning of 'full' rectangle:



The resultant rectangle is specified in bar control coordinates. The Item parameter defines the item control.

See Also

TCustomdxBarControl.GetItemRegion; TdxBarItemControl

GetItemRectEx method

Returns the rectangle in the bar control coordinates the specified item control occupies on the bar control.

```
function GetItemRectEx(Item: TdxBarItemControl; IsBeginGroup: Boolean):  
TRect; virtual;
```

Description

The IsBeginGroup parameter specifies whether an item control starts a group. If True, the resulting rectangle includes the item control's rectangle and (if the item control starts the group) the group separator's rectangle. For TCustomdxBarControl this parameter is not taken into consideration, but descendants use it to access proper rectangle's of item controls.

See Also

TCustomdxBarControl.GetItemRegion; TdxBarItemControl; TdxBarItemLink.BeginGroup

GetItemRegion method

Returns the region occupied by an item control on a bar control.

```
function GetItemRegion(Item: TdxBarItemControl): HRGN; virtual;
```

Description

Returns the region an item control occupies on a bar control. The item control is specified by the Item parameter. If an item control is the first in a group, the result includes an item control's region and that of the group separator.

See Also

TCustomdxBarControl.GetFullItemRect; *TCustomdxBarControl.GetItemRectEx*;
TdxBarItemLink.BeginGroup

HideAll method

Closes a given bar control.

```
procedure HideAll; virtual;
```

Description

This method closes and deactivates the current bar control and hides all open controls linked to it. For instance, if a set of open submenus exist, HideAll closes them (both the child and parent).

HideOnClick method

Determines whether a control is hidden on mouse button press events.

```
function HideOnClick: Boolean; virtual;
```

Description

For toolbars, the HideOnClick method always returns False – toolbars remain visible upon clicking item controls. For sub items, this method returns True – sub items are hidden upon pressing any button within them.

ItemAtPos method

Returns an item control by its coordinates.

```
function ItemAtPos(Pos: TPoint): TdxBarItemControl; virtual;
```

Description

Returns an item control if its rectangle includes a point specified by the Pos parameter. This function is used during drag-and-drop procedures and when displaying an item's hint.

ItemAtPosEx

Determines an item control by specified coordinates.

```
function ItemAtPosEx(Pos: TPoint; var IsBeginGroup, IsFirstPart,  
IsVerticalDirection: Boolean): TdxBarItemControl; virtual;
```

Description

This procedure is used internally to access an item control specified by the coordinates in the Pos parameter. If an item control is the first in the group, then this function sets the IsBeginGroup parameter to True. IsFirstPart and IsVerticalDirection are used during drag-and-drop event processing. If the point specified by Pos belongs to the first part of the item control, the IsFirstPart is set to True. IsVerticalDirection is set to True if the bar control has a vertical layout, that is, a bar control's DockingStyle is dsLeft or dsRight.

IsChildBar method

Checks if a given bar control is the child of another bar control.

```
function IsChildBar(Value: TCustomdxBarControl): Boolean; virtual;
```

Description

Returns True if a bar control has a parent bar control. An example of a child bar control is an active submenu of a toolbar.

See Also

TCustomdxBarControl.ChildBar; TCustomdxBarControl.ParentBar

NotHandleMouse method

```
function NotHandleMouseMove(P: TPoint): Boolean; virtual;
```

Description

For internal use only. NotHandleMouseMove determines whether the toolbar traces mouse movement if the form is not focused. The P parameter determines mouse coordinates.

PaintItem method

```
procedure PaintItem(AControl: TdxBarItemControl); virtual;
```

Description

TCustomDxBarControl PaintItem is used for drawing a particular item control on a bar control. The TCustomDxBarControl PaintItem method performs no actions. Descendants can override it to draw the item control specified by the Item parameter on a bar control.

PaintSelectedItem method

Repaints the new and previously selected item controls.

```
procedure PaintSelectedItem(OldSelectedItem: TdxBarItemControl); virtual;
```

Description

This method is called when a selected item changes to another item. OldSelectedItem specifies the previous item. PaintSelectedItem first repaints the OldSelectedItem and then paints the currently selected bar item control.

See Also

TCustomdxBarControl.SelectedItem

RepaintBar method

Repaints a TCustomdxBarControl object.

```
procedure RepaintBar; virtual;
```

Description

RepaintsBar recreates all item controls, calculates new toolbar size and draws the toolbar. Use this method to redraw a bar control after adding new item controls or when changing bar control size. For instance, a bar control's size changes when changing the TdxBarManager Font or LargeIcons properties.

SetKeySelectedItem method

Selects an item control when using a keyboard.

```
procedure SetKeySelectedItem(Value: TdxBarItemControl); virtual;
```

Description

SetKeySelectedItem is called when a user selects an item control via a keyboard. This method sets the SelectedItem property to an item, whose linked item control is specified by the Value parameter. If your need to select an item control via program code, you should use SetKeySelectedItem instead of the SetMouseSelectedItem method.

See Also

TCustomdxBarControl.SelectedItem; *TCustomdxBarControl.SetMouseSelectedItem*

SetMouseSelectedItem method

Selects an item control when using a mouse.

```
procedure SetMouseSelectedItem(Value: TdxBarItemControl); virtual;
```

Description

SetMouseSelectedItem is called when an item control is selected via a mouse, when the mouse cursor enters a control's rectangle.

See Also

TCustomdxBarControl.SelectedItem; TCustomdxBarControl.SetMouseSelectedItem

TdxBarSubMenuControl

TdxBarSubMenuControl represents submenus on screen.

Unit

dxBar

```
TdxBarSubMenuControl = class(TCustomdxBarControl)
```

Description

TdxBarSubMenuControl visually represents objects of type TdxBarPopupMenu and TCustomdxBarSubItem; that is, popup menus and sub items on toolbars. TdxBarSubMenuControl is inherited from TCustomdxBarControl and thus it is a bar control.

Generally, the TCustomdxBarControl ChildBar property is an object of type TdxBarSubMenuControl.

Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomControl

TCustomdxBarControl

TdxBarSubMenuControl Properties

TdxBarSubMenuControl

MarkSize	protected / read only	OwnerHeight	public / read / write
MaxVisibleCount	protected / read only	TopIndex	protected / read / write
OwnerWidth	public / read / write		

Derived from TCustomdxBarControl

BarManager	public / read only	IsCustomizing	public / read only
BkBrush	public / read only	ItemLinks	public / read / write
ChildBar	protected / read / write	MarkState	protected / read / write
ComboBoxArrowWidth	protected / read only	MenuArrowWidth	protected / read only
DockingStyle	protected / read / write	ParentBar	protected / read / write
EditFontHandle	public / read only	SelectedItem	protected / read / write
EditTextSize	public / read only	TextSize	public / read only
IsActive	public / read / write		

MarkSize property

read only

```
property MarkSize: Integer;
```

Description

Used internally. Returns the indicator height which appears in a sub menu when rarely used menu items are hidden.

MaxVisibleCount property

read only

Returns the maximum number of available and visible menu items.

```
property MaxVisibleCount: Integer;
```

Description

Returns the maximum number of visible menu items. If the number of menu items is greater than MaxVisibleCounts, the up or down arrow appears at the top/bottom of the menu.

OwnerHeight property**read / write**

Specifies an item control's height which activates TdxBarSubMenuControl.

property OwnerHeight: Integer;

Description

Specifies the height of an item control which activates TdxBarSubMenuControl in order to help avoid overlapping this particular item control. For example, if TdxBarSubMenuControl is displayed upon a button click event, OwnerHeight equals button height to avoid button area overlaps by TdxBarSubMenuControl.

OwnerWidth property**read / write**

Determines an item control's width which activates TdxBarSubMenuControl active.

property OwnerWidth: Integer;

Description

Specifies the width of an item control which activates TdxBarSubMenuControl in order to help avoid overlapping this particular item control. For example, if TdxBarSubMenuControl is displayed upon a button click event, OwnerWidth equals button width to avoid button area overlaps by TdxBarSubMenuControl.

TopIndex property**read / write**

Specifies the index of the top visible item.

property TopIndex: Integer;

Description

Specifies the index of the top visible menu item. TopIndex is the index of the top item in the VisibleItems array of the ItemLinks property.

TdxBarSubMenuControl Methods

TdxBarSubMenuControl

DoCloseUp	protected	Show	public
DoPopup	protected	UpArrowExists	protected
DownArrowExists	protected	VisibleCount	protected
Hide	public		

Derived from TCustomdxBarControl

AdjustSize	protected	HideOnClick	protected
CalcDrawingConsts	protected	IsChildBar	protected
Create	public	ItemAtPos	protected
CreateControls	protected	ItemAtPosEx	protected
Destroy	public	NotHandleMouseMove	protected
DestroyControls	protected	PaintItem	protected
GetFullItemRect	protected	PaintSelectedItem	protected
GetItemRectEx	protected	RepaintBar	public
GetItemRegion	protected	SetKeySelectedItem	protected
HideAll	public	SetMouseSelectedItem	protected

DoCloseUp method

This procedure generates an OnCloseUp event.

```
procedure DoCloseUp; dynamic;
```

Description

Override this method to perform special operations when closing a menu without writing a TdxBarPopupMenu OnCloseUp event handler.

See Also

TdxBarSubMenuControl.OnCloseUp

DoPopup method

```
procedure DoPopup; dynamic;
```

Description

Generates an OnPopup event. Override this method to perform special operations when activating a menu without writing a TdxBarPopupMenu OnPopup event handler.

See Also

TdxBarSubMenuControl.OnPopup

DownArrowExists method

Determines if a menu includes a down arrow.

```
function DownArrowExists: Boolean;
```

Description

Returns True, if a menu includes a down arrow to scroll through items when their numbers are greater than that of the VisibleCount property.

Note: *This method is protected, but not virtual. Descendants can use it, but not override.*

See Also

TdxBarSubMenuControl.VisibleCount

Hide method

Makes TdxBarSubMenuControl invisible.

```
procedure Hide;
```

Description

This method hides TdxBarSubMenuControl and its child bar controls. Though the bar control is hidden, its properties and methods are still available.

See Also

TCustomdxBarControl.HideAll; TdxBarSubMenuControl.Show

Show method

Shows a TdxBarSubMenuControl.

```
procedure Show;
```

Description

This method first generates an OnPopup event and then displays TdxBarSubMenuControl using animation effects which are specified within the TdxBarManager MenuAnimations property.

See Also

TCustomdxBarControl.HideAll

UpArrowExists method

Determines if a menu includes an up arrow.

```
function UpArrowExists: Boolean;
```

Description

Returns True, if a menu includes an up arrow to scroll through items when their numbers are greater than that of the VisibleCount property.

Note: *This method is protected, but not virtual. Descendants can use it, but not override.*

See Also

TdxBarSubMenuControl.VisibleCount

VisibleCount method

Returns the number of a menu's visible items.

```
function VisibleCount: Integer;
```

Description

Returns the number of a menu's items displayed by TdxBarSubMenuControl. If the number of all menu items is greater than that of VisibleCount, the menu displays an up/down arrow at the top or bottom of the menu.

Note: *This method is protected, but not virtual. Descendants can use it, but not override.*

TdxBarSubMenuControl Events

OnCloseUp	protected
OnPopup	protected

OnCloseUp event

This event occurs before closing TdxBarSubMenuControl.

property OnCloseUp: TNotifyEvent;

Description

Use OnCloseUp to perform special actions when closing a menu without writing an OnClose event handler. The handler specified by OnCloseUp might test certain items to make certain they have the valid contents before allowing the menu to close.

OnPopup event

Occurs just before a popup menu appears.

property OnPopup: TNotifyEvent;

Description

Write an OnPopup event handler to perform specific actions before the appearance of a popup menu. For instance, use an OnPopup event handler to set properties of TdxBarSubMenuControl.

TdxBarDBNavigator

TdxBarDBNavigator is used to navigate data in a dataset and perform operations on data including events like insert and post via toolbar buttons.

Unit

dxBarDBNav

```
TdxBarDBNavigator = class(TComponent);
```

Description

TdxBarDBNavigator is similar to the Delphi TDBNavigator object. TdxBarDBNavigator button type, placement, and look are determined by an application. In addition, you may create links between TdxBarManager items and a dataset's state using specific conditions.

Note: *TdxBarManager can work with only one TdxBarDBNavigator object.*

Hierarchy

TObject

TPersistent

TComponent

TdxBarDBNavigator Properties

BarManager	published / read / write	DataSource	published / read / write
Buttons	protected / read / write	DBCCheckLinks	published / read / write
CategoryName	published / read / write	VisibleButtons	published / read / write
ConfirmDelete	published / read / write		

BarManager property

read / write

```
property BarManager: TdxBarManager;
```

Description

Specifies the TdxBarManager to which a given TdxBarDBNavigator belongs. TdxBarManager supports one TdxBarDBNavigator object only.

See Also

TdxBarManager

Buttons property

read / write

Contains a list of available buttons.

```
property Buttons: array[TdxBarDBNavButtonType] of TdxBarDBNavButton;
```

Description

This property determines the available TdxBarDBNavigator buttons, that is, the buttons that appear in the item list in the customizing form.

See Also

TdxBarDBNavButtonType

CategoryName property**read / write**

Defines the TdxBarManager category to which TdxBarDBNavigator buttons belong.

property CategoryName: string;

The CategoryName specifies the name of the default TdxBarManager category for TdxBarDBNavigator buttons.

See Also

TdxBarManager.Categories

ConfirmDelete property**read / write**

Determines whether to warn users before record deletion.

property ConfirmDelete: Boolean;

If ConfirmDelete equals True, pressing the TdxBarDBNavigator Delete button activates a confirmation message box; Else the record is deleted without any user notification.

DataSource property**read / write**

Refers to a TDataSource object accessed by TdxBarDBNavigator.

property DataSource: TDataSource;

Description

DataSource connects TdxBarDBNavigator to a dataset. Once set, TdxBarDBNavigator buttons can work directly with dataset records.

DBCheckLinks property**read / write**

Contains the list of item links between items and a dataset's state.

property DBCheckLinks: TdxBarDBCheckLinks;

Description

If there is a link between an item and a dataset's state, the item Enabled property is changed to the dataset state accordingly. One link is defined by the TdxBarDBCheckLink object; the TdxBarDBCheckLinks object contains a list of these links.

See Also

TdxBarDBCheckLinks

VisibleButtons property**read / write**

Defines the button collection shown in the item list of the customizing form.

type

TdxBarDBNavButtonTypes = **set of** TdxBarDBNavButtonType;

property VisibleButtons : TdxBarDBNavButtonTypes;

This property contains a set of button types created in a TdxBarManager category. The visible button items can be placed on any TdxBarManager toolbar or TdxBarPopupMenu. You can move the button to any TdxBarManager category. Use the TdxBarManager CategoryItemsVisible to set the visibility of different buttons to restrict user actions on data. You would make use of this when, for instance, some users can add, delete, and edit records; while others can only read data.

See Also

TdxBarDBNavButtonType;TdxBarManager.CategoryItemsVisible

TdxBarDBNavigator Methods

Create	public
Destroy	public

Create method

Creates a TdxBarDBNavigator object.

constructor Create(AOwner: TComponent); **override;**

This method creates and initializes an instance of the TdxBarDBNavigator type. AOwner specifies the form that owns a given TdxBarDBNavigator. The Create method sets the BarManager property to the TdxBarManager of a form, loads the default value for the CategoryName from resources and adds the given TdxBarDBNavigator object to the TdxBarDBNavigator list.

Destroy method

Destroys a TdxBarDBNavigator instance.

destructor Destroy; **override;**

Before calling the inherited method, the Destroy method frees the VisibleButtons list and deletes a given TdxBarDBNavigator object from the TdxBarDBNavigator list.

TdxBarDBNavigatorList

Contains a list of all TdxBarDBNavigator application objects.

Unit

dxBarDBNav

```
TdxBarDBNavigatorList = class (TObject);
```

Description

This object with the list of all TdxBarDBNavigator application objects is similar to the TdxBarManagerList containing the list of all TdxBarManager application objects.

There is a global object of the TdxBarDBNavigatorList type named dxBarDBNavigatorList.

Hierarchy

TObject

TdxBarDBNavigatorList Properties

Count	public / read only
Items	public / read only

Count property

read only

Returns the total number of the TdxBarDBNavigator objects in a list.

property Count: Integer;

Description

Use this property to get a TdxBarDBNavigator number in the list. This is a ReadOnly property. The index of the first TdxBarDBNavigator in the list is 0, the index of the last TdxBarDBNavigator in the list is one less than the number returned by the Count property.

Items property

read only

Contains a TdxBarDBNavigator object list.

property Items[Index: Integer]: TdxBarDBNavigator;

Use this property to get a particular TdxBarDBNavigator object from the list by index. The index numeration begins with zero.

See Also

TdxBarNavigator

TdxBarDBNavigatorList Methods

TdxBarDBNavigatorList

GetDBNavigatorByBarManager **public**

GetFirstFreeBarManager **public**

GetDBNavigatorByBarManager method

Returns a TdxBarDBNavigator object which belongs to the TdxBarManager specified by the ABarManager parameter.

```
function GetDBNavigatorByBarManager(ABarManager: TdxBarManager):  
TdxBarDBNavigator;
```

Searches for a TdxBarDBNavigator with the BarManager property equal to the ABarManager parameter and returns (if it exists) the TdxBarDBNavigator. If matches are not found, the function returns nil.

GetFirstFreeBarManager method

Returns the first 'free' TdxBarManager.

```
function GetFirstFreeBarManager: TdxBarManager;
```

Description

The 'free' TdxBarManager is a TdxBarManager object that does not have a TdxBarDBNavigator connected to it. If there is no such TdxBarManager, the function returns nil.

TdxBarDBNavButton

This class represents a TdxBarDBNavigator button.

Unit

dxBarDBNav

```
TdxBarDBNavButton = class(TdxBarButton);
```

Description

TdxBarDBNavButton is derived from the TdxBarButton. The TdxBarDBNavigator button provides several properties specific to data-aware components. An item of the TdxBarDBNavButton type performs specific actions on datasets such as navigating through the dataset records, editing records, data refresh, etc.

Hierarchy

TObject

TPersistent

TComponent

TdxBarItem

TdxBarButton

TdxBarDBNavButton Properties

TdxBarDBNavButton

BarDBNavigator	public / read only
NavButton	published / read / write

Derived from TdxBarButton

AllowAllUp	published / read / write	DropDownEnabled	published / read / write
ButtonStyle	published / read / write	DropDownMenu	published / read / write
CloseSubMenuOnClick	published / read / write	Lowered	published / read / write
GroupIndex	published / read / write	PaintStyle	published / read / write
Down	published / read / write		

Derived from TdxBarItem

ActuallyVisible	public / read / write	Glyph	published / read / write
Action	published / read / write	HelpContext	published / read / write
ActionLink	protected / read / write	Hint	published / read / write
Align	published / read / write	ImageIndex	published / read / write
BarManager	public / read / write	Index	public / read / write
Caption	published / read / write	LinkCount	public / read / write
Category	published / read / write	Links	public / read / write
ClickItemLink	public / read / write	ShortCut	published / read / write
CurlItemLink	public / read / write	UnclickingAfterDoing	published / read / write
Data	public / read / write	Visible	published / read / write
Description	published / read / write	VisibleForUser	public / read / write
Enabled	published / read / write		

TdxBarDBNavButton.BarDBNavigator property

read only

Specifies the TdxBarDBNavigator to which a given button item belongs.

```
property BarDBNavigator: TdxBarDBNavigator;
```

Description

BarDBNavigator specifies the TdxBarDBNavigator managing a given button.

NavButton property**read / write**

Determines button type.

property NavButton: TdxBarDBNavButtonType;

Description

This property specifies button type. The type of a button defines the action performed on the TdxBarDBNavigator DataSource DataSet.

TdxBarDBNavButton Methods

TdxBarDBNavButton

Destroy	public
DoClick	public

Derived from TdxBarButton

ButtonStyleChanged	protected	DoDropDown	protected
CanChangePaintStyle	protected	DropDownEnabledChanged	protected
DownChanged	protected	PaintStyleChanged	protected

Derived from TdxBarItem

ActionChange	protected	GlyphChanged	protected
BarManagerChanged	protected	HasAccel	protected
CanClicked	protected	HasParent	public
CanSelected	protected	HotGlyphChanged	protected
CaptionChanged	protected	LargeGlyphChanged	protected
Click	public	SetParentComponent	protected
Create	public	ShortCutChanged	protected
Destroy	public	Update	protected
DirectClick	public	UpdateEx	protected
DoClick	public	UseHotImages	protected
EnabledChanged	protected	UseLargeImages	protected
GetActionLinkClass	protected	VisibleChanged	protected
GetParentComponent	public		

Destroy method

Destroys an instance of the TdxBarDBNavButton type.

```
destructor Destroy; override;
```

Description

Before calling the inherited method, Destroy removes reference to itself from the TdxBarDBNavigator Buttons and VisibleButtons properties.

DoClick method

Processes mouse click events.

```
procedure DoClick; override;
```

Description

After calling the inherited method, DoClick generates the OnClick event and then performs specific actions on a dataset. Button type determines the action performed on the dataset.

TdxBarDBNavButton Events

Derived from TdxBarItem

OnClick	published / read / write
OnCreate	protected / read / write
OnDestroy	published / read / write

TdxBarPopupMenu

The TdxBarPopupMenu component is designed to help you create popup menus from items stored in a TdxBarManager.

Unit

dxBar

```
TdxBarPopupMenu = class(TComponent);
```

Description

Use this component when you wish to create a popup menu with links to items of a particular TdxBarManager. TdxBarManager is specified by the TdxBarPopupMenu BarManager property. TdxBarManager and the popup menu may belong to different forms; therefore, you may use the TdxBarPopupMenu on a form which does not include a TdxBarManager.

Hierarchy

TObject

 TPersistent

 TComponent

TdxBarPopupMenu Properties

BarManager	published / read / write	ItemLinks	published / read / write
BarSize	published / read / write	UseOwnFont	published / read / write
Font	published / read / write	UseRecentItems	published / read / write

BarManager property

read / write

Represents the TdxBarManager object which owns a given TdxBarPopupMenu object.

property BarManager: TdxBarManager;

Description

This property is an object of type TdxBarManager. TdxBarManager items are used for popup menu generation.

See Also

TdxBarManager

BarSize property

read / write

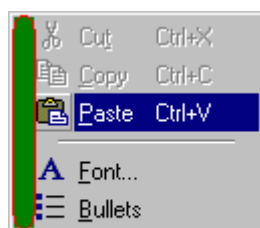
Determines the size of a bar at the left side of a popup menu.

property BarSize: Integer;

Description

A bar is drawn at the left side of a popup menu window. Use the BarSize property to set the size of this bar in pixels. The default value for this property is 0.

Use the OnPaintBar event to draw this bar based upon your specific needs.



BarSize = 10

See Also

TdxBarPopupMenu.OnPaintBar

Font property**read / write**

Specifies the font of a popup menu.

property Font: TFont;

Description

Determines the font used for all items which belong to a given popup menu.

Note: *If it is modified, the UseOwnFont property is True.*

See Also

TdxBarPopupMenu.UseOwnFont

ItemLinks property**read / write**

Determines a TdxBarItemLinks array.

property ItemLinks: TdxBarItemLinks;

Description

This property is an array of item links to TdxBarManager items. The Popup method uses the value of this property in order to create a bar control of type TdxBarSubMenuControl.

UseOwnFont property**read / write**

Specifies the font used to draw a popup menu.

property UseOwnFont: Boolean;

Description

If the UseOwnFont property is False (by default), then use the font parameter set for a given popup menu from the TdxBarManager.Font property. Else, use the font parameter set from the TdxBarPopupMenu.Font property.

See Also

TdxBarManager.Font; TdxBarPopupMenu.Font

UseRecentItems property**read / write**

Determines whether new features are used for a given popup menu.

property UseRecentItems: Boolean;

Description

If set to True (by default), only the most frequently used items are displayed. Else, all items will be displayed in a given popup menu.

TdxBarPopupMenu Methods

Create	public	PopupEx	public
Destroy	public	PopupFromCursorPos	public
Popup	public		

Create method

Creates a TdxBarPopupMenu instance.

```
constructor Create(AOwner: TComponent); override;
```

Description

If a form includes TdxBarManager, the BarManager property is set to the form's TdxBarManager object; Else, the BarManager property is set to the first available TdxBarManager object in the TdxBarManagerList list.

Destroy method

```
destructor Destroy; override;
```

Description

This method destroys a TdxBarPopupMenu instance. Descendants can override this method to perform special actions when destroying.

Popup method

Displays a popup menu on screen.

```
procedure Popup(X, Y: Integer);
```

Description

This method is used to activate a popup menu at a screen point with (X,Y) coordinates. The coordinates are relative to the desktop's upper left corner.

See Also

TdxBarPopupMenu.PopupEx; TdxBarPopupMenu.PopupFromCursorPos

PopupEx method

Displays a popup menu using additional parameters.

```
procedure PopupEx(X, Y, AOwnerWidth, AOwnerHeight: Integer; AShowAnimation: Boolean);
```

Description

This method performs the same action as the Popup method but uses additional parameters. AOwnerWidth and AOwnerHeight are area sizes used to avoid overlapping this area by a popup menu. The bottom right corner of the area is defined as (X, Y). If the popup menu does not fit within the desktop, it shifts in relation to the AOwnerWidth and AownerHeight parameters.

Upon shifting, the popup menu will not overlap the area specified by X, Y, AOwnerWidth, AOwnerHeight. If one of the parameters is 0, it is ignored. By assigning AShowAnimation to False, you may cancel animation effects when calling the popup menu. Animation effects are specified within the TdxBarManager MenuAnimations property.

See Also

TdxBarPopupMenu.Popup; TdxBarPopupMenu.PopupFromCursorPos

PopupFromCursorPos method

```
procedure PopupFromCursorPos;
```

Description

Displays a PopupMenu at the current cursor position. This procedure calls the Popup method and supplies it with mouse pointer coordinates.

See Also

TdxBarPopupMenu.Popup; TdxBarPopupMenu.PopupEx

TdxBarPopupMenu Events

OnCloseUp	published
OnPaintBar	published
OnPopup	published

OnCloseUp event

Occurs when closing a popup menu.

property OnCloseUp: TNotifyEvent;

Description

This event occurs after closing a popup menu and destroying its handle. Write a OnCloseUp event handler to perform special actions when a menu is closed.

OnPaintBar event

Occurs when drawing a bar at the left side of a popup menu.

type

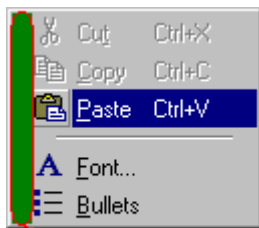
```
TdxBarPaintSubMenuBarEvent = procedure(Sender: TObject; Canvas: TCanvas;  
const R: TRect) of object;  
property OnPaintBar: TdxBarPaintSubMenuBarEvent;
```

Description

Write an OnPaintBar event handler to draw the left side bar based upon your specific needs. The Canvas parameter determines canvas for drawing. The R parameter specifies the rectangle for drawing.

The following example allows you to draw this bar and fill it with a green color.

```
Canvas.Brush.Color := clGreen;  
Canvas.FillRect(R);
```



BarSize = 10

See Also

TdxBarPopupMenu.BarSize

OnPopup event

```
property OnPopup: TNotifyEvent;
```

Description

This event occurs before displaying a menu on screen. Write an OnPopup event handler to perform special actions just before a popup menu appears.

TdxBarPopupMenuLink

Unit

dxBar

```
TdxBarPopupMenuLink = class(TCollectionItem);
```

Description

TdxBarPopupMenuLink is a separate menu link which connects a control on a form with a popup menu of type TdxBarPopupMenu. This popup menu activates upon pressing the right mouse button on a control.

Hierarchy

TObject

TPersistent

TCollectionItem

TdxBarPopupMenuLink Properties

BarManager	public / read only
Control	published / read / write
PopupMenu	published / read / write

BarManager property

read only

property BarManager: TdxBarManager;

Description

Specifies a TdxBarManager which controls a given menu link.

Control property

read / write

property Control: TWinControl;

Description

Specifies the control associated with a popup menu.

See Also

TdxBarPopupMenuLink.PopupMenu

PopupMenu property

read / write

property PopupMenu: TdxBarPopupMenu;

Description

Specifies the popup menu associated with a control.

See Also

TdxBarPopupMenuLink.Control

TdxBarPopupMenuLink Methods

DoAction **public**

DoAction method

```
function DoAction(P: TPoint): Boolean;
```

Description

Prepares a popup menu to be displayed on screen and generates an OnAction event. The P parameter specifies the coordinates of the leftmost corner of the menu. If DoAction returns True, the popup menu will be displayed on screen, otherwise, the popup menu is hidden.

See Also

TdxBarPopupMenuLink.OnAction

TdxBarPopupMenuLink Events

OnAction

published

OnAction event

This event occurs before a popup menu is displayed on screen.

type

```
TdxBarPopupMenuLinkActionEvent = procedure(Sender: TdxBarPopupMenuLink;  
var X, Y: Integer; ClickedByMouse: Boolean; var AllowPopup: Boolean) of  
object;  
property OnAction: TdxBarPopupMenuLinkActionEvent;
```

Description

Write an OnAction event handler to perform specific actions before a menu is displayed on screen. The Sender parameter points to the menu link for which this event was generated. X and Y determines the coordinates of the leftmost corner of the menu (in control coordinates). AllowPopup determines whether the menu will be displayed on screen. Set AllowPopup to False within the event handler to avoid displaying the menu on screen. ClickedByMouse determines whether the menu is displayed upon a mouse button press event.

TdxBarPopupMenuLinks

Unit

dxBar

```
TdxBarPopupMenuLinks = class(TCollection);
```

Description

Contains a menu of links used to associate controls on a form with TdxBarPopupMenu.

Hierarchy

TObject

TPersistent

TCollection

TdxBarPopupMenuLinks Properties

BarManager	public / read only
Items	public / read / write

BarManager property

read only

property BarManager: TdxBarManager;

Description

Defines a TdxBarManager object which handles a given menu links collection.

Items property

read / write

Includes a list of menu links in a given TdxBarPopupMenuItemLinks object.

property Items[Index: Integer]: TdxBarPopupMenuLink;

Description

Use this property to work with separate menu links. Index of the first menu link in the array is 0.

See Also

TdxBaPopupMenuLinks.Add

TdxBarPopupMenuLinks Methods

Add	public
Create	public
DoAction	public

Add method

```
function Add: TdxBarPopupMenuLink;
```

Description

Adds a new menu link to a TdxBarPopupMenuLinks collection.

Create method

```
constructor Create(ABarManager: TdxBarManager);
```

Description

Creates a new TdxBarPopupMenuLinks object. ABarManager indicates which Bar manager will handle new menu links.

DoAction method

```
function DoAction(AControl: TwinControl; const P: TPoint): Boolean;
```

Description

Called when a user presses the right mouse button on a control. If DoAction returns True, then the popup menu was called for the control. Otherwise, DoAction returns False.

See Also

TdxBarPopupMenuLink.OnAction

Additional Objects

TdxBarConverter

This class is intended for the conversion of TMenu and TMenuItems into TdxBarManager items.

Unit

dxBarConverter

```
TdxBarConverter = class(TComponent);
```

Description

Use TdxBarConverter to convert your existing menus into ExpressBar menus. The TdxBarConverter converts either TMainMenu or TPopupMenu into TdxBarManager items.

Hierarchy

TObject

 TPersistent

 TComponent

TdxBarConverter Properties

BarManager	published / read / write
Execute	published / read / write
Menu	published / read / write

BarManager property

```
property BarManager: TdxBarManager;
```

Description

Specifies the TdxBarManager to which converted menu items belong.

Execute property

```
property Execute: Boolean;
```

Description

Performs the menu conversion process. Click this item within the Object Inspector at design time to convert menus into ExpressBar items.

See Also

TdxBarCoverter.Convert

Menu property

Specifies the menu to be converted.

```
property Menu: TMenu;
```

Description

Menus to be converted may be of type TMenu or TPopupMenu. TMenu items are converted into TdxBarManager items and assigned to a category called 'MainMenu'. Popup menu items are placed into a category with the same name as the popup menu itself.

See Also

TdxBarConverter.Menu; TdxBarConverter.Menu

TdxBarConverter Methods

Convert **public**

Convert method

```
procedure Convert;
```

Description

This method performs menu conversions. TMenu and TPopupMenu items are converted into TdxBarManager items. If a TdxBarManager does not have a toolbar with the IsMainMenu property set to True, Convert automatically creates it.

TdxBarItemActionLink

Specifies the link between an item control and a TAction object.

Unit

dxBar

```
TdxBarItemActionLink = class (TActionLink);
```

Description

The TAction object can be associated with an item by using the action list editor at design-time. Actions are a way of centralizing user responses to commands. The TdxBarItemActionLink class is used to organize appropriate interaction between items and TAction. This class is only available in Delphi 4™.

Hierarchy

TObject

 TBasicActionLink

 TActonLink

TdxBarItemActionLink Methods

AssignClient	protected	SetCaption	protected
IsCaptionLinked	protected	SetChecked	protected
IsCheckedLinked	protected	SetEnabled	protected
IsEnabledLinked	protected	SetHelpContext	protected
IsHelpContextLinked	protected	SetHint	protected
IsHintLinked	protected	SetImageIndex	protected
IsImageIndexLinked	protected	SetShortCut	protected
IsShortCutLinked	protected	SetVisible	protected
IsOnExecuteLinked	protected	SetOnExecute	protected

AssignClient method

Sets the link between TAction and an item.

```
procedure AssignClient(AClient: TObject); override;
```

Description

AClient is an object of type TdxBarItem associated with an action. This method is only available in Delphi 4™.

IsCaptionLinked method

Introduces a method prototype to determine whether the action for caption and an item control are linked.

```
function IsCaptionLinked: Boolean; override;
```

Description

IsCaptionLinked is derived from the TActionLink IsCaptionLinked method. IsCaptionLinked ensures that the action type supports appropriate events for the item. Descendant classes override IsCaptionLinked to determine whether the action for caption is linked with the appropriate item property. Returns True, if TAction controls the caption and the item caption equals the TAction caption. This method is only available in Delphi 4™.

IsCheckedLinked method

Introduces a method to determine whether an action for checked state and the item are linked.

```
function IsCheckedLinked: Boolean; override;
```

Description

IsCheckedLinked is derived from the TActionLink IsCheckedLinked method. Descendant classes override IsCheckedLinked to determine whether an action for checked state is linked with the appropriate item property. IsCheckedLinked returns True, if TAction controls this event, the item is TdxBarButton, and the Checked property of the action equals the TdxBarButton Down property. This method is only available in Delphi 4™.

IsEnabledLinked method

Introduces a method to determine whether an action for enabled state is linked with an item.

```
function IsEnabledLinked: Boolean; override;
```

Description

IsEnabledLinked is derived from the TActionLink IsEnabledLinked method. IsEnabledLinked ensures that the action type supports the appropriate event for an item. Descendant classes override IsEnabledLinked to determine whether the action for enabled state links with the appropriate item property. IsEnabledLinked returns True, if TAction controls this event and the Enabled property of the linked item equals the TAction Enabled property. This method is only available in Delphi 4™.

IsHelpContextLinked method

Introduces a method to determine whether an action for help context is linked with an item.

```
function IsHelpContextLinked: Boolean; override;
```

Description

IsHelpContextLinked is derived from the TActionLink IsHelpContextLinked method. IsHelpContextLinked ensures that action type supports the appropriate event for an item. IsHelpContextLinked returns True, if TAction controls this event and item HelpContext equals the action HelpContext property. This method is only available in Delphi 4™.

IsHintLinked method

Introduces a method prototype to determine whether an action for hint and an item are linked.

```
function IsHintLinked: Boolean; override;
```

Description

IsHintLinked is derived from the TActionLink IsHintLinked method. IsHintLinked ensures that action type supports the appropriate event for an item. IsHintLinked returns True, if TAction controls this event and the item Hint property equals the action Hint property. This method is only available in Delphi 4™.

IsImageIndexLinked method

Introduces a method to determine whether an action for image index and an item are linked.

```
function IsImageIndexLinked: Boolean; override;
```

Description

IsImageIndexHintLinked is derived from the TActionLink IsImageIndexLinked method. IsImageIndexLinked ensures that action type supports the appropriate event for an item. Returns True, if TAction controls this event and the item ImageIndex property equals the action ImageIndex. This method is only available in Delphi 4™.

IsShortcutLinked method

Introduces a method to determine whether an action for shortcut and an item are linked.

```
function IsShortcutLinked: Boolean; override;
```

Description

IsShortcutLinked is derived from the TActionLink IsShortcutLinked method. IsShortcutLinked ensures that action type supports the appropriate event for an item. Returns True, if TAction controls this event. This method is only available in Delphi 4™.

IsOnExecuteLinked method

Indicates whether an action for the OnExecute event handler and an item are linked.

```
function IsOnExecuteLinked: Boolean; override;
```

Description

IsOnExecuteLinked is derived from the TActionLink IsOnExecuteLinked method. IsOnExecuteLinked indicates whether an action for the OnExecute event handler is linked to an OnClick event. IsOnExecuteLinked returns True if TAction controls this event and the OnClick item event handler is the same as an OnExecute action. This method is only available in Delphi 4™.

SetCaption method

Introduces an interface for the action of setting a caption on an item.

```
procedure SetCaption(const Value: string); override;
```

Description

SetCaption is a method which introduces an interface for the action of setting a caption on a linked item. Value specifies a string to which the caption should be set. This method is only available in Delphi 4™.

SetChecked method

Introduces an interface for the action of setting an item's checked state.

```
procedure SetChecked(Value: Boolean); override;
```

Description

SetChecked is a method which introduces an interface for the action of setting an item's checked state via an action link. Value specifies whether the checked state should be set. This method is only available in Delphi 4™.

SetEnabled method

Introduces an interface for the action of setting an item's Enabled state.

```
procedure SetEnabled(Value: Boolean); override;
```

Description

Introduces an interface for the action of setting an item's enabled state via an action link. Value specifies whether the enabled state should be set. This method is only available in Delphi 4™.

SetHelpContext method

Introduces an interface for the action of setting an item's help context.

```
procedure SetHelpContext(Value: THelpContext); override;
```

Description

Introduces an interface for the action of setting the help context of an item via an action link. Value specifies the value to which a help context should be set. This method is available in Delphi 4™ only.

SetHint method

Introduces an interface for the action of setting an item's hint.

```
procedure SetHint(const Value: string); override;
```

Description

Value specifies a hint string to be set.

SetImageIndex method

Introduces an interface for the action of setting the image index for an item control.

```
procedure SetImageIndex(Value: Integer); override;
```

Description

Value specifies the index to set for an item. This method is only available in Delphi 4™.

SetShortCut method

Introduces an interface for the action of setting the shortcut for a linked item.

```
procedure SetShortCut(Value: TShortCut); override;
```

Description

This method is only available in Delphi 4™.

SetVisible method

Introduces an interface for the action of setting an item's visible state.

```
procedure SetVisible(Value: Boolean); override;
```

Description

Value specifies whether the enabled state should be set. This method is only available in Delphi 4™.

SetOnExecute

Sets the execution event handler for an item if the item is linked to an action for this event.

```
procedure SetOnExecute(Value: TNotifyEvent); override;
```

Description

SetOnExecute sets the OnClick event handler of an item. SetOnExecute is called only if the item is linked for this event. Value specifies the procedure that should be used for this event. This method is only available in Delphi 4™.

TdxBarDateNavigator

Provides date selection in a TdxBarDateCombo.

Unit

dxBarExtItems

```
TdxBarCustomCalendar = class(TCustomControl);  
TdxBarDateNavigator = class(TdxBarCustomCalendar);
```

Description

This class represents a dropdown calendar. The calendar appears in the dropdown window of the TdxBarDateCombo. The TdxBarDateNavigator class is used internally.

Hierarchy

TObject

 TPersistent

 TComponent

 TControl

 TWinControl

 TCustomControl

TdxBarDBCheckLink

Defines the link between the item Enabled property and a dataset's state.

Unit

dxBarDBNav

```
TdxBarDBCheckLink = class(TCollectionItem);
```

Description

Use this object to establish a link between a dataset's state and the Enable property of the control on the screen. For example, some controls can be disabled on changes to a dataset from dsEdit to dsBrowse. The TdxBarDBCheckLink object provides just this link.

Note: *You may never need to use objects of this class directly.*

Hierarchy

TObject

TPersistent

TCollectionItem

TdxBarDBCheckLink Properties

Item	published
EnableTypes	published

EnableTypes property

read / write

Determines the conditions that affect the item Enabled property.

property EnableTypes: TdxBarDBEnableTypes;

EnabledTypes defines the type of link between an item and a dataset's state. The item Enabled property is equal to True if a dataset is active and all corresponding EnableTypes conditions are equal to True, otherwise it is False.

See Also

TdxBarDBEnableTypes

Item property

read / write

Specifies the item whose Enabled property is changed in accordance with changes to a dataset's state.

property Item: TdxBarItem;

Use this property to refer to the item whose Enable property changes according to a dataset's state. The Enabled property of all item controls corresponding to this item change when a specified dataset's state changes. For example, your program can make a font combo box disabled when a dataset's state changes from dsEdit to dsBrowse.

TdxBarDBCheckLinks

The TdxBarDBCheckLinks object contains the TdxBarDBCheckLinks list managed by TdxBarDBNavigator.

Unit

dxBarDBNav

```
TdxBarDBCheckLinks = class(TCollection);
```

Description

This list stores information about all links between items and a dataset's state.

Note: *You may never need to use this class.*

Hierarchy

TObject

TPersistent

TCollection

TdxBarDBCheckLinks Properties

Items

public / read only

Items property

read only

Determines an array of links between items and dataset state.

property Items[Index: Integer]: TdxBarDBCheckLink;

Description

This property returns the link of a TdxBarDBCheckLink type stored in a given object by Index. The index of the first TdxBarDBCheckLink is 0.

TdxBarDBCheckLinks Methods

Add	public
GetOwner	protected
RemoveItem	protected

Add method

This method adds a new link to the TdxBarDBCheckLink links list.

```
function Add: TdxBarDBCheckLink;
```

The TdxBarDBCheckLink object describes one link between dataset state and the item Enabled property. The function adds a new blank link to the list and returns it. The new link is placed at the end of the links list.

GetOwner method

```
function GetOwner: TPersistent; override;
```

Returns the TdxBarDBNavigator object that owns a TdxBarDBCheckLink instance. This method is only available in Delphi 3™ or higher.

RemoveItem method

Removes a link between an item and dataset state from the list.

```
procedure RemoveItem(Item: TdxBarItem);
```

This method removes the link between an item and dataset state from the TdxBarDBCheckLinks list. The Item parameter specifies the item.

TdxBarTreeView

Represents a tree view in the dropdown window.

Unit

dxBarExtItems

```
TdxBarTreeView = class(TCustomTreeView);
```

Description

This class is used to represent a tree view in the dropdown window of TdxBarTreeViewCombo. TdxBarTreeView derives the TCustomTreeView methods and properties. It provides the proper tree view interface for the dropdown tree.

Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomTreeView

TdxBarLookupLink

Unit

dxBarExtDBItems

```
TdxBarLookupLink = class(TDataLink);
```

Description

TdxBarLookupLink is a class used by TdxBarLookupCombo. It uses TdxBarLookupLink to coordinate TDataSource and TDataSet actions and to respond to data events.

This class is used internally.

Hierarchy

TObject

TPersistent

TDataLink

TdxBarPopupLookupControl

Unit

dxBarExtDBItems

```
TdxBarPopupLookupControl = class(TCustomControl);
```

Description

Represents a dropdown window of the TdxBarLookupCombo item. Use the TdxBarLookupCombo ListField property to specify the fields whose values are displayed in a dropdown list. The number of visible rows in the dropdown list is specified by the TdxBarLookupCombo RowCount property.

TdxBarPopupLookupControl is for internal use only.

Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomControl

Global Types, Procedures and Variables

ActiveBarControl procedure

Unit

dxBar

```
function ActiveBarControl: TCustomdxBarControl;
```

Description

This function returns the active bar control (the toolbar or submenu that receives keyboard messages). ActiveBarControl returns nil if there is no such object.

dxBarDBNavigatorList variable

Unit

dxBarDBNav

Description

This is a global variable of the TdxBarDBNavigatorList type. It contains the list of all TdxBarDBNavigator application objects. Such a list is useful when it is necessary to work with several TdxBarDBNavigator objects.

dxBarCustomizingForm variable

Unit

dxBarCustForm

type

```
TdxBarCustomizingForm = class(TForm);
```

```
function dxBarCustomizingForm: TdxBarCustomizingForm;
```

Description

This function returns a customizing form or nil if there is no such a form. You can discover which TdxBarManager is currently in customizing mode by reading the CustomizingBarManager property of the TdxBarManagerList object. Use the dxBarManagerList variable to get the list of all available TdxBarManager objects.

See Also

dxBarManagerList; TdxBarManagerList; TdxBarManagerList.CustomizingBarManager

dxBarHintWindowClass variable

Unit

dxBar

type

```
TdxBarHintWindowClass = class of TdxBarHintWindow;  
dxBarHintWindowClass: TdxBarHintWindowClass;
```

Description

This global variable is designed to specify the window class used to display hints. By default, the dxBarHintWindowClass refers to the TdxBarHintWindow class.

See Also

TdxBarHintWindow

dxBarRegisterItem procedure

Registers an item class.

Unit

dxBar

type

```
TdxBarItemClass = class of TdxBarItem;  
TdxBarItemControlClass = class of TdxBarItemControl;  
procedure dxBarRegisterItem(AItemClass: TdxBarItemClass;  
    AItemControlClass: TdxBarItemControlClass; AVisible: Boolean);
```

Description

This procedure is necessary when creating custom items. Use dxBarRegisterItem to register your own item classes. dxBarRegisterItem links an item control class, specified by the AItemControlClass to the item class, specified by the AItemClass. If AVisible is True, the new item will be visible in the TdxBarManager list of registered items; Else, the item is not visible in the list, but you can still create it at run time and use it as any other item.

See Also

dxBarUnregisterItem; TdxBarItem

dxBarUnregisterItem procedure

Unregisters an item class.

Unit

dxBar

type

TdxBarItemClass = class of TdxBarItem;

procedure dxBarUnregisterItem(AItemClass: TdxBarItemClass);

Description

Use this procedure to unregister an item class. The AItemClass parameter specifies the item class to unregister. After unregistering, you can no longer work with the item.

See Also

dxBarRegisterItem; TdxBarItem

GetBarManagerByForm procedure

Unit

dxBar

function GetBarManagerByForm(AForm: TCustomForm): TdxBarManager;

Description

This function returns the TdxBarManager type object belonging to the form specified by the AForm parameter or nil if there is no such object. Use the dxBarManagerList variable to get the list of all available TdxBarManager objects of an application.

See Also

dxBarManagerList

IsVertical procedure

Determines whether a bar control has a vertical layout.

Unit

dxBar

```
function IsVertical(ABarControl: TCustomdxBarControl): Boolean;
```

Description

This function returns True, if DockingStyle of the bar control, specified by the ABarControl parameter, equals dsLeft or dsRight, otherwise it is False.

See Also

TCustomdxBarControl; TdxBar.DockingStyle

dxBarColorDialog function

Unit

dxBarExtItems

```
function dxBarColorDialog: TColorDialog;
```

Description

The dxBarColorDialog function returns the color dialog object that is used by a color combo item to pick an appropriate color.

dxBarFontDialog function

Unit

dxBarExtItems

```
function dxBarFontDialog: TFontDialog;
```

Description

The dxBarFontDialog function returns the font dialog object that is used by a font name combo item to pick an appropriate font.

TdxBarDBNavButtonType

This class defines TdxBarDBNavigator button types.

Unit

dxBarDBNav

type

```
TdxBarDBNavButtonType = (dxbnFirst, dxbnPrior, dxbnNext, dxbnLast,
dxbnInsert, dxbnDelete, dxbnEdit, dxbnPost, dxbnCancel, dxbnRefresh);
TdxBarDBNavButtonTypes = set of TdxBarDBNavButtonType;
```

Description

You can customize a set of the TdxBarDBNavigator buttons to arrive at a collection appropriate to your applications. TdxBarDBNavigator objects provide buttons similar to those of TDBNavigator. TdxBarDBNavButtonTypes includes the following:

Value	Meaning
dxbnFirst	Corresponds to the 'First' button. Pressing this button moves the cursor to the beginning of a dataset.
dxbnPrior	Corresponds to the 'Prior' button. Pressing this button moves the cursor to the previous record.
dxbnNext	Corresponds to the 'Next' button. Pressing this button moves the cursor to the next record.
dxbnLast	Corresponds to the 'Last' button. Pressing this button moves the cursor to the end of a dataset.
dxbnInsert	Corresponds to the 'Insert' button. Pressing this button inserts a record into the dataset.
dxbnDelete	Corresponds to the 'Delete' button. Pressing this button deletes a record from the dataset.
dxbnEdit	Corresponds to the 'Edit' button. Pressing this button allows editing within a dataset.
dxbnPost	Corresponds to the 'Post' button. Pressing this button posts changes to a dataset.
dxbnCancel	Corresponds to the 'Cancel' button. Pressing this button discards changes made to a dataset.
dxbnRefresh	Corresponds to the 'Refresh' button. Pressing this button refreshes a dataset's view of data.

TdxBarDBEnableType

Unit

dxBarDBNav

type

```
TdxBarDBEnableType = (dxdbtCanModify, dxdbtNotEOF, dxdbtNotBOF,  
dxdbtHasRecords, dxdbtIsModified, dxdbtIsNotModified);
```

```
TdxBarDBEnableTypes = set of TdxBarDBEnableType;
```

Description

The TdxBarDBEnableType defines the types of changes to a dataset's state that affect the items Enabled property via the TdxBarDBCheckLink object. The item is enabled, while all specified types correspond to dataset state. The following values are available:

Value	Meaning
dxdbtCanModify	If set, the Enabled property is True when the application can modify data in a dataset.
dxdbtNotEOF	If set, the Enabled property is True when the cursor has not reached the end of the table.
dxdbtNotBOF	If set, the Enabled property is True when the cursor has not reached the beginning of the table.
dxdbtHasRecords	If set, the Enabled property is True if the dataset is not empty.
dxdbtIsModified	If set, the Enabled property is True when a dataset's state is dsEdit.
dxdbtIsNotModified	If set, the Enabled property is True when a dataset's state is dsBrowse.

TdxBarHintWindow

Unit

dxBar

```
TdxBarHintWindow = class(TCustomControl);
```

Description

This class is a window used to display hints. Your application can use its own window class to display hints based upon this class.

Hierarchy

TObject

TPersistent

TComponent

TControl

TWinControl

TCustomControl

See Also

dxBarHintWindowClass

TdxBarHintWindow Methods

ActivateHint	public
Paint	protected

ActivateHint method

```
procedure ActivateHint(P: TPoint; const AHint: string; BarManager:
TdxBarManager); virtual;
```

Description

This procedure is called before displaying a hint window for an item handled by a BarManager. The P parameter determines the coordinates of the top left corner of the hint window. AHint is the string used as a hint.

Paint method

```
procedure Paint; override;
```

Description

This procedure draws a hint window on screen.

Index

A

ActivateHint.....	204
ActiveBarControl.....	198
Add.....	77, 96, 105, 179, 194
AdjustSize	138
Align.....	122
AlignControls	112
AllowCallFromAnotherForm	11
AllowClose	56
AllowCustomizing	57
AllowQuickCustomizing	57
AllowReset.....	11, 58
AllowZeroSizeInDesignTime	122
AlwaysMerge	11
AlwaysSaveText	12
AssignClient	184
AutoDockColor	12
AutoHideEmptyBars	12
AvailableIndex.....	79
AvailableIndexOf.....	96
AvailableItemCount	92
AvailableItems	91

B

Bar	127
BarByCaption.....	34
BarByName	34
BarControl.....	79, 92, 114
BarControlFocused.....	13
BarDBNavigator	161
BarGetFocus.....	129
BarLostFocus.....	129
BarManager	58, 75, 80, 92, 102, 108, 133, 153, 167, 174, 178, 181
BarManagers	54
Bars.....	58, 108
BarSize	167
BeforeFingersSize	13
BeginGroup.....	80
BkBrush.....	133
BorderStyle	59
BringToTopInRecentList.....	88
ButtonArrowWidth.....	13
ButtonHeight	13
Buttons.....	153
ButtonWidth	14

C

CalcDrawingConsts	138
CanCustomize	14
CanVisibleIndex.....	80
CanVisibleIndexOf	96
CanVisibleItemCount.....	93
CanVisibleItems	93

Caption.....	59, 80
CaptionChanged.....	130
Categories.....	14
CategoryItemsVisible	15
CategoryName	154
CategoryVisible.....	15
ChangeDockingStyle	72
ChildBar	133
ColCount	118
ColList.....	118
Color.....	123
Cols.....	119
ComboBoxArrowWidth	134
ConfirmDelete	154
Control	59, 81, 174
Convert	182
Count.....	54, 102, 158
Create ...	35, 72, 77, 88, 97, 115, 120, 138, 156, 169, 179
CreateBarControl.....	97
CreateControls	139
CreateEx	112, 130
CreateGroup.....	35
CreateToolbarsPopupList.....	35
Customizing.....	36
CustomizingBarManager.....	54

D

Data	81
DataSource	154
DBCheckLinks	154
Delete	105
Designing	15
Destroy.....	36, 72, 77, 88, 97, 120, 139, 156, 163, 169
DestroyControls	139
DoAction	175, 179
DoBarAfterReset.....	36
DoBarBeforeReset.....	36
DoBarClose.....	37
DoBarDockingStyleChanged.....	37
DockColor	16
DockControl	60, 117
DockControlCount.....	16
DockControls	16, 75
DockedDockControl.....	60
DockedDockingStyle	60
DockedLeft	61
DockedTop.....	61
DockingStyle	61, 108, 134
DockingZoneSize	75
DockRow.....	114
DoClick.....	164
DoClickItem.....	37
DoCloseButtonClick.....	37
DoCloseUp.....	148
DoDocking.....	38

DoHelpButtonClick	38
DoPopup	148
DoShowCustomizingPopup	38
DoShowToolbarsPopup	39
DownArrowExists	149
Dragging	17
DraggingItem	17
DraggingItemLink	17
dxBarColorDialog	201
dxBarCustomizingForm	198
dxBarDBNavigatorList	198
dxBarFontDialog	201
dxBarHintWindowClass	199
dxBarRegisterItem	199
dxBarUnregisterItem	200

E

EditFontHandle	134
EditTextSize	134
Enabled	102
EnableTypes	191
ExchangeItems	39
Execute	181
ExpressBars Overview	6

F

FingersSize	17
FlatCloseButton	18
FloatClientHeight	62
FloatClientWidth	62
FloatLeft	62
FloatTop	62
FocusedBarControl	18
Font	18, 63, 168

G

GetAllItemsByCategory	39
GetBarManagerByForm	200
GetCountByCategory	40
GetDBNavigatorByBarManager	159
GetFirstFreeBarManager	159
GetFullItemRect	140
GetItemByCategory	40
GetItemByName	40
GetItemRectEx	140
GetItemRegion	141
GetItemsByCategory	41
GetOwner	194
GetUniqueToolbarName	77
Glyph	81
GlyphSize	18
GroupCount	19
Groups	19

H

HasItem	97
HelpButtonGlyph	19
HelpContext	20
Hidden	63
Hide	149

HideAll	41, 141
HideOnClick	141
HotImages	20

I

ImageListBkColor	20
Images	21
Index	102
IndexOf	98, 105
IniFileName	21
IsActive	134
IsCaptionLinked	184
IsCheckedLinked	185
IsChildBar	142
IsCustomizing	21, 135
IsDowned	127
IsEnabledLinked	185
IsHelpContextLinked	185
IsHintLinked	185
IsImageIndexLinked	186
IsMainMenu	63
IsOnExecuteLinked	186
IsPredefined	64
IsShortCutLinked	186
IsVertical	201
Item	6, 7, 81, 191
Item control	7
ItemAtPos	141
ItemAtPosEx	130, 142
ItemCount	22
ItemLinks	64, 135, 168
ItemRect	82
Items	22, 76, 94, 103, 158, 178, 193

L

LargeButtonArrowWidth	22
LargeIcons	22
LargeImages	23
LoadFromIniFile	41
LoadFromRegistry	42
LockUpdate	23, 65

M

MainForm	23
MainMenuBar	24
MainMenuControl	24
MarkSize	146
MarkState	136
MaxVisibleCount	146
MDIButtonHeight	128
MDIButtonWidth	128
Menu	181
MenuAnimations	24
MenuArrowWidth	136
MenusShowRecentItemsFirst	25
Modified	25
MostRecentItemsPercents	25
MostRecentlyUsed	82
Move	106
MoveItem	42

Moving..... 128
MultiLine.....64

N

Name64
NavButton..... 162
NotDocking.....26, 65
NotHandleMouse..... 142
NotHandleMouseMove..... 142

O

OnAction..... 176
OnBarAdd.....45
OnBarAfterReset.....45
OnBarBeforeReset.....46
OnBarClose.....46
OnBarDelete46
OnBarDockingStyleChange.....47
OnBarVisibleChange47
OnChange.....99
OnClickItem47
OnCloseButtonClick.....48
OnCloseUp 151, 171
OnDocking48
OneOnRow66
OnHelpButtonClick.....49
OnHideCustomizingForm.....49
OnItemLinkAdd.....49
OnItemLinkChange.....50
OnItemLinkDelete50
OnMenuMerge.....51
OnPaintBar..... 172
OnPopup..... 151, 172
OnShowCustomizingForm.....51
OnShowCustomizingPopup.....51
OnShowToolbarsPopup.....52
Owner.....94
OwnerHeight 147
OwnerWidth 147

P

Paint..... 204
PaintItem 130, 142
PaintSelectedItem..... 143
PaintStyle83
ParentBar 136
ParentColor 123
Popup..... 169
PopupEx..... 170
PopupFromCursorPos..... 170
PopupMenu..... 174
PopupMenuLinks26
Pos..... 114

R

RealButtonArrowWidth.....26
RealDockControl66
RealItemLink.....83

RealVisibleItemCount.....94
RegistryPath.....27
Remove 106
RemoveItem..... 194
Repaint..... 131
RepaintBar 143
Reset.....73
ResetUsageData42
ResetUsageDataWithConfirmation42
ResetWithConfirmation.....73
RotateWhenVertical.....67
Row67
RowCount 109
RowHeight83
RowList..... 110
Rows..... 111

S

SaveToIniFile43
SaveToRegistry44
Scaled27
SelectedItem.....27, 137
SendToBottomInRecentList.....89
SetCaption 186
SetChecked 187
SetEnabled 187
SetHelpContext 187
SetHint 187
SetImageIndex..... 188
SetKeySelectedItem..... 143
SetMouseSelectedItem..... 144
SetOnExecute 188
SetShortCut 188
SetVisible 188
Show..... 149
ShowCloseButton28
ShowFullMenusAfterDelay28
ShowHelpButton.....28
ShowHint.....29
ShowHintForDisabledItems29
ShowMark68
ShowShortCutInHint29
SizeGrip68
StoreInIniFile29
StoreInRegistry30
StretchGlyphs30
Style.....31
SubMenuBeginGroupIndent.....32
SunkenBorder32, 123

T

TCustomdxBarControl..... 132
AdjustSize 138
BarManager..... 133
BkBrush..... 133
CalcDrawingConsts..... 138
ChildBar 133
ComboBoxArrowWidth..... 134

Create	138	ItemLinks	64
CreateControls	139	LockUpdate.....	65
Destroy.....	139	MultiLine.....	64
DestroyControls	139	Name	64
DockingStyle	134	NotDocking.....	65
EditFontHandle	134	OneOnRow	66
EditTextSize	134	RealDockControl	66
GetFullItemRect.....	140	Reset.....	73
GetItemRectEx	140	ResetWithConfirmation	73
GetItemRegion	141	RotateWhenVertical.....	67
HideAll	141	Row	67
HideOnClick.....	141	ShowMark	68
IsActive.....	134	SizeGrip	68
IsChildBar.....	142	UseOwnFont	69
IsCustomizing.....	135	UseRecentItems	69
ItemAtPos.....	141	UseRestSpace.....	69
ItemAtPosEx.....	142	Visible	70
ItemLinks	135	WholeRow	71
MarkState	136	TdxBar Methods.....	72
MenuArrowWidth.....	136	TdxBar Properties	56
NotHandleMouse.....	142	TdxBarControl.....	126
PaintItem	142	Bar	127
PaintSelectedItem	143	BarGetFocus.....	129
ParentBar	136	BarLostFocus.....	129
RepaintBar	143	CaptionChanged.....	130
SelectedItem.....	137	CreateEx	130
SetKeySelectedItem.....	143	IsDowned	127
SetMouseSelectedItem.....	144	ItemAtPosEx.....	130
TextSize	137	MDIButtonHeight.....	128
TCustomdxBarControl Methods.....	138	MDIButtonWidth.....	128
TCustomdxBarControl Properties	133	Moving.....	128
TdxBaPopupMenuLinks		PaintItem.....	130
Add.....	179	Repaint	131
TdxBar	55	TdxBarControl Methods.....	129
AllowClose.....	56	TdxBarControl Properties.....	127
AllowCustomizing.....	57	TdxBarConverter.....	180
AllowQuickCustomizing.....	57	BarManager	181
AllowReset.....	58	Convert	182
BarManager	58	Execute	181
Bars.....	58	Menu	181
BorderStyle	59	TdxBarDateNavigator.....	189
Caption.....	59	TdxBarDBCheckLink.....	190
ChangeDockingStyle	72	EnableTypes	191
Control.....	59	Item.....	191
Create	72	TdxBarDBCheckLinks.....	192
Destroy.....	72	Add.....	194
DockControl	60	GetOwner	194
DockedDockControl.....	60	Items	193
DockedDockingStyle	60	RemoveItem.....	194
DockedLeft	61	TdxBarDBEnableType	203
DockedTop.....	61	TdxBarDBNavButton.....	160
DockingStyle	61	BarDBNavigator.....	161
FloatClientHeight.....	62	Destroy.....	163
FloatClientWidth.....	62	DoClick.....	164
FloatLeft	62	NavButton	162
FloatTop	62	TdxBarDBNavButtonType.....	202
Font.....	63	TdxBarDBNavigator	152
Hidden.....	63	BarManager	153
IsMainMenu.....	63	Buttons.....	153
IsPredefined	64	CategoryName	154

ConfirmDelete.....	154	TdxBarItemActionLink Methods.....	184
Create.....	156	TdxBarItemLink.....	78
DataSource.....	154	AvailableIndex.....	79
DBCheckLinks.....	154	BarControl.....	79
Destroy.....	156	BarManager.....	80
VisibleButtons.....	155	BeginGroup.....	80
TdxBarDBNavigatorList.....	157	BringToTopInRecentList.....	88
Count.....	158	CanVisibleIndex.....	80
GetDBNavigatorByBarManager.....	159	Caption.....	80
GetFirstFreeBarManager.....	159	Control.....	81
Items.....	158	Create.....	88
TdxBarDockControl.....	121	Data.....	81
Align.....	122	Destroy.....	88
AllowZeroSizeInDesignTime.....	122	Glyph.....	81
Color.....	123	Item.....	81
ParentColor.....	123	ItemRect.....	82
SunkenBorder.....	123	MostRecentlyUsed.....	82
UseOwnColor.....	123	PaintStyle.....	83
UseOwnSunkenBorder.....	124	RealItemLink.....	83
TdxBarDockControl Methods.....	125	RowHeight.....	83
TdxBarDockControl Properties.....	122	SendToBottomInRecentList.....	89
TdxBarGroup.....	100	UserCaption.....	84
Add.....	105	UserDefine.....	84
BarManager.....	102	UserGlyph.....	85
Count.....	102	UserPaintStyle.....	86
Delete.....	105	UserWidth.....	86
Enabled.....	102	Visible.....	87
Index.....	102	VisibleIndex.....	87
IndexOf.....	105	Width.....	87
Items.....	103	TdxBarItemLink Methods.....	88
Move.....	106	TdxBarItemLink Properties.....	79
Remove.....	106	TdxBarItemLinks.....	90
Visible.....	104	Add.....	96
TdxBarGroup Methods.....	105	AvailableIndexOf.....	96
TdxBarGroup Properties.....	102	AvailableItemCount.....	92
TdxBarHintWindow.....	204	AvailableItems.....	91
ActivateHint.....	204	BarControl.....	92
Paint.....	204	BarManager.....	92
TdxBarHintWindow Methods.....	204	CanVisibleIndexOf.....	96
TdxBarItemActionLink.....	183	CanVisibleItemCount.....	93
AssignClient.....	184	CanVisibleItems.....	93
IsCaptionLinked.....	184	Create.....	97
IsCheckedLinked.....	185	CreateBarControl.....	97
IsEnabledLinked.....	185	Destroy.....	97
IsHelpContextLinked.....	185	HasItem.....	97
IsHintLinked.....	185	IndexOf.....	98
IsImageIndexLinked.....	186	Items.....	94
IsOnExecuteLinked.....	186	OnChange.....	99
IsShortCutLinked.....	186	Owner.....	94
SetCaption.....	186	RealVisibleItemCount.....	94
SetChecked.....	187	Update.....	98
SetEnabled.....	187	VisibleIndexOf.....	98
SetHelpContext.....	187	VisibleItemCount.....	95
SetHint.....	187	VisibleItems.....	95
SetImageIndex.....	188	TdxBarItemLinks Events.....	99
SetOnExecute.....	188	TdxBarItemLinks Methods.....	96
SetShortCut.....	188	TdxBarItemLinks Properties.....	91
SetVisible.....	188	TdxBarLookupLink.....	196

TdxBarManager.....	9	IsCustomizing.....	21
AllowCallFromAnotherForm.....	11	ItemCount.....	22
AllowReset.....	11	Items.....	22
AlwaysMerge.....	11	LargeButtonArrowWidth.....	22
AlwaysSaveText.....	12	LargeIcons.....	22
AutoDockColor.....	12	LargeImages.....	23
AutoHideEmptyBars.....	12	LoadFromIniFile.....	41
BarByCaption.....	34	LoadFromRegistry.....	42
BarByName.....	34	LockUpdate.....	23
BarControlFocused.....	13	MainForm.....	23
BeforeFingersSize.....	13	MainMenuBar.....	24
ButtonArrowWidth.....	13	MainMenuControl.....	24
ButtonHeight.....	13	MenuAnimations.....	24
ButtonWidth.....	14	MenusShowRecentItemsFirst.....	25
CanCustomize.....	14	Modified.....	25
Categories.....	14	MostRecentItemsPercents.....	25
CategoryItemsVisible.....	15	MoveItem.....	42
CategoryVisible.....	15	NotDocking.....	26
Create.....	35	OnBarAdd.....	45
CreateGroup.....	35	OnBarAfterReset.....	45
CreateToolbarsPopupList.....	35	OnBarBeforeReset.....	46
Customizing.....	36	OnBarClose.....	46
Designing.....	15	OnBarDelete.....	46
Destroy.....	36	OnBarDockingStyleChange.....	47
DoBarAfterReset.....	36	OnBarVisibleChange.....	47
DoBarBeforeReset.....	36	OnClickItem.....	47
DoBarClose.....	37	OnCloseButtonClick.....	48
DoBarDockingStyleChanged.....	37	OnDocking.....	48
DockColor.....	16	OnHelpButtonClick.....	49
DockControlCount.....	16	OnHideCustomizingForm.....	49
DockControls.....	16	OnItemLinkAdd.....	49
DoClickItem.....	37	OnItemLinkChange.....	50
DoCloseButtonClick.....	37	OnItemLinkDelete.....	50
DoDocking.....	38	OnMenuMerge.....	51
DoHelpButtonClick.....	38	OnShowCustomizingForm.....	51
DoShowCustomizingPopup.....	38	OnShowCustomizingPopup.....	51
DoShowToolbarsPopup.....	39	OnShowToolbarsPopup.....	52
Dragging.....	17	PopupMenuLinks.....	26
DraggingItem.....	17	RealButtonArrowWidth.....	26
DraggingItemLink.....	17	RegistryPath.....	27
ExchangeItems.....	39	ResetUsageData.....	42
FingersSize.....	17	ResetUsageDataWithConfirmation.....	42
FlatCloseButton.....	18	SaveToIniFile.....	43
FocusedBarControl.....	18	SaveToRegistry.....	44
Font.....	18	Scaled.....	27
GetAllItemsByCategory.....	39	SelectedItem.....	27
GetCountByCategory.....	40	ShowCloseButton.....	28
GetItemByCategory.....	40	ShowFullMenusAfterDelay.....	28
GetItemByName.....	40	ShowHelpButton.....	28
GetItemsByCategory.....	41	ShowHint.....	29
GlyphSize.....	18	ShowHintForDisabledItems.....	29
GroupCount.....	19	ShowShortCutInHint.....	29
Groups.....	19	StoreInIniFile.....	29
HelpButtonGlyph.....	19	StoreInRegistry.....	30
HelpContext.....	20	StretchGlyphs.....	30
HideAll.....	41	Style.....	31
HotImages.....	20	SubMenuBeginGroupIndent.....	32
ImageListBkColor.....	20	SunkenBorder.....	32
Images.....	21	UseF10ForMenu.....	32
IniFileName.....	21	UseFullReset.....	32

UseSystemFont.....	33	DoPopup.....	148
WaitForDockingTime.....	33	DownArrowExists	149
TdxBarManager Events.....	45	Hide.....	149
TdxBarManager Methods.....	34	MarkSize	146
TdxBarManager Properties	10	MaxVisibleCount	146
TdxBarManagerList.....	53	OnCloseUp	151
BarManagers	54	OnPopup.....	151
Count	54	OwnerHeight.....	147
CustomizingBarManager	54	OwnerWidth.....	147
TdxBarManagerList Properties	54	Show	149
TdxBarPopupLookupControl	197	TopIndex.....	147
TdxBarPopupMenu.....	166	UpArrowExists	150
BarManager	167	VisibleCount.....	150
BarSize	167	TdxBarSubMenuControl Events.....	151
Create	169	TdxBarSubMenuControl Methods.....	148
Destroy	169	TdxBarSubMenuControl Properties	146
Font.....	168	TdxBarTreeView.....	195
ItemLinks	168	TdxDockCol.....	113
OnCloseUp	171	BarControl	114
OnPaintBar	172	Create.....	115
OnPopup.....	172	DockRow	114
Popup.....	169	Pos.....	114
PopupEx.....	170	TdxDockCol Methods.....	115
PopupFromCursorPos.....	170	TdxDockCol Properties	114
UseOwnFont.....	168	TdxDockControl	107
UseRecentItems	168	AlignControls	112
TdxBarPopupMenu Events.....	171	BarManager.....	108
TdxBarPopupMenu Methods.....	169	Bars	108
TdxBarPopupMenu Properties	167	CreateEx.....	112
TdxBarPopupMenuLink.....	173	DockingStyle	108
BarManager.....	174	RowCount.....	109
Control.....	174	RowList.....	110
DoAction.....	175	Rows	111
OnAction.....	176	TdxDockControl Methods.....	112
PopupMenu.....	174	TdxDockControl Properties	108
TdxBarPopupMenuLink Events.....	176	TdxDockRow	116
TdxBarPopupMenuLink Methods.....	175	ColCount.....	118
TdxBarPopupMenuLink Properties	174	ColList.....	118
TdxBarPopupMenuLinks	177	Cols	119
BarManager.....	178	Create.....	120
Create	179	Destroy.....	120
DoAction.....	179	DockControl	117
Items	178	TdxDockRow Methods.....	120
TdxBarPopupMenuLinks Methods.....	179	TdxDockRow Properties	117
TdxBarPopupMenuLinks Properties	178	TextSize	137
TdxBars	74	TopIndex	147
Add.....	77		
BarManager.....	75		
Create	77		
Destroy	77		
DockControls	75		
DockingZoneSize	75		
GetUniqueToolbarName	77		
Items	76		
TdxBars Methods.....	77		
TdxBars Properties	75		
TdxBarSubMenuControl	145		
DoCloseUp	148		

U

UpArrowExists	150
Update.....	98
UseF10ForMenu	32
UseFullReset.....	32
UseOwnColor.....	123
UseOwnFont.....	69, 168
UseOwnSunkenBorder	124
UserCaption	84
UserDefine	84
UseRecentItems	69, 168

UseRestSpace.....	69
UserGlyph	85
UserPaintStyle	86
UserWidth	86
UseSystemFont.....	33

V

Visible	70, 87, 104
VisibleButtons	155
VisibleCount	150

VisibleIndex.....	87
VisibleIndexOf	98
VisibleItemCount.....	95
VisibleItems	95

W

WaitForDockingTime	33
WholeRow.....	71
Width.....	87